

滤波设备性能退化建模与维护策略优化

摘要

针对十台滤波设备的性能退化与维护决策，本文将不规则高频记录聚合为日级面板，以“绩效解释—寿命情景—策略评价—价格压力测试”为统一主线。

问题一中，用加权面板回归分离设备差异、年度周期和维护后污染累积，再通过维护事件窗口与设备簇自助法估计恢复效果。完整模型 $R^2 = 0.8601$ ；中维护的即时恢复量和 8—30 日保持量中位数分别为 17.16 和 11.86，大维护分别为 15.41 和 9.17。

问题二中，建立长期退化率非负、设备间部分汇聚的分层状态模型，并以多起点时间留出检验其外推能力。在现行维护规律和结构扰动情景下，2,000 条路径给出的总寿命中位数为 5.63—14.15 年。由于观测期仅约两年且无真实失效，该结果定义为联合判据下的情景外推，不视为精确报废日期。

问题三至四中，对 129 组周期/状态触发规则进行路径成本评价，再对购置价和维护价进行线性重定价。逐设备精细化方案的估计年均成本为 603.41 万元/年；考虑执行简洁性与模型风险，本文主推全厂“每年 1 次大维护、不额外中维护”，其成本 607.86 万元/年，仅比精细化方案高 0.74%，且在 288/289 个共同价格场景中仍为最佳统一规则。

关键字： 性能退化 状态空间模型 寿命预测 维护优化 敏感性分析

一、问题重述

1.1 问题背景

过滤设备投入运行后，透水率会同时受到季节环境、污染累积、自然老化和维护操作影响。题目给出十台设备约两年的高频性能记录以及中维护、大维护日期，需要在区分这些作用的基础上预测寿命并制定经济可行的维护策略。

1.2 问题提出

本文需要解决以下问题：

- **问题一：**处理监测数据，识别周期、设备趋势和不同维护类型的恢复及保持效果；
- **问题二：**在现行维护规律延续的条件下预测十台设备寿命及不确定区间；
- **问题三：**在透水率约束下设计使全生命周期年均成本最小的维护方案；
- **问题四：**分析购置价和维护价格变化时最优方案的稳定区域与调整边界。

二、问题分析

原始数据同时具有不规则采样、年度周期、设备异质性和维护跳变。本文先建立等权日级面板，再以问题一模型识别季节和维护周期；随后将其转化为可向前模拟的退化状态模型，用同一状态转移完成寿命预测、维护优化和价格敏感性分析。

2.1 问题一的分析

若直接比较维护前后均值，季节回升和维护触发时的低状态会造成混杂。故问题一先控制设备固定效应、设备线性趋势与公共季节项，再在残差序列上构造维护前 7 日、维护后 1—7 日和维护后 8—30 日事件窗口。原始极值不直接删除，而是在完整模型残差上用设备内 MAD 规则标记。

2.2 问题二的分析

问题一中的设备线性趋势从负到正差异很大，若直接外推，会得到部分设备永久改善或过早失效的不合理结论。问题二因此把观测期短期漂移限制为有界项，把真正用于寿命外推的长期退化率约束为非负并在设备间部分汇聚。模型选择只使用按时间排序的留交集；寿命须等滚动年均跌破阈值后，再由后续大维护的 8—30 日表现确认。

2.3 问题三的分析

问题二对中、大维护共用一个污染重置，若直接优化会机械偏向便宜动作。因此先用事件后 1—7 日和 8—30 日增益建立分类响应，对小样本大维护部分汇聚；再枚举周期与状态触发规则，以路径级全生命周期年均成本的期望为目标。阈值下穿后强制诊断性大维护，避免模型通过“永不大维护”人为延长寿命。

2.4 问题四的分析

价格不进入退化状态转移，因此既定方案的寿命与运行约束不会随价格改变。问题四先把问题三路径成本拆成三个价格线性系数，再在价格网格上重定价和重新选优；分别用“方案是否完全相同”“是否在最优成本 1% 内”和“后悔值”判断经济稳定性，并求跨场景最小最大后悔方案。

三、 模型假设

- 1. 假设题目数据的设备编号、日期和维护类型无系统性错位；随机测量波动由日中位数、质量权重与残差扰动吸收；
- 2. 假设除维护事件外，设备没有发生附件中未记录的结构性改造或更换；
- 3. 假设年度环境变化形成可重复的季节项，设备不可逆健康状态在日尺度连续变化；
- 4. 题目没有记录的小维护视为费用可忽略的固定背景过程，不作为免费高频优化动作。

四、 符号说明

符号	意义
$y_{i,t}$	设备 i 在第 t 日的观测性能
$H_{i,t}$	设备不可逆健康状态
$F_{i,t}$	维护后重新累积的可恢复污染状态
$J_{i,t}$	中维护或大维护引起的状态跳变
L_i	设备 i 的预测寿命（天）

表 1 符号说明

注：其余符号在首次出现处说明。

五、 问题一模型的建立与求解

5.1 模型建立

将不规则高频数据聚合为日级面板，主响应量取日中位数，保留每日有效样本数、缺失率与插值标记。每天达到 4 个有效样本后权重不再增加，避免小时采样阶段获得过高权重。

对设备 i 在日期 t 的透水率建立加权面板模型：

$$y_{i,t} = \alpha_i + \beta_i T_t + \sum_{k=1}^2 (a_k \sin \frac{2\pi kt}{365.25} + b_k \cos \frac{2\pi kt}{365.25}) + \gamma_0 I_{i,t} + \gamma_1 g_1(u_{i,t}) + \gamma_2 g_2(u_{i,t}) + \epsilon_{i,t}$$

其中 α_i 为设备固定效应, β_i 是观测期调整后线性趋势; $u_{i,t}$ 为距最近维护天数, g_1, g_2 分别描述 0—30 日和 30—90 日污染累积。标准误按设备聚类。

5.2 模型求解

主规格用 1 年和半年 Fourier 项表示长周期季节变化^[1], 并用月份固定效应作为稳健性对照。维护效果先减去设备趋势和公共季节预测, 再计算维护前后差; 95% 区间通过按设备整簇重采样 2,000 次得到^[2]。模型拟合后, 在每台设备内以 MAD 稳健尺度标记绝对稳健 Z 分数大于 3.5 的残差异常日, 但不从主分析中直接删除。

5.3 结果分析

十台设备日序列及维护位置如图 1 所示。不同设备的长期方向并不完全一致, 且维护带来的阶跃与年度变化幅度相近, 因此不能用一条总体直线代替分解模型。

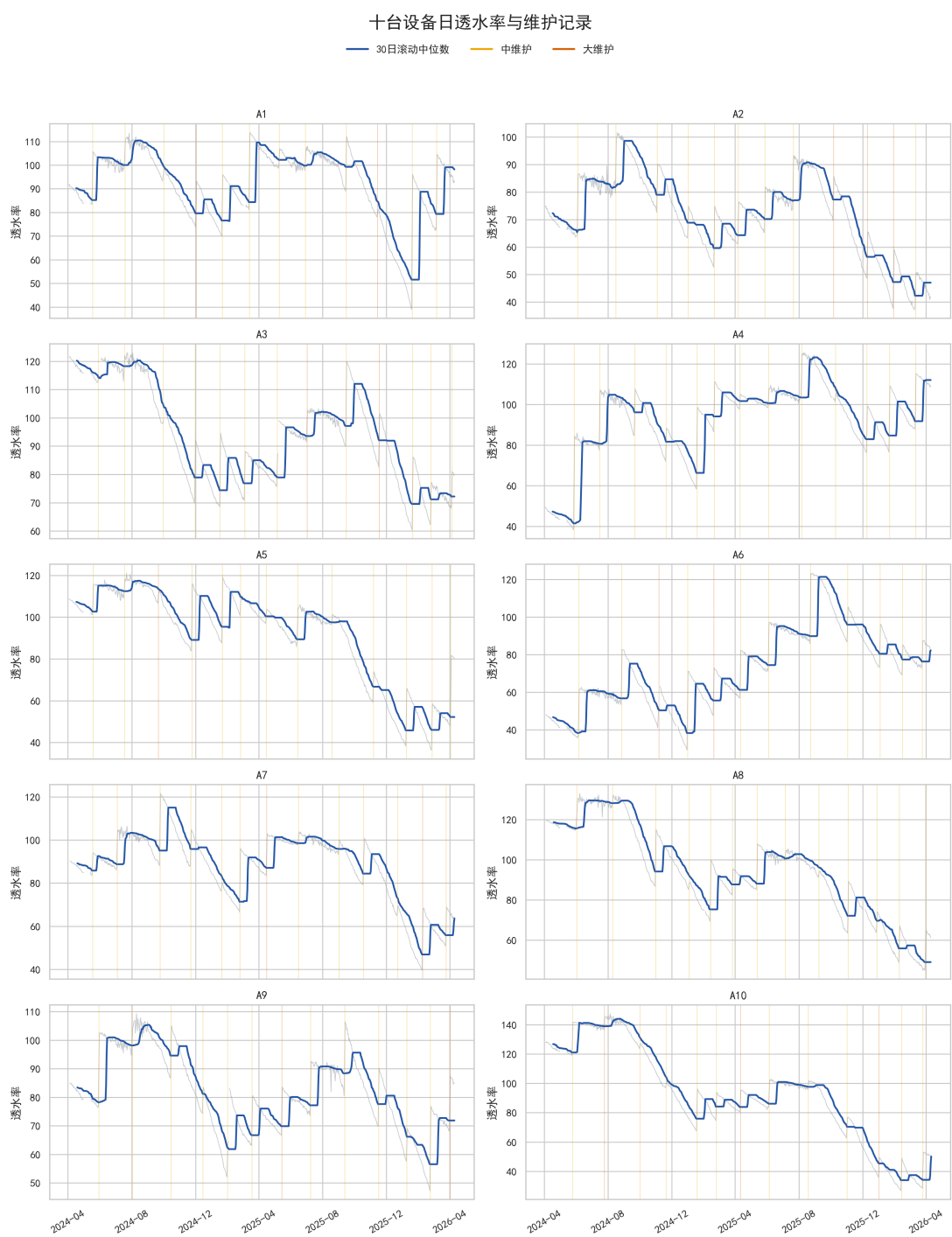


图1 十台设备日透水率与维护记录

加入维护历史与分段污染变量后，模型 R^2 从 0.7996 提高至 0.8601，RMSE 从 9.566 降至 8.091。月份固定效应完整模型的 R^2 为 0.8574，说明结果不依赖单一季节表达方式。

模型	R^2	RMSE	AIC
Fourier 基线	0.7996	9.566	53413.6
Fourier 完整	0.8601	8.091	50828.8
月份基线	0.7968	9.636	53526.9
月份完整	0.8574	8.167	50981.8

表 2 问题一模型拟合比较

公共季节效应约在年内第 236 天达到峰值、第 15 天达到低谷，半峰谷差约为 15.24，完整峰谷差约为 30.48，见图 2。观察窗口只有约两个年度周期，图中区间仅表示给定模型下的参数不确定性。

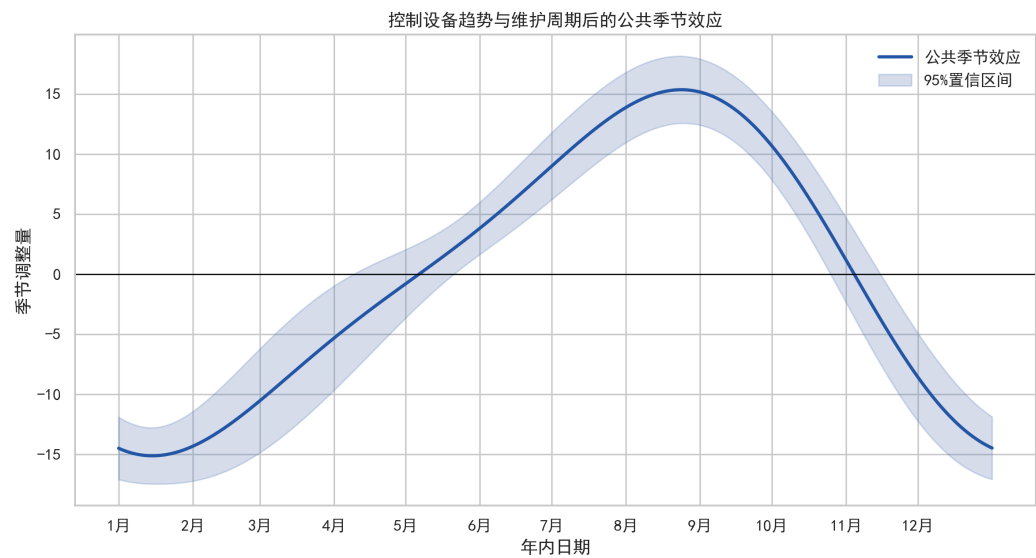


图 2 控制设备趋势与维护周期后的公共季节效应

维护事件研究结果见表 2 和图 3。中维护的即时恢复量中位数为 17.16，8—30 日保持量为 11.86；大维护分别为 15.41 和 9.17。使用月份固定效应重新计算后，各项变化不超过约 1 个透水率单位，方向保持一致。

类型	事件数	即时恢复	95% 区间	8—30 日保持量
中维护	110	17.16	[14.90, 19.89]	11.86
大维护	17	15.41	[10.84, 18.28]	9.17

表 3 控制趋势与季节后的维护恢复量

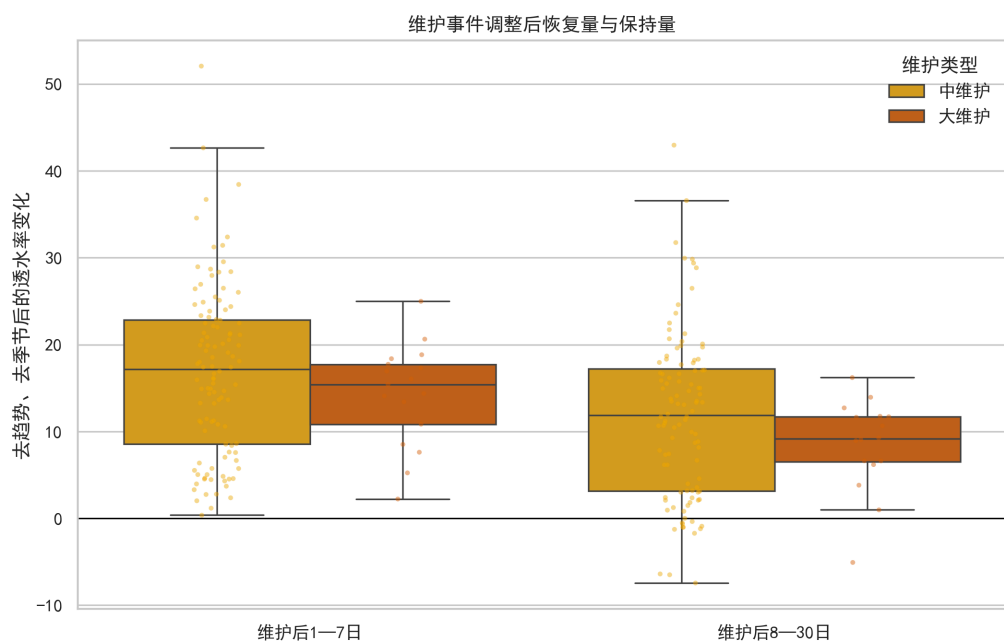


图3 维护事件调整后恢复量与保持量分布

中维护历史记录的单位费用保持量约为 3.95 个透水率单位/万元，大维护约为 0.76。但大维护仅有 17 次且通常在状态更差时触发，存在指征偏差，不能据此直接得出取消大维护的策略结论。

十台设备当前滚动 365 日平均性能均高于阈值 37，阈值裕度最低的是 A2（约 30.04），最高的是 A4（约 64.47）。控制季节和维护周期后，8 台设备观测期线性趋势为负，A4、A6 为正；这些斜率只作为问题二的初值和对照，不能直接线性外推寿命。

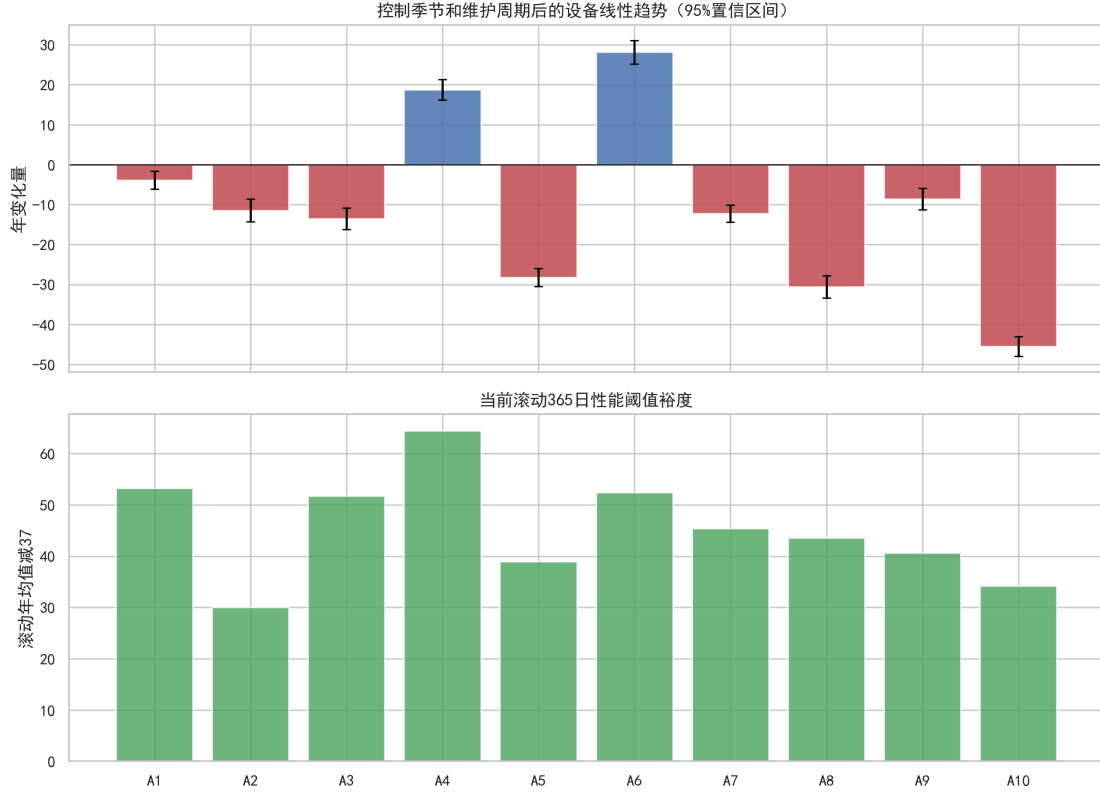


图 4 设备调整后线性趋势与当前性能阈值裕度

完整模型共标记 55 个残差异常日，占有效建模日约 0.76%。异常清单单独保存，后续寿命模型将比较保留与排除异常点两种结果。

六、问题二模型的建立与求解

6.1 模型建立

对设备 i 建立可向前模拟的分层退化状态模型。这一表达借鉴状态空间将趋势、季节与扰动分解的思路^[3]，并通过设备间收缩降低短面板的个体参数方差^[4]：

$$y_{i,t} = \alpha_i - d_i T_t + u_i \left(1 - \exp\left(-\frac{T_t}{\tau}\right) \right) + S(t) + G(r_{i,t}) + \varepsilon_{i,t}, \quad d_i \geq 0 \quad (2)$$

其中 d_i 为长期不可逆退化率； $u_i(1 - \exp(-\frac{T_t}{\tau}))$ 为有界的早期运行漂移， $\tau = 2$ 年； $S(t)$ 沿用一年和半年 Fourier 项； $G(r_{i,t})$ 为距上次维护时间的 0—30 日、30—90 日分段污染状态。参数通过下式估计：

$$\min \sum_{i,t} w_{i,t} (y_{i,t} - \hat{y}_{i,t})^2 + \lambda \sum_i (d_i - |(d)|)^2 + \kappa \sum_i u_i^2 \quad (3)$$

主规格取 $\lambda = 1600, \kappa = 5$ 。强制 $d_i \geq 0$ 可避免把 A4、A6 在两年观测窗中的正斜率永久外推；汇聚项则避免仅凭两个年度周期得到过度分散的寿命。

附件中各设备相邻维护的中位间隔为 54—60 日，未来按该固定间隔延续；大维护频率按设备历史比例延续。A4、A8 没有大维护记录，主情景用全队列约每 7 次记录维护

一次大维护补充，并在结果中单独标记。小维护因无日期记录，作为固定背景过程吸收到 $G(r_{i,t})$ 中。

寿命采用联合判据。令 $|(y)_{i,t}^{365}$ 为滚动 365 日平均值，则先找到

$$t_i^0 = \min\{t : |(y)_{i,t}^{365} < 37\}. \quad (4)$$

再寻找 t_i^0 后第一次满足维护后 8—30 日平均性能仍低于 37 的大维护，观察窗结束日才记为正式寿命终点。这样不会把季节低谷或一次暂时下穿直接当作报废。

6.2 模型求解

先以 90、180、270 日三个滚动时间留出比较设备独立线性、带维护外生量的 SARIMAX-AR(1) 和分层退化状态模型。平均结果见表 4。

模型	日 MAE	日 RMSE	月均 RMSE	滚动 365 日 RMSE
设备独立线性	12.732	15.487	14.904	3.669
SARIMAX-AR(1)	12.361	16.124	15.134	3.497
分层退化状态	12.731	15.683	14.954	3.615

表 4 问题二三组时间留出的平均误差

分层状态模型在最长的 270 日留出中日 RMSE 为 16.376，是三者最低；其三组平均日 RMSE 比设备独立线性高约 1.3%，也未在所有平均误差指标上占优。选择它的原因不是“回测全面最优”，而是最长留出表现较好，且同时满足长期退化非负、有界早期漂移和设备间部分汇聚三个外推约束。模型整体加权 $R^2 = 0.8499$ 、RMSE 为 8.288，共同长期退化率为 12.26 个透水率单位/年。结构超参数变化下，各设备长期退化率范围见图 6。

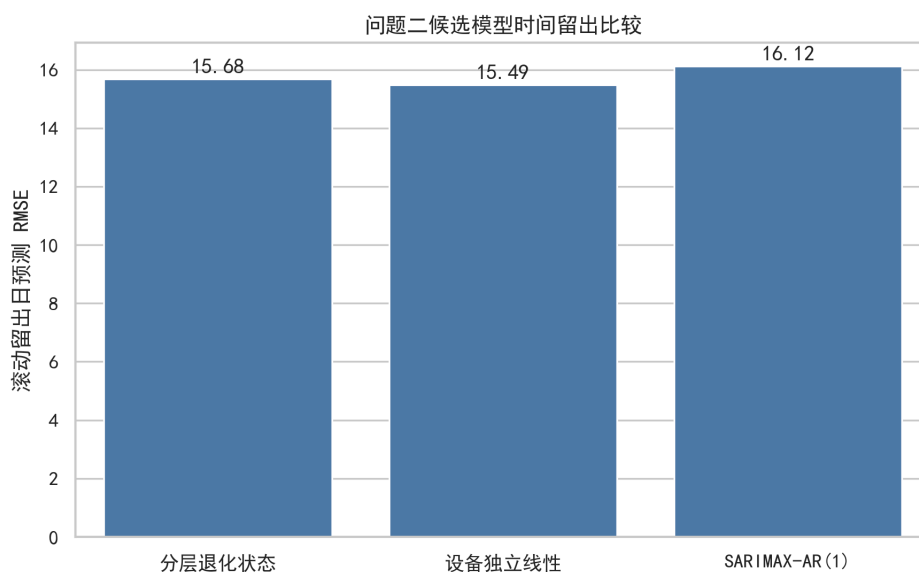


图 5 问题二候选模型时间留出比较

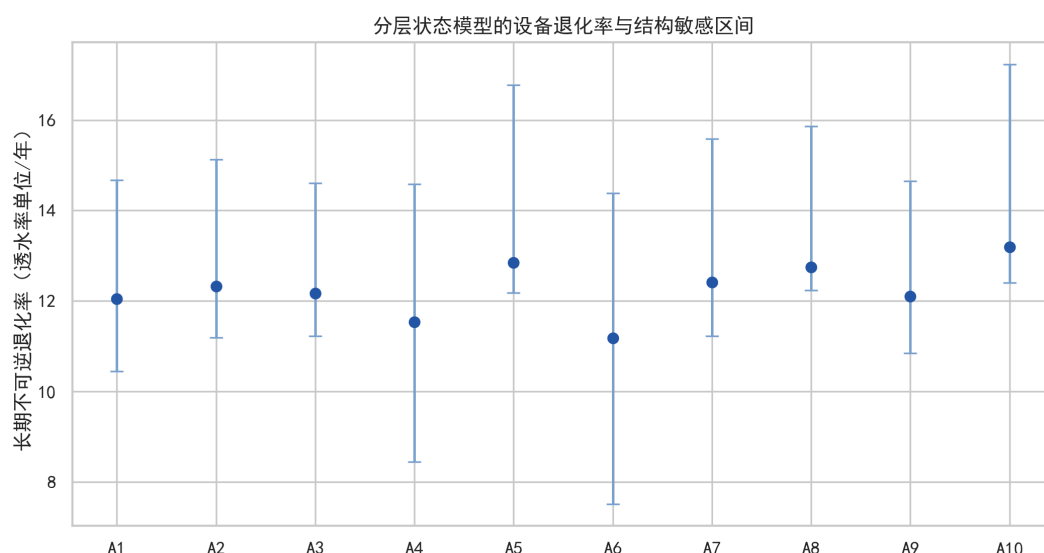


图 6 设备长期退化率与结构敏感区间

最终在 9 组结构超参数、残差异异常点情景和参数扰动上，每台设备生成 2,000 条路径，预测上限为 30 年。未在上限内满足联合判据的路径按右删失保留。

6.3 结果分析

寿命结果见表 5。表中寿命从 2022 年 4 月 1 日投用时计，“触发月份”是联合判据在主情景中被确认的月份，情景区间为第 10%—90% 分位数。A10 中位寿命最短，为 5.63 年；A6 最长，为 14.15 年。这些月份用于安排绩效复核和诊断性大维护，不是设备的保证报废日期。

设备	寿命中位数/年	中位触发月份	80% 情景区间
A1	9.60	2031-11	2030-11—2033-10
A2	6.84	2029-01	2029-01—2030-04
A3	9.42	2031-09	2030-09—2032-08
A4	13.86	2036-02	2032-10—2044-11
A5	6.72	2028-12	2028-12—2031-02
A6	14.15	2036-05	2031-11—2040-01
A7	9.33	2031-08	2029-10—2031-08
A8	7.18	2029-06	2029-06
A9	7.76	2030-01	2030-01—2030-12
A10	5.63	2027-11	2027-11—2028-07

表 5 现行维护规律下十台设备寿命预测

现行维护规律下滚动年均透水率预测（失效判据附近）

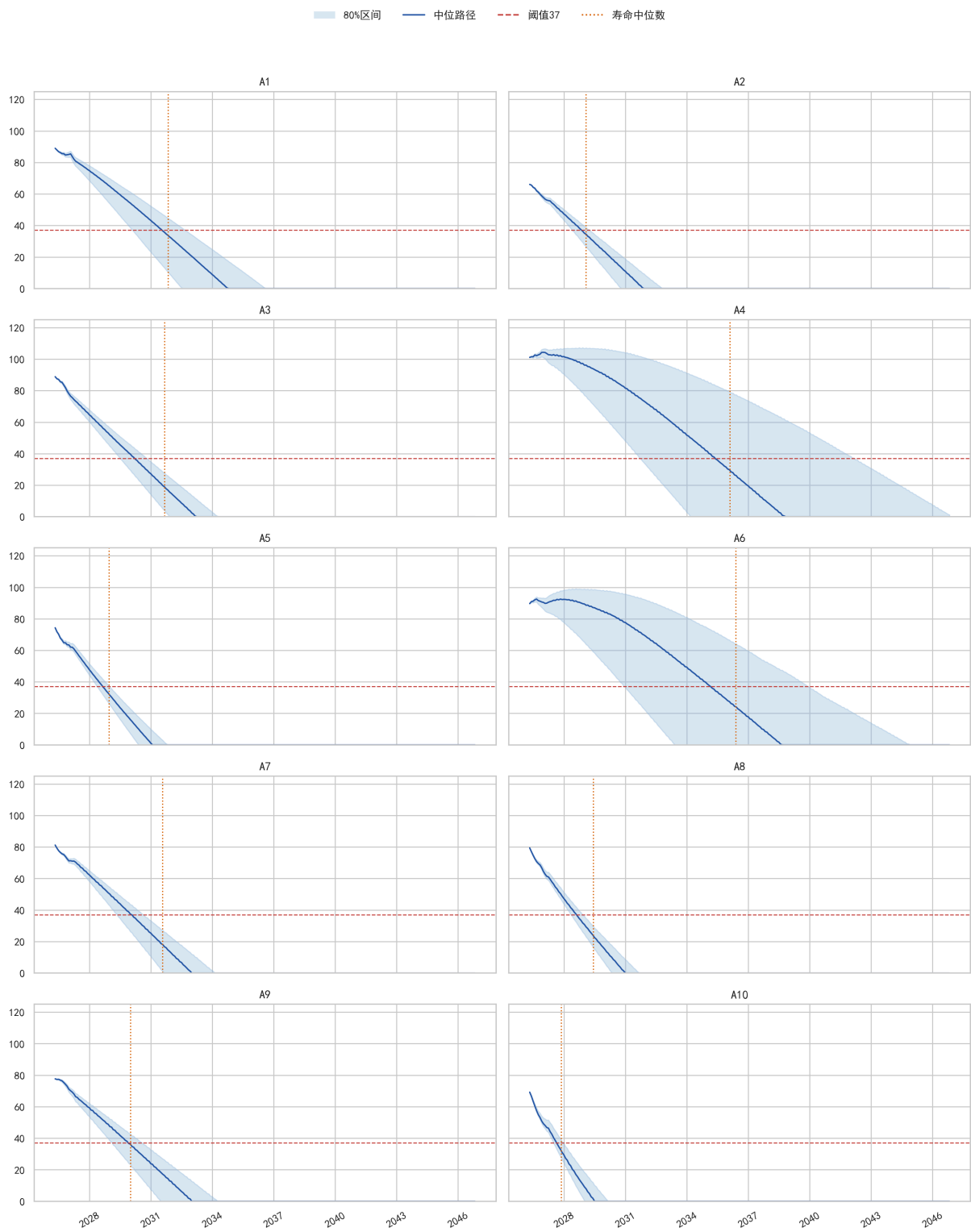


图 7 现行维护规律下滚动年均透水率预测带

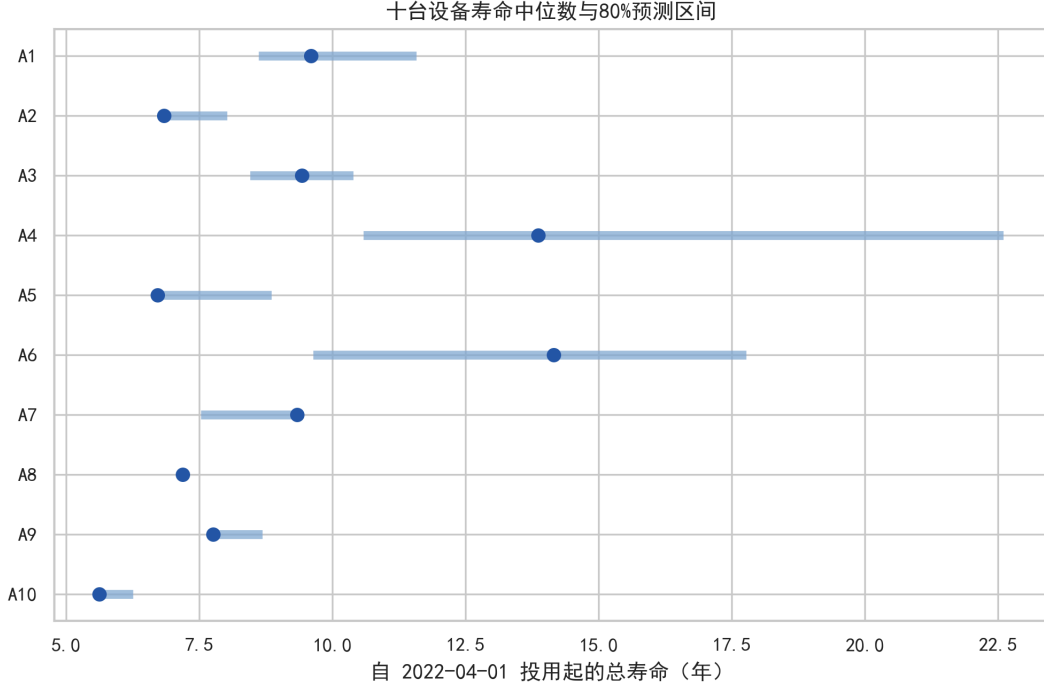


图 8 设备总寿命中位数与 80% 情景区间

A4、A6 的区间明显更宽，反映观测期正趋势必须由有界漂移与长期退化共同解释；A6 约有 1.3% 模拟路径在 30 年上限内仍未触发联合判据。A4、A8 的大维护频率来自队列补充，其情景区间应比完整记录设备更谨慎解释。A8 的两个分位数落在同一月，源于联合判据只在离散大维护窗口确认，不代表其不确定性为零。

七、问题三模型的建立与求解

7.1 模型建立

问题二的 $G(r_{i,t})$ 已给出任意一次记录维护后的平均污染重置。对中维护 M 和大维护 B 再增加类型修正 R^M, R^B :

$$y_{i,t} = H_{i,t} + S(t) - F_{i,t} + R_{i,t}^M + R_{i,t}^B + \varepsilon_{i,t}. \quad (5)$$

事件研究中 8—30 日平均保持量为 11.67 和 8.44。由于大维护仅 17 次，将两类效果向总体均值部分汇聚，得 11.61 和 9.95。类型修正在 31—60 日线性衰减为 0，后续污染由 G 承担。

维护损伤与自然老化难以由两年数据分离。设维护相关损伤占问题二总退化的比例为 ρ ，主情景取 $\rho = 0.15$ ，并比较 0 与 0.30。中、大维护损伤强度比取 1:4，同时校正自然退化，使历史频率下的期望修正为 0。

状态触发与周期性预防维护都是退化设备的典型决策类型^[5,6]；本文将监测、状态预测与维护决策按状态维护链路衔接^[7]。对策略 π 的路径级目标为

$$J_{i(\pi)} = E \left[\frac{300 + 3N_{i,M}(\pi) + 12N_{i,B}(\pi)}{\frac{L_{i(\pi)}}{365}} \right]. \quad (6)$$

小维护固定为背景过程。主约束为每台每年 1—4 次大维护、相邻记录维护满足冷却期，且年均值低于 37 后必须在诊断窗内大维护。

7.2 模型求解

搜索空间共 129 组：1 组现行日历；中维护间隔 35—365 日、大维护间隔 91—365 日的 44 组周期规则；触发线 40—70、冷却期 30—60 日与同样大维护间隔组成的 84 组状态触发规则。

先用主结构点估计计算全部候选，再以 200 条公共随机路径筛选前沿，最后对现行、筛选第一和全厂统一候选各生成 2,000 条路径。路径同时抽样 9 组退化结构、残差异异常点情景、维护效果标准误和损伤比例。独立随机种子再用 1,000 条路径复验方向。

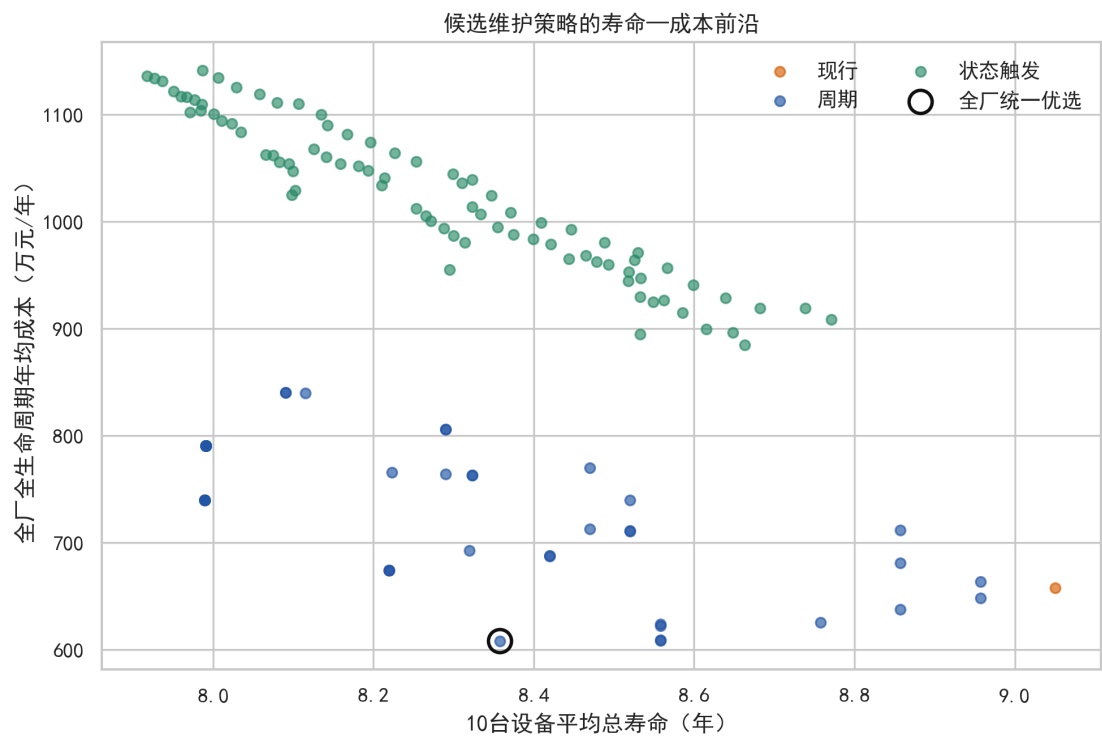


图 9 候选维护策略的寿命—成本前沿

7.3 结果分析

主推方案为全厂统一“年 1 大”：每年一次大维护，不安排额外中维护；若滚动年均率先下穿 37，则将当次大维护作为诊断性维护，并在后 8—30 日复核。该制度的估计年均成本为 607.86 万元/年，相比现行制度低 7.50%；与逐设备精细化候选的 603.41 万元/年相比仅高 0.74%，但实施、备件和工单管理更简单。

表 6 给出的是在已枚举 129 组规则、既定损伤情景和成本口径下的“逐设备精细化候选”，主要用于衡量统一制度的机会成本，不声称是所有可能制度中的全局最优。成本单位为万元/年，寿命从投用日起算。

设备	精细化候选	现行→候选成本	节省	现行→候选寿命
A1	年 1 大	58.97→53.03	10.15%	9.60→8.70
A2	150 日 1 中+年 1 大	77.10→71.73	6.88%	6.84→6.91
A3	年 1 大	61.71→60.77	1.49%	9.42→7.71
A4	年 1 大	50.42→40.60	20.08%	13.86→14.02
A5	年 1 大	66.86→65.77	1.27%	6.72→7.17
A6	年 1 大	52.48→39.96	24.85%	13.25→14.14
A7	年 1 大	64.06→61.58	3.84%	9.33→7.94
A8	90 日 1 中+年 1 大	72.21→70.63	2.19%	7.18→7.08
A9	年 1 大	66.28→59.72	9.93%	8.68→8.08
A10	年 1 大	87.06→79.63	8.47%	5.63→5.83

表 6 逐设备精细化候选与现行制度比较

精细化候选将全厂期望年均成本由 657.14 万元/年降至 603.41 万元/年，估计节省 8.18%。但它只在 A2、A8 上比统一规则增加中维护，因此 4.45 万元/年的额外差额不足以抵消复杂制度的组织成本与参数风险。

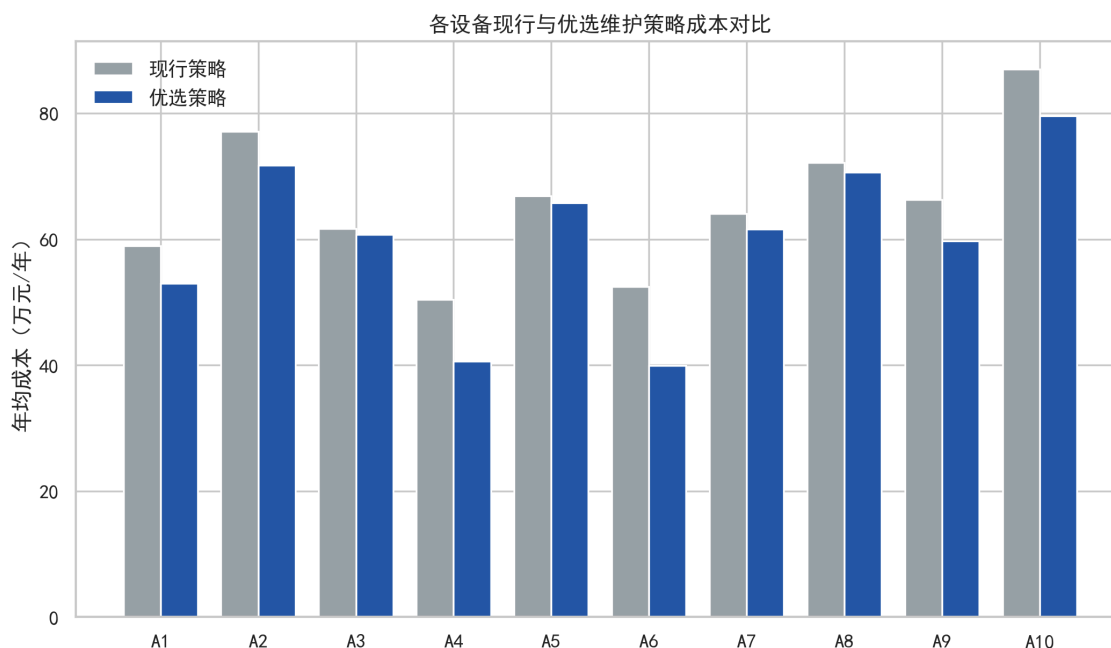


图 10 各设备现行与精细化候选的年均成本

目标优化的是年均成本而不是最长寿命，因此 A1、A3、A7、A8、A9 在精细化候选中出现寿命中位数缩短。其中 A3、A7 节省的 80% 仿真区间已跨过 0，A5 的正节省概率仅 39.0%，且 A5、A7 在部分维护效果/损伤情景中结论翻转。因此现场实施应以统一“年 1 大”为试行基线，而不应把表 6 当作对每台设备的刚性指令。

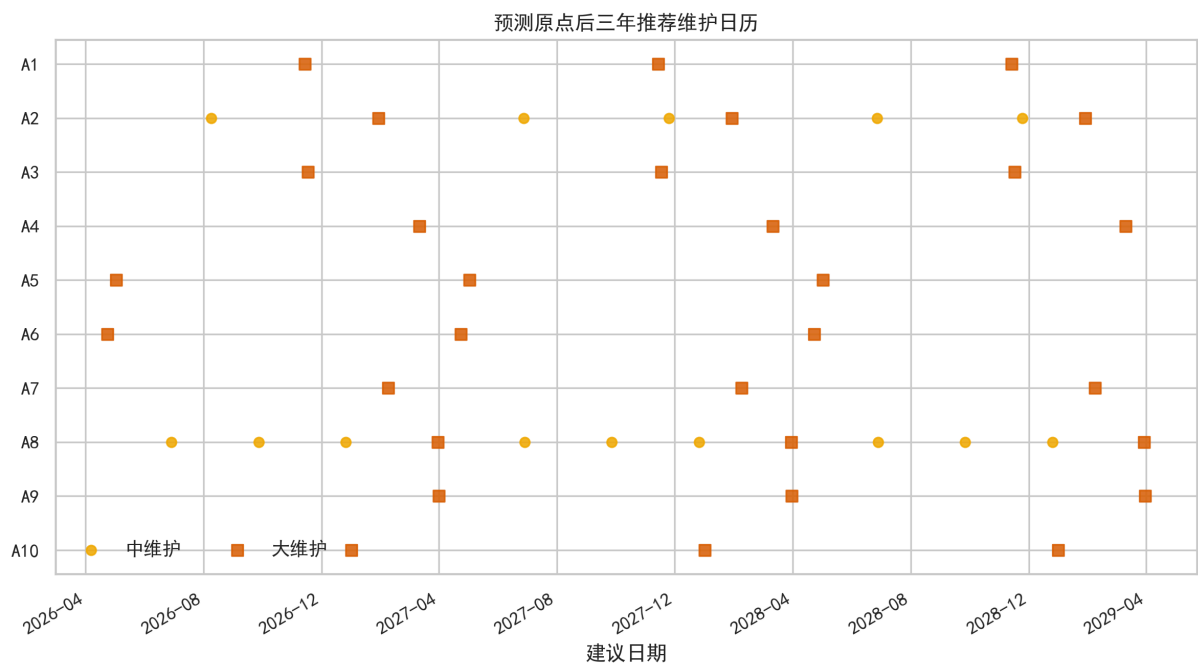


图 11 预测原点后三年推荐维护日历

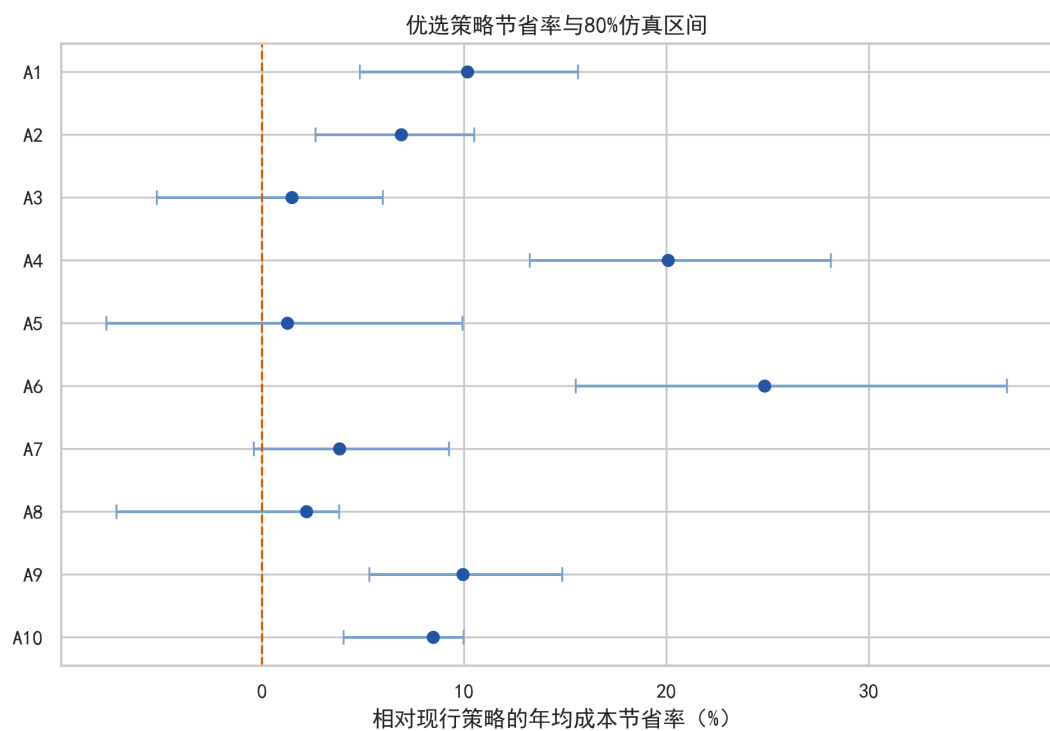


图 12 精细化候选节省率与 80% 仿真区间

八、问题四模型的建立与求解

8.1 成本重定价模型

对设备 i 、策略 π 和问题三的每条物理仿真路径，记总寿命为 T 年，中、大维护总次数为 N_M, N_B 。价格分别为购置价 P 、中维护价 M 和大维护价 B ，则期望年均成本可以精确分解为

$$J_{i,\pi}(P, M, B) = Pa_{i,\pi} + Mb_{i,\pi} + Bc_{i,\pi}, \quad (7)$$

其中

$$a_{i,\pi} = E\left(\frac{1}{T}\right), \quad b_{i,\pi} = E\left(\frac{N_M}{T}\right), \quad c_{i,\pi} = E\left(\frac{N_B}{T}\right). \quad (8)$$

三个系数只由问题三的物理路径和策略决定。价格波动时固定公共随机路径，只线性重算成本，不重复拟合退化模型；基准价格下的结果与问题三完全一致。因而价格变化不会改变既定方案的寿命、维护次数、冷却期和每年 1—4 次大维护约束，只可能改变经济最优性。

8.2 价格场景与求解

先对问题三全部 129 组策略在所有价格点做点估计筛选，将每台设备的前两名、现行策略、问题三方案和统一候选取并集，得到 56 个“设备—策略”对，再以相同随机种子各生成 2,000 条路径复验。正式价格集合包括：

1. 购置价和中/大维护共同价格分别取基准的 0.60—1.40 倍，步长 0.05，共 $17 \times 17 = 289$ 个场景；
2. 购置价固定，中维护价和大维护价分别取 0.50—2.00 倍，步长 0.10，共 $16 \times 16 = 256$ 个场景；
3. 购置、共同维护、中维护和大维护四类单因素各取 0.25—3.00 倍，步长 0.01，共 1,104 个价格点。

每个场景分别重选逐设备方案和全厂统一方案。设场景 s 下方案成本为 C_s ，该场景可达最小成本为 C_s^* ，定义后悔值 $r_s = \frac{C_s - C_s^*}{C_s^*}$ 。除精确选中外，将 $r_s \leq 1\%$ 作为无需为微小成本差频繁改变制度的近优判据。跨场景鲁棒方案依照统计决策中的最小最大思想^[8]，最小化 289 个共同价格场景中的最大后悔值。

8.3 结果分析

价格稳定区域见图 13。图中 P0 为问题三逐设备精细化候选，P1—P18 只是将十台设备的策略映射压缩为可视化编号，不用作现场执行标签；论文的可操作结论以表 7 和统一规则为准。289 个共同价格场景中，P0 在 31.49% 的场景被原样选中，但在 95.85% 的场景仍处于最优成本 1% 以内，最大后悔值为 1.533%。这说明精确方案切换主要来自少数设备的中维护间隔微调，并不等于原方案失去经济可行性。

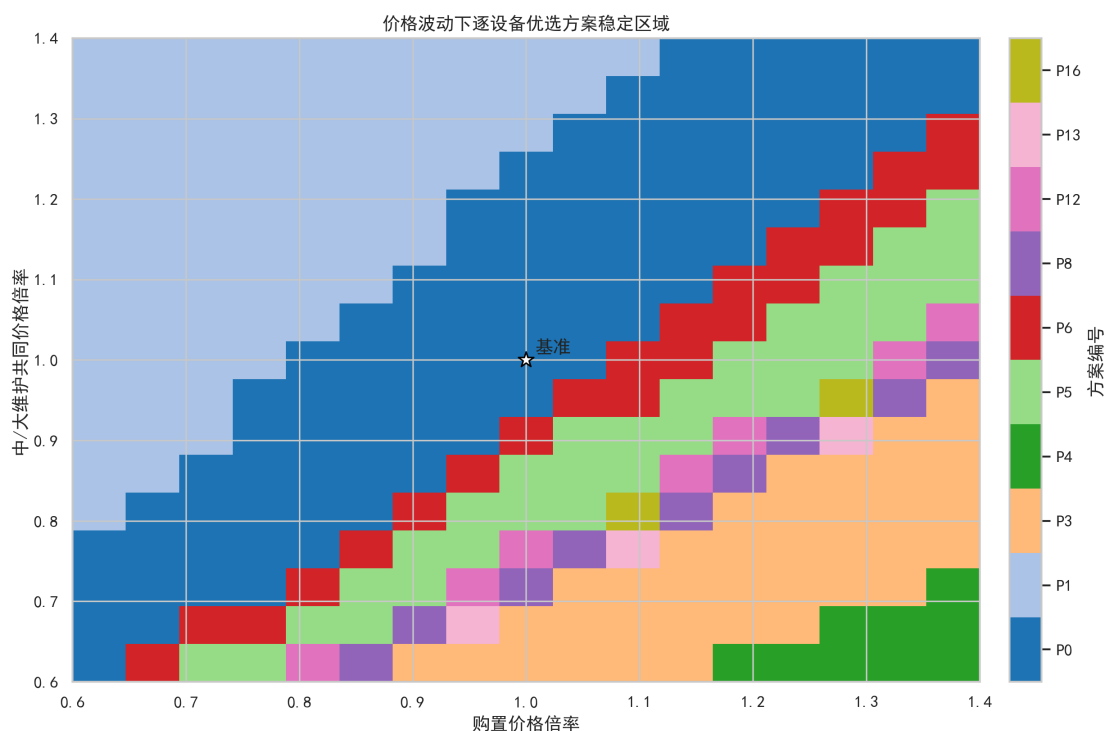


图 13 购置价与共同维护价波动下的逐设备优选方案区域

包含基准价格倍率 1 的连续稳定区间见表 7。“ $\leq$ ”或“ $\geq$ ”表示稳定区延伸到扫描边界，端点不是已识别出的真实临界价。

价格因素	逐设备精确最优	逐设备成本差 $\leq 1\%$	统一一年 1 大精确最优
购置价	0.79—1.07	≤ 0.25 —1.89	≤ 0.25 —2.33
全部维护价	0.94—1.26	0.53— ≥ 3.00	0.43— ≥ 3.00
中维护价	0.94—1.27	0.55— ≥ 3.00	0.44— ≥ 3.00
大维护价	≤ 0.25 —1.37	≤ 0.25 — ≥ 3.00	≤ 0.25 — ≥ 3.00

表 7 问题三方案的单因素价格稳定区间

问题三全厂统一方案“每年 1 次大维护、无额外中维护”在 288/289 个共同价格场景中仍是最佳统一策略，在全部 289 个场景中均处于最佳统一成本 1% 以内，其相对最佳统一策略的最大后悔值仅 0.004%。该指标的比较对象是统一策略集合；若与允许逐设备不同制度的理论最小成本相比，基准场景差距仍为 0.74%。

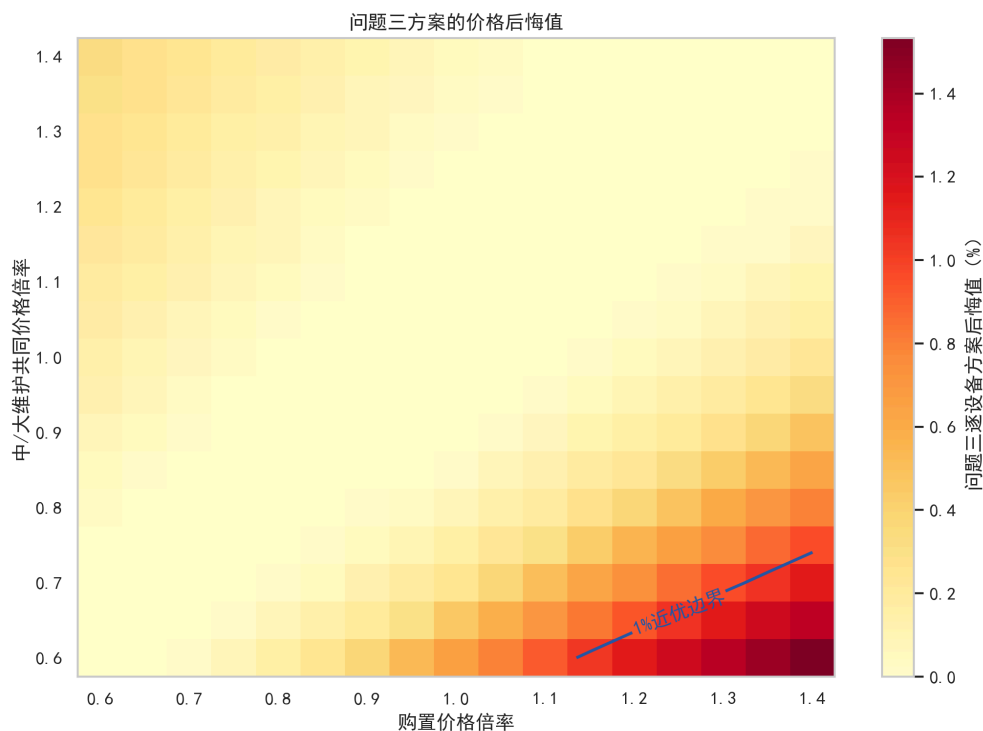


图 14 问题三逐设备方案在共同价格网格中的后悔值

共同网格中最不利于问题三逐设备方案的角点是购置价上升 40%、维护价下降 40%：原方案为 665.75 万元/年，重新优化为 655.70 万元/年。中、大维护分别变化时，若中维护价降至 0.5 倍而大维护价升至 2.0 倍，原方案后悔值为 1.304%，此时增加 A2、A3、A7、A8 的中维护更有利。反之，大维护相对便宜时，“年 1 大”区域明显扩大。

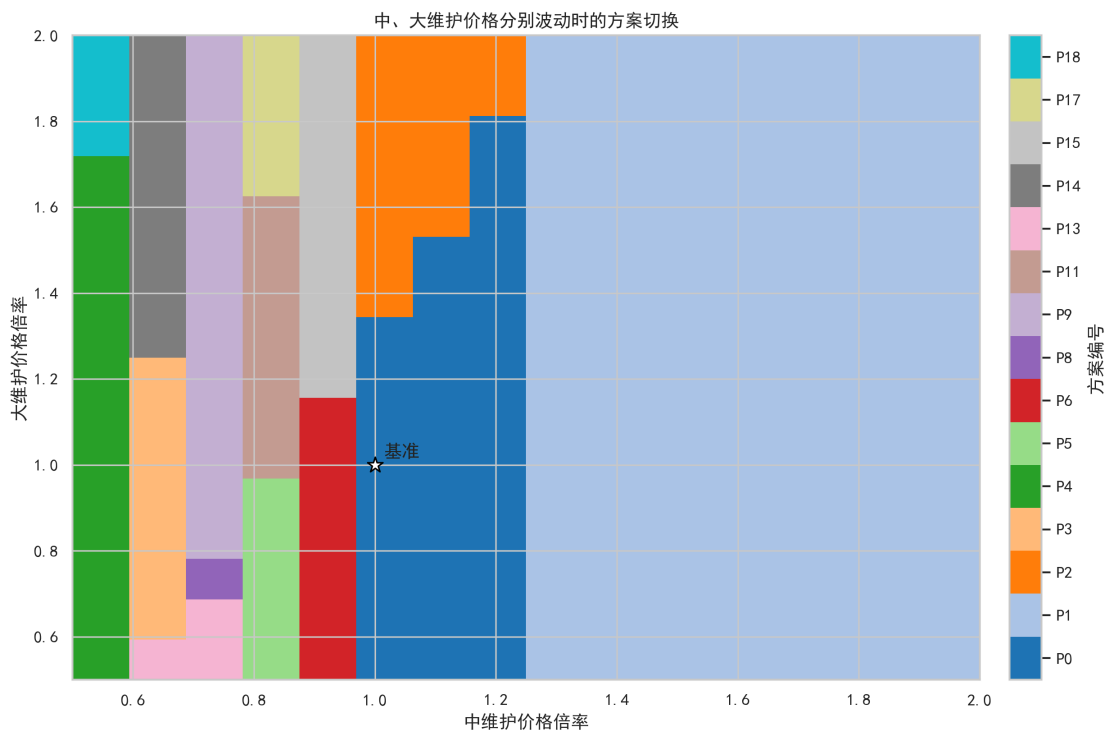


图 15 中、大维护价格分别变化时的逐设备方案切换

在 289 个共同价格场景上求最小最大后悔，得到鲁棒逐设备方案：A1、A4、A5、A6、A9、A10 每年 1 次大维护；A2 每 90 天、A3 与 A7 每 240 天、A8 每 75 天做中维护，且均保留每年 1 次大维护。该方案在基准价下为 604.50 万元/年，仅比基准最优高 0.18%；跨场景最大后悔值由原方案的 1.533% 降至 1.171%，平均为 0.372%。若更重视制度简单和现场一致性，则统一“每年 1 次大维护”本身就是统一策略集合中的最小最大后悔方案。

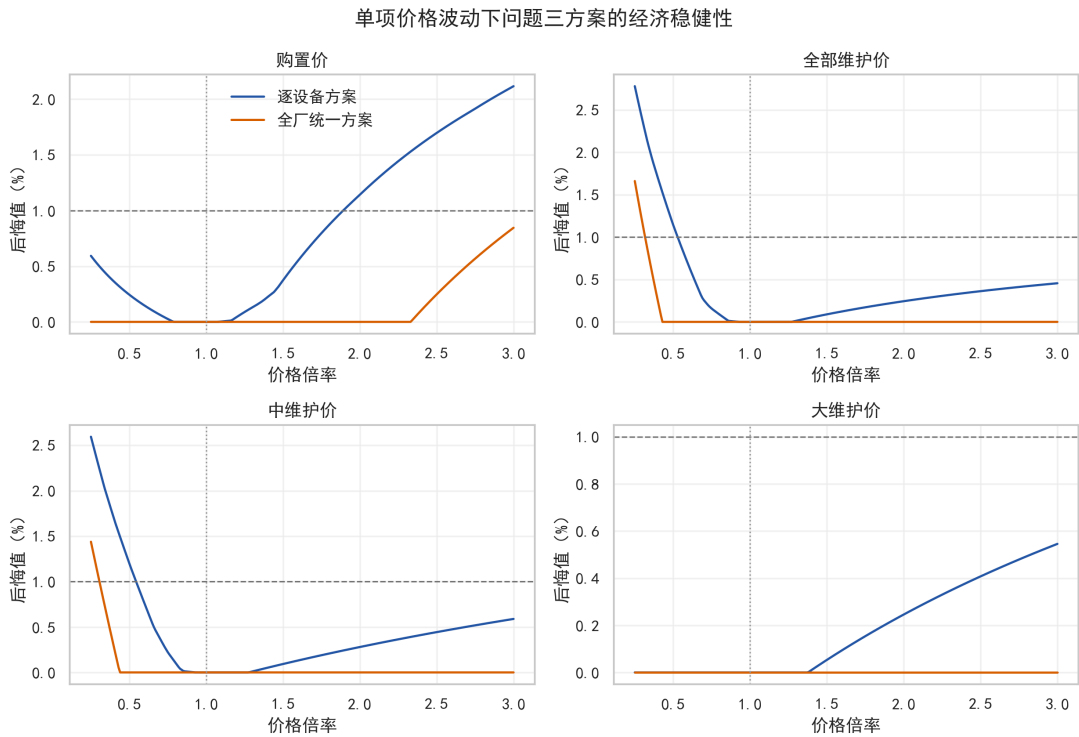


图 16 单项价格波动下问题三方案的经济稳健性

综上，问题三方案在所考察价格范围内运行上始终可行；经济上虽会出现局部策略切换，但绝大多数共同价格场景的损失低于 1%。只有当购置价显著上涨且维护相对变便宜，或中维护与大维护相对价格强烈分化时，才有必要缩短部分设备的中维护间隔。

九、模型的检验与灵敏度分析

问题一已用月份固定效应替换 Fourier 项，维护效应方向和量级保持一致。问题二改变汇聚强度、早期漂移时间尺度和惩罚强度，形成 9 组结构模型，同时比较残差异常日和恢复线。问题三再比较维护损伤 0%/15%/30%、三种动作效果结构、独立随机种子和放宽大维护约束情景。问题四固定同一批物理路径，分别改变三类价格，并用精确选中、1% 近优和最大后悔三种口径检验经济稳定性。

敏感性项目	结果	解释
排除 55 个残差异常日	退化率最大变化 4.0%	主结果不由少数极值驱动
恢复线 35/37/39	多数寿命中位数不变	离散大维护日主导确认时点
9 组状态结构	纳入 80%/95% 情景区间	反映长期外推结构不确定性
留一设备冷启动	平均 RMSE 34.85	个体历史不可由队列均值替代
问三独立随机种子	10 台平均节省均为正	主方向可复现
损伤比例+动作效果	A5、A7 可翻转	小节省设备需风险标记
问四共同价格网格	95.85% 场景成本差 $\leq$ 1%	局部切换不等于方案失效

表 8 问题一至问题四稳健性检验

十、模型的评价、改进与推广

10.1 模型的优点

1. 把季节、可恢复污染、短期运行漂移和不可逆退化分开，避免把维护跳升或正斜率永久外推；
2. 寿命使用年均阈值和维护后恢复联合判据，比单日阈值更符合题意；
3. 同一状态模型连接寿命与维护决策，同时给出逐设备精细化候选和便于执行的全厂统一方案；
4. 将价格与物理路径解耦为线性成本系数，能无额外仿真噪声地扫描价格边界，并以最小最大后悔给出价格未知时的备选方案；
5. 所有结果由统一流水线重建，并通过时间留出、留一设备、异常点、结构情景和独立随机种子检验。

10.2 模型的缺点

1. 观测期只有约两年且没有真实失效样本，寿命属于退化轨迹阈值外推；
2. A4、A8 缺少大维护记录，小维护完全无日期，维护制度变化可能造成模型外失真；
3. 大维护只有 17 次且存在指征偏差，损伤比例只能作情景识别，A5、A7 的小节省结论不够稳健。
4. 价格分析未包含停机损失和并联产能约束，且边界只对已枚举的 129 组可解释策略成立。

10.3 模型的改进与推广

若获得更长历史，可用真实失效或更换记录校准联合判据，并让长期退化率随工况协变量变化；若补充小维护和耗材记录，可把当前背景污染状态拆成可控动作。该框架也适用于膜过滤、换热器结垢等同时具有季节、污染积累和维护复位的设备。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于题意与数据结构梳理、建模方案辅助设计、Python 代码编写与调试、结果检验与可视化、Typst 论文起草与排版，以及按 2026 年试行规定补正声明，详细使用情况见支撑材料。

参考文献

- [1] HYNDMAN R J, ATHANASOPOULOS G. Forecasting: Principles and Practice[M/OL]. 3 版. Melbourne: OTexts, 2021. <https://otexts.com/fpp3/>.
- [2] EFRON B. Bootstrap Methods: Another Look at the Jackknife[J/OL]. The Annals of Statistics, 1979, 7(1): 1-26. DOI:10.1214/aos/1176344552.
- [3] DURBIN J, KOOPMAN S J. Time Series Analysis by State Space Methods[M/OL]. 2 版. Oxford University Press, 2012. DOI:10.1093/acprof:oso/9780199641178.001.0001.
- [4] LAIRD N M, WARE J H. Random-Effects Models for Longitudinal Data[J/OL]. Biometrics, 1982, 38(4): 963-974. DOI:10.2307/2529876.
- [5] WANG H. A Survey of Maintenance Policies of Deteriorating Systems[J/OL]. European Journal of Operational Research, 2002, 139(3): 469-489. DOI:10.1016/S0377-2217(01)00197-7.
- [6] GRALL A, BÉRENGUER C, DIEULLE L. A Condition-Based Maintenance Policy for Stochastically Deteriorating Systems[J/OL]. Reliability Engineering & System Safety, 2002, 76(2): 167-180. DOI:10.1016/S0951-8320(01)00148-X.
- [7] JARDINE A K S, LIN D, BANJEVIC D. A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance[J/OL]. Mechanical Systems and Signal Processing, 2006, 20(7): 1483-1510. DOI:10.1016/j.ymssp.2005.09.012.
- [8] SAVAGE L J. The Theory of Statistical Decision[J/OL]. Journal of the American Statistical Association, 1951, 46(253): 55-67. DOI:10.1080/01621459.1951.10500768.

附录 1：支撑材料文件列表

支撑材料/

```
|-- AI_工具使用详情.pdf
|-- README.md
|-- requirements-lock.txt
|-- run_pipeline.py
|-- src/filter_monitoring/
|   |-- __init__.py
|   |-- config.py
|   |-- io.py
|   |-- preprocess.py
|   |-- q1_analysis.py
|   |-- q2_lifetime.py
|   |-- q3_optimization.py
|   |-- q4_sensitivity.py
|-- data/results/
|   |-- q1_data_quality.csv
|   |-- q1_device_metrics.csv
|   |-- q1_event_effects_detail.csv
|   |-- q1_key_findings.json
|   |-- q1_maintenance_effects.csv
|   |-- q1_model_diagnostics.csv
|   |-- q2_forecast_bands.csv
|   |-- q2_key_findings.json
|   |-- q2_leave_one_asset_validation.csv
|   |-- q2_lifetime_sensitivity.csv
|   |-- q2_lifetime_summary.csv
|   |-- q2_model_backtest.csv
|   |-- q2_structural_sensitivity.csv
|   |-- q3_current_vs_optimal.csv
|   |-- q3_key_findings.json
|   |-- q3_optimal_policy_by_asset.csv
|   |-- q3_policy_evaluation.csv
|   |-- q3_recommended_calendar.csv
|   |-- q3_robustness.csv
|   |-- q4_key_findings.json
|   |-- q4_one_factor_sensitivity.csv
|   |-- q4_price_grid.csv
|   |-- q4_robust_policy.csv
|   |-- q4_split_maintenance_grid.csv
|-- q4_switch_intervals.csv
```

附录 2：工程入口与公共配置源程序

run_pipeline.py

```
1 #!/usr/bin/env python3
2 """本题统一运行入口。"""
3
4 from __future__ import annotations
5
6 import argparse
7 import sys
8 from pathlib import Path
9
10 PROJECT_DIR = Path(__file__).resolve().parent
11 sys.path.insert(0, str(PROJECT_DIR / "src"))
12
13 from filter_monitoring.io import audit_inputs # noqa: E402
14 from filter_monitoring.q1_analysis import run_q1_formal, run_q1_scaffold # noqa: E402
15 from filter_monitoring.q2_lifetime import run_q2_formal # noqa: E402
16 from filter_monitoring.q3_optimization import run_q3_formal # noqa: E402
17 from filter_monitoring.q4_sensitivity import run_q4_formal # noqa: E402
18
19
20 def main() -> int:
21     parser = argparse.ArgumentParser(description="2026 校赛 B 题运行入口")
22     parser.add_argument("stage", choices=("audit", "q1-data", "q1", "q2", "q3", "q4"))
23     args = parser.parse_args()
24
25     if args.stage == "audit":
26         report = audit_inputs()
27         print(report[["input", "exists", "size_bytes"]].to_string(index=False))
28         print("输入校验通过。")
29         return 0
30
31     if args.stage == "q1-data":
32         outputs = run_q1_scaffold()
33         print(f"日级面板: {len(outputs['daily_panel'])} 行")
34         print(f"设备数: {outputs['data_quality']['asset'].nunique()}")
35         print(f"维护汇总: {int(outputs['maintenance_summary']['events'].sum())} 条事件")
36         print("问题一数据层已生成。")
37         return 0
38
39     if args.stage == "q1":
40         outputs = run_q1_formal()
41         diagnostics = outputs["model_diagnostics"].set_index("model")
42         print(f"日级面板: {len(outputs['daily_panel'])} 行")
43         print(f"设备指标: {len(outputs['device_metrics'])} 台")
44         print(f"维护事件: {len(outputs['event_detail'])} 条")
45         print(f"完整模型 R²: {diagnostics.loc['full', 'r_squared']:.4f}")
46         print(f"论文图: {len(outputs['figures'])} 张")
47         print("问题一正式分析结果已写入 data/results 和 figures/generated。")
48         return 0
49
50     if args.stage == "q2":
51         outputs = run_q2_formal()
52         lifetimes = outputs["lifetimes"]
53         finite = lifetimes.dropna(subset=["median_total_lifetime_years"])
54         print(f"回溯组合: {len(outputs['backtest'])} 组")
55         print(f"留一设备验证: {len(outputs['leave_one_asset'])} 台")
56         print(f"寿命模拟: {outputs['key_findings']['simulation_paths']} 条路径/设备")
57         if len(finite):
58             print(
59                 "寿命中位数范围: "
60                 f"{finite['median_total_lifetime_years'].min():.2f}—"
61                 f"{finite['median_total_lifetime_years'].max():.2f} 年"
62             )
63         print(f"论文图: {len(outputs['figures'])} 张")
64         print("问题二正式结果已写入 data/results 和 figures/generated。")
65         return 0
66
67     if args.stage == "q3":
68         outputs = run_q3_formal()
69         comparison = outputs["comparison"]
70         findings = outputs["key_findings"]
71         print(f"候选策略: {findings['candidate_policies']} 组")
72         print(f"正式复验: {findings['final_paths']} 条路径/设备-策略")
73         print(
74             "全厂年均成本: "
75             f"{findings['current_fleet_annual_cost']:.2f} -> "
76             f"{findings['optimized_fleet_annual_cost']:.2f} 万元/年"
77         )
78         print(f"年均成本节省: {findings['fleet_savings_percent']:.2f}%")
79         print(f"逐设备方案: {len(comparison)} 台")
80         print(f"论文图: {len(outputs['figures'])} 张")
81         print("问题三正式结果已写入 data/results 和 figures/generated。")
82         return 0
83
```

```

84
85     if args.stage == "q4":
86         outputs = run_q4_formal()
87         findings = outputs["key_findings"]
88         print(f"问题三候选策略: {findings['point_policy_space']} 组")
89         print(
90             "共同价格二维网格: "
91             f"{findings['common_price_grid_scenarios']} 个场景"
92         )
93         print(
94             "分拆维护价格网格: "
95             f"{findings['split_maintenance_grid_scenarios']} 个场景"
96         )
97         print(
98             "问题三逐设备方案在共同价格网格内保持1%近优: "
99             f"{findings['q3_asset_plan_within_1pct_share_common_grid']:.1%}"
100         )
101         print(
102             "全厂鲁棒统一策略: "
103             f"{findings['uniform_minimax_policy_id']}"
104         )
105         print(f"论文图: {len(outputs['figures'])} 张")
106         print("问题四正式结果已写入 data/results 和 figures/generated。")
107         return 0
108
109     raise AssertionError(f"未处理的运行阶段: {args.stage}")
110
111
112 if __name__ == "__main__":
113     raise SystemExit(main())
114

```

src/filter_monitoring/__init__.py

```

1  """2026 校赛 B 题：滤波设备监测与维护决策。"""
2
3  __version__ = "0.1.0"
4
5

```

src/filter_monitoring/config.py

```

1  """全项目唯一的路径、阈值、成本和日期口径。"""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from datetime import date
7  from pathlib import Path
8
9
10 PROJECT_DIR = Path(__file__).resolve().parents[2]
11 REPO_ROOT = PROJECT_DIR.parents[1]
12
13 RAW_DIR = PROJECT_DIR / "data" / "raw"
14 PROCESSED_DIR = PROJECT_DIR / "data" / "processed"
15 RESULTS_DIR = PROJECT_DIR / "data" / "results"
16 FIGURES_DIR = PROJECT_DIR / "figures" / "generated"
17
18 PROBLEM_DOCX = RAW_DIR / "B题.docx"
19 TELEMETRY_XLSX = RAW_DIR / "附件1.xlsx"
20 MAINTENANCE_XLSX = RAW_DIR / "附件2.xlsx"
21
22 ASSETS = tuple(f"A_{i}" for i in range(1, 11))
23 EXPECTED_TELEMETRY_SHEETS = tuple(f"A_{i}" for i in range(1, 11))
24
25 COMMISSION_DATE = date(2022, 4, 1)
26 OBSERVATION_START = date(2024, 4, 1)
27 OBSERVATION_END = date(2026, 4, 30)
28 LIFETIME_THRESHOLD = 37.0
29 ROLLING_YEAR_DAYS = 365
30 SHORT_GAP_LIMIT_DAYS = 2
31
32
33 @dataclass(frozen=True)
34 class CostConfig:
35     """费用单位统一为万元。"""
36
37     purchase: float = 300.0
38     medium: float = 3.0
39     major: float = 12.0
40     minor: float = 0.0
41
42
43 BASE_COSTS = CostConfig()
44
45

```

```
46 def ensure_output_dirs() -> None:
47     """只创建可重建的输出目录，不触碰原始附件。"""
48
49     for directory in (PROCESSED_DIR, RESULTS_DIR, FIGURES_DIR):
50         directory.mkdir(parents=True, exist_ok=True)
51
52
```

附录 3：数据读取与预处理源程序

src/filter_monitoring/io.py

```
1 """ 原始附件读取与结构校验。 """
2
3 from __future__ import annotations
4
5 from pathlib import Path
6
7 import pandas as pd
8
9 from .config import (
10     ASSETS,
11     EXPECTED_TELEMETRY_SHEETS,
12     MAINTENANCE_XLSX,
13     PROBLEM_DOCX,
14     TELEMETRY_XLSX,
15 )
16
17
18 def normalize_asset(value: object) -> str:
19     """ 把 A_1、a1 等写法统一为 A1。 """
20
21     text = str(value).strip().upper().replace("_", "").replace(" ", "")
22     return text
23
24
25 def audit_inputs() -> pd.DataFrame:
26     """ 检查三份输入是否存在，并校验 Excel 的基本结构。 """
27
28     rows: list[dict[str, object]] = []
29     for label, path in {
30         ("problem", PROBLEM_DOCX),
31         ("telemetry", TELEMETRY_XLSX),
32         ("maintenance", MAINTENANCE_XLSX),
33     }:
34         rows.append(
35             {
36                 "input": label,
37                 "path": str(path),
38                 "exists": path.is_file(),
39                 "size_bytes": path.stat().st_size if path.is_file() else 0,
40             }
41         )
42
43     report = pd.DataFrame(rows)
44     missing = report.loc[~report["exists"], "path"].tolist()
45     if missing:
46         raise FileNotFoundError("缺少原始附件: " + "; ".join(missing))
47
48     telemetry_sheets = tuple(pd.ExcelFile(TELEMETRY_XLSX).sheet_names)
49     if telemetry_sheets != EXPECTED_TELEMETRY_SHEETS:
50         raise ValueError(
51             "附件1工作表不符合预期: "
52             f"expected={EXPECTED_TELEMETRY_SHEETS}, actual={telemetry_sheets}"
53         )
54
55     maintenance_columns = set(pd.read_excel(MAINTENANCE_XLSX, nrows=3).columns)
56     required = {"编号", "日期", "维护类型"}
57     if not required.issubset(maintenance_columns):
58         raise ValueError(f"附件2缺少字段: {sorted(required - maintenance_columns)}")
59     return report
60
61
62 def load_telemetry(path: Path = TELEMETRY_XLSX) -> pd.DataFrame:
63     """ 读取 10 个工作表并合并，保留性能缺失值供质量审计。 """
64
65     frames: list[pd.DataFrame] = []
66     with pd.ExcelFile(path) as workbook:
67         for sheet in workbook.sheet_names:
68             frame = pd.read_excel(workbook, sheet_name=sheet, usecols=["time", "per"])
69             frame = frame.rename(columns={"time": "timestamp", "per": "performance"})
70             frame.insert(0, "asset", normalize_asset(sheet))
71             frame["timestamp"] = pd.to_datetime(frame["timestamp"], errors="coerce")
72             frame["performance"] = pd.to_numeric(frame["performance"], errors="coerce")
73             frames.append(frame)
74
75     telemetry = pd.concat(frames, ignore_index=True)
76     telemetry = telemetry.dropna(subset=["timestamp"])
77     telemetry = telemetry.sort_values(["asset", "timestamp"],
78                                     kind="stable").reset_index(drop=True)
79     unknown = sorted(set(telemetry["asset"]) - set(ASSETS))
80     if unknown:
81         raise ValueError(f"附件1出现未知设备: {unknown}")
82     return telemetry
83
84 def load_maintenance(path: Path = MAINTENANCE_XLSX) -> pd.DataFrame:
```

```

85     """读取维护记录，把中文维护类型转为稳定的英文枚举。"""
86
87     frame = pd.read_excel(path, usecols=["编号", "日期", "维护类型"])
88     frame = frame.rename(
89         columns={"编号": "asset", "日期": "maintenance_date", "维护类型": "maintenance_type"}
90     )
91     frame["asset"] = frame["asset"].map(normalize_asset)
92     frame["maintenance_date"] = pd.to_datetime(frame["maintenance_date"],
errors="coerce").dt.normalize()
93     type_map = {"中维护": "medium", "大维护": "major", "小维护": "minor"}
94     frame["maintenance_type_raw"] = frame["maintenance_type"]
95     frame["maintenance_type"]
= frame["maintenance_type"].astype(str).str.strip().map(type_map)
96     if frame[["asset", "maintenance_date", "maintenance_type"]].isna().any().any():
97         bad = frame[frame[["asset", "maintenance_date",
98 "maintenance_type"]].isna().any(axis=1)]
99         raise ValueError(f"附件2存在无法解析的维护记录，共 {len(bad)} 行")
100     unknown = sorted(set(frame["asset"]) - set(ASSETS))
101     if unknown:
102         raise ValueError(f"附件2出现未知设备: {unknown}")
103     return frame.sort_values(["asset", "maintenance_date"],
kind="stable").reset_index(drop=True)
104
105

```

src/filter_monitoring/preprocess.py

```

1     """把不规则高频监测值变为可建模的日级面板。"""
2
3     from __future__ import annotations
4
5     import numpy as np
6     import pandas as pd
7
8     from .config import SHORT_GAP_LIMIT_DAYS
9
10
11     def build_daily_panel(
12         telemetry: pd.DataFrame,
13         maintenance: pd.DataFrame,
14         short_gap_days: int = SHORT_GAP_LIMIT_DAYS,
15     ) -> pd.DataFrame:
16         """日聚合、短缺口插值并对齐维护事件。
17
18         `performance_median` 始终保存观测事实；`performance_model` 才允许填补不超过
19         `short_gap_days` 的内部短缺口，避免长期缺失被平滑掩盖。
20         """
21
22         raw = telemetry.copy()
23         raw["date"] = raw["timestamp"].dt.normalize()
24         grouped = raw.groupby(["asset", "date"], observed=True)["performance"]
25         daily = grouped.agg(
26             performance_mean="mean",
27             performance_median="median",
28             performance_std="std",
29             performance_min="min",
30             performance_max="max",
31             valid_rows="count",
32         )
33         daily["sample_rows"] = grouped.size()
34         daily["missing_rate"] = 1.0 - daily["valid_rows"] / daily["sample_rows"]
35         daily = daily.reset_index()
36
37         completed: list[pd.DataFrame] = []
38         for asset, asset_daily in daily.groupby("asset", sort=True):
39             index = pd.date_range(asset_daily["date"].min(), asset_daily["date"].max(),
freq="D")
40             block = asset_daily.set_index("date").reindex(index)
41             block.index.name = "date"
42             block["asset"] = asset
43             block["sample_rows"] = block["sample_rows"].fillna(0).astype(int)
44             block["valid_rows"] = block["valid_rows"].fillna(0).astype(int)
45             block["missing_rate"] = np.where(
46                 block["sample_rows"] > 0,
47                 1.0 - block["valid_rows"] / block["sample_rows"],
48                 1.0,
49             )
50             block["performance_model"] = block["performance_median"].interpolate(
51                 method="linear",
52                 limit=short_gap_days,
53                 limit_area="inside",
54             )
55             block["was_imputed"] = block["performance_median"].isna() &
block["performance_model"].notna()
56             completed.append(block.reset_index())
57
58         panel = pd.concat(completed, ignore_index=True)

```

```

59 event_rank = {"minor": 1, "medium": 2, "major": 3}
60 events = maintenance.copy()
61 events["event_rank"] = events["maintenance_type"].map(event_rank)
62 events = (
63     events.sort_values("event_rank")
64     .groupby(["asset", "maintenance_date"], as_index=False)
65     .tail(1)[["asset", "maintenance_date", "maintenance_type"]]
66     .rename(columns={"maintenance_date": "date"})
67 )
68 panel = panel.merge(events, how="left", on=["asset", "date"])
69 panel = panel.sort_values(["asset", "date"], kind="stable").reset_index(drop=True)
70 panel["last_maintenance_date"]
71
72 = panel["date"].where(panel["maintenance_type"].notna())
73 panel["last_maintenance_date"] = panel.groupby("asset")
74 ["last_maintenance_date"].ffill()
75 panel["days_since_maintenance"] = (
76     panel["date"] - panel["last_maintenance_date"]
77 ).dt.days.astype("Int64")
78 return panel
79
80 def data_quality_summary(telemetry: pd.DataFrame) -> pd.DataFrame:
81     """生成每台设备的数据范围、缺失率、重复和典型采样间隔。"""
82
83     rows: list[dict[str, object]] = []
84     for asset, block in telemetry.groupby("asset", sort=True):
85         cadence = block["timestamp"].sort_values().diff().dt.total_seconds().div(3600)
86         rows.append(
87             {
88                 "asset": asset,
89                 "start": block["timestamp"].min(),
90                 "end": block["timestamp"].max(),
91                 "rows": len(block),
92                 "valid_performance": int(block["performance"].notna().sum()),
93                 "missing_performance": int(block["performance"].isna().sum()),
94                 "missing_rate": float(block["performance"].isna().mean()),
95                 "duplicate_timestamps": int(block["timestamp"].duplicated().sum()),
96                 "median_cadence_hours": float(cadence.median()),
97             }
98         )
99     return pd.DataFrame(rows)
100
101

```


附录 4：问题一完整源程序

src/filter_monitoring/q1_analysis.py

```
1 """问题一：数据质量、周期退化规律和维护事件效应。"""
2
3 from __future__ import annotations
4
5 import json
6 import os
7 import tempfile
8 from pathlib import Path
9 from typing import Any, Dict, Iterable, Tuple
10
11 # Matplotlib 必须在导入 pyplot 前指定可写缓存目录。
12 _CACHE_ROOT = Path(tempfile.gettempdir()) / "math2026-filter-monitoring"
13 os.environ.setdefault("MPLCONFIGDIR", str(_CACHE_ROOT / "matplotlib"))
14 os.environ.setdefault("XDG_CACHE_HOME", str(_CACHE_ROOT / "xdg"))
15
16 import matplotlib
17
18 matplotlib.use("Agg")
19
20 import matplotlib.dates as mdates
21 import matplotlib.pyplot as plt
22 import numpy as np
23 import pandas as pd
24 import seaborn as sns
25 import statsmodels.formula.api as smf
26 from matplotlib.font_manager import FontProperties, fontManager
27 from matplotlib.lines import Line2D
28
29 from config import (
30     ASSETS,
31     BASE_COSTS,
32     FIGURES_DIR,
33     LIFETIME_THRESHOLD,
34     PROCESSED_DIR,
35     PROJECT_DIR,
36     RESULTS_DIR,
37     ensure_output_dirs,
38 )
39 from .io import load_maintenance, load_telemetry
40 from .preprocess import build_daily_panel, data_quality_summary
41
42 ANNUAL_PERIOD_DAYS = 365.25
43 BOOTSTRAP_SEED = 20260722
44 BOOTSTRAP_REPETITIONS = 2000
45
46
47 def maintenance_summary(maintenance: pd.DataFrame) -> pd.DataFrame:
48     """统计各设备各类有记录维护次数。"""
49
50     return (
51         maintenance.groupby(["asset", "maintenance_type"], observed=True)
52         .size()
53         .rename("events")
54         .reset_index()
55         .sort_values(["asset", "maintenance_type"])
56     )
57
58
59 def run_q1_scaffold() -> Dict[str, pd.DataFrame]:
60     """生成问题一确定性数据层，不提前宣称统计模型已经完成。"""
61
62     ensure_output_dirs()
63     telemetry = load_telemetry()
64     maintenance = load_maintenance()
65     panel = build_daily_panel(telemetry, maintenance)
66     quality = data_quality_summary(telemetry)
67     events = maintenance_summary(maintenance)
68
69     panel.to_csv(PROCESSED_DIR / "daily_panel.csv", index=False, date_format="%Y-%m-%d")
70     quality.to_csv(RESULTS_DIR / "q1_data_quality.csv", index=False)
71     events.to_csv(RESULTS_DIR / "q1_maintenance_summary.csv", index=False)
72     return {"daily_panel": panel, "data_quality": quality, "maintenance_summary": events}
73
74
75 def prepare_analysis_panel(panel: pd.DataFrame) -> pd.DataFrame:
76     """构造可解释的趋势、季节和维护周期特征。"""
77
78     required = {
79         "asset",
80         "date",
81         "performance_model",
82         "performance_median",
83         "valid_rows",
84         "was_imputed",
85         "days_since_maintenance",
86     }
```

```

87     "maintenance_type",
88 }
89 missing = required - set(panel.columns)
90 if missing:
91     raise ValueError(f"问题一面板缺少字段: {sorted(missing)}")
92
93 out = panel.copy()
94 out["date"] = pd.to_datetime(out["date"], errors="coerce")
95 out["asset"] = pd.Categorical(out["asset"], categories=ASSETS, ordered=True)
96 out = out.dropna(subset=["date", "asset"]).sort_values(["asset",
"date"]).reset_index(drop=True)
97
98 start = out["date"].min()
99 out["time_days"] = (out["date"] - start).dt.days.astype(float)
100 out["time_years"] = out["time_days"] / ANNUAL_PERIOD_DAYS
101 phase = 2.0 * np.pi * out["date"].dt.dayofyear.astype(float) / ANNUAL_PERIOD_DAYS
102 out["sin_1"] = np.sin(phase)
103 out["cos_1"] = np.cos(phase)
104 out["sin_2"] = np.sin(2.0 * phase)
105 out["cos_2"] = np.cos(2.0 * phase)
106 out["month"] = out["date"].dt.month.astype(int)
107
108 since = pd.to_numeric(out["days_since_maintenance"], errors="coerce")
109 out["has_maintenance_history"] = since.notna().astype(float)
110 since = since.fillna(0.0).clip(lower=0.0)
111 out["fouling_0_30"] = since.clip(upper=30.0) / 30.0
112 out["fouling_30_90"] = (since - 30.0).clip(lower=0.0, upper=60.0) / 60.0
113
114 valid_rows = pd.to_numeric(out["valid_rows"], errors="coerce").fillna(0.0)
115 # 四个有效样本已足以形成稳健日中位数。避免给小时采样阶段过高权重。
116 out["quality_weight"] = np.clip(valid_rows / 4.0, 0.25, 1.0)
117 out.loc[out["was_imputed"], "quality_weight"] = 0.5
118 out.loc[out["performance_model"].isna(), "quality_weight"] = 0.0
119 return out
120
121
122 def fit_panel_models(panel: pd.DataFrame) -> Dict[str, Any]:
123     """拟合趋势季节基线并加入维护周期后的完整解释模型。"""
124
125     model_data = panel.loc[panel["performance_model"].notna()].copy()
126     if model_data["asset"].nunique() < 2 or len(model_data) < 100:
127         raise ValueError("问题一正式模型至少需要两个设备和 100 个有效日观测")
128
129     trend_formula = (
130         "performance_model ~ 0 + C(asset) + C(asset):time_years + "
131         "sin_1 + cos_1 + sin_2 + cos_2"
132     )
133     full_formula = (
134         trend_formula
135         + " + has_maintenance_history + fouling_0_30 + fouling_30_90"
136     )
137     month_trend_formula = (
138         "performance_model ~ 0 + C(asset) + C(asset):time_years + C(month)"
139     )
140     month_full_formula = (
141         month_trend_formula
142         + " + has_maintenance_history + fouling_0_30 + fouling_30_90"
143     )
144
145     def fit(formula: str) -> Any:
146         return smf.wls(
147             formula,
148             data=model_data,
149             weights=model_data["quality_weight"],
150         ).fit(
151             cov_type="cluster",
152             cov_kws={"groups": model_data["asset"].astype(str), "use_correction": True},
153         )
154
155     return {
156         "trend_season": fit(trend_formula),
157         "full": fit(full_formula),
158         "month_trend": fit(month_trend_formula),
159         "month_full": fit(month_full_formula),
160         "data": model_data,
161     }
162
163
164 def seasonal_curve(model: Any) -> pd.DataFrame:
165     days = np.arange(1, 367, dtype=float)
166     phase = 2.0 * np.pi * days / ANNUAL_PERIOD_DAYS
167     names = ["sin_1", "cos_1", "sin_2", "cos_2"]
168     design = np.column_stack(
169         [np.sin(phase), np.cos(phase), np.sin(2.0 * phase), np.cos(2.0 * phase)]
170     )
171     beta = model.params.reindex(names).to_numpy(dtype=float)
172     covariance = model.cov_params().loc[names, names].to_numpy(dtype=float)
173     # 显式逐项求和，规避部分 macOS Accelerate/NumPy 组合在重复自助抽样后
174     # 对小矩阵 matmul 产生的伪浮点告警。
175     effect = np.sum(design * beta[np.newaxis, :], axis=1)
176     effect = effect - effect.mean()
177     standard_error = np.sqrt(np.maximum(np.einsum("ij,jk,ik->i", design, covariance,
design), 0.0))
178     return pd.DataFrame(

```

```

179         {
180             "day_of_year": days.astype(int),
181             "seasonal_effect": effect,
182             "ci95_low": effect - 1.96 * standard_error,
183             "ci95_high": effect + 1.96 * standard_error,
184         }
185     )
186
187
188 def _asset_parameter(model: Any, asset: str, suffix: str) -> Tuple[float, float, float]:
189     candidates = [name for name in model.params.index if f"[{asset}]" in name and
190 name.endswith(suffix)]
191     if len(candidates) != 1:
192         raise KeyError(f"无法唯一定位设备 {asset} 的参数 {suffix}: {candidates}")
193     name = candidates[0]
194     interval = model.conf_int().loc[name]
195     return float(model.params[name]), float(interval.iloc[0]), float(interval.iloc[1])
196
197 def build_device_metrics(panel: pd.DataFrame, models: Dict[str, Any]) -> pd.DataFrame:
198     """生成连接问题一与问题二的设备级指标。"""
199
200     work = panel.copy()
201     model_data = work.loc[work["performance_model"].notna()].copy()
202     model_data["full_prediction"] = models["full"].predict(model_data)
203     model_data["full_residual"] = model_data["performance_model"] -
204 model_data["full_prediction"]
205     season = _seasonal_curve(models["full"])
206     seasonal_amplitude = float((season["seasonal_effect"].max() -
207 season["seasonal_effect"].min()) / 2.0)
208
209     rows = []
210     for asset in ASSETS:
211         block = work.loc[work["asset"].astype(str) == asset].sort_values("date").copy()
212         fitted = model_data.loc[model_data["asset"].astype(str) == asset]
213         rolling = block["performance_model"].rolling(365, min_periods=330).mean()
214         latest_rolling = float(rolling.dropna().iloc[-1]) if rolling.notna().any()
215         else np.nan
216         slope, slope_low, slope_high = _asset_parameter(models["full"], asset,
217 ":time_years")
218         rows.append(
219             {
220                 "asset": asset,
221                 "start": block["date"].min(),
222                 "end": block["date"].max(),
223                 "calendar_days": len(block),
224                 "observed_days": int(block["performance_model"].notna().sum()),
225                 "model_days": int(block["performance_model"].notna().sum()),
226                 "imputed_days": int(block["was_imputed"].fillna(False).sum()),
227                 "mean_performance": float(block["performance_model"].mean()),
228                 "latest_rolling_365_mean": latest_rolling,
229                 "threshold_margin": latest_rolling - LIFETIME_THRESHOLD,
230                 "adjusted_linear_trend_per_year": slope,
231                 "trend_ci95_low": slope_low,
232                 "trend_ci95_high": slope_high,
233                 "common_seasonal_amplitude": seasonal_amplitude,
234                 "residual_rmse":
235 float(np.sqrt(np.mean(np.square(fitted["full_residual"])))),
236             }
237         )
238     return pd.DataFrame(rows)
239
240 def maintenance_event_effects(panel: pd.DataFrame, trend_model: Any) -> pd.DataFrame:
241     """在去除设备趋势和公共季节项后计算每次维护的恢复与保持量。"""
242
243     work = panel.copy()
244     valid = work["performance_model"].notna()
245     work.loc[valid, "trend_season_prediction"] = trend_model.predict(work.loc[valid])
246     work["adjusted_performance"] = work["performance_model"] -
247 work["trend_season_prediction"]
248
249     rows = []
250     events = work.loc[work["maintenance_type"].notna(), ["asset", "date",
251 "maintenance_type"]]
252     for event in events.itertuples(index=False):
253         asset = str(event.asset)
254         event_date = pd.Timestamp(event.date)
255         block = work.loc[work["asset"].astype(str) == asset].copy()
256         delta = (block["date"] - event_date).dt.days
257         pre = block.loc[delta.between(-7, -1)]
258         post_7 = block.loc[delta.between(1, 7)]
259         post_30 = block.loc[delta.between(8, 30)]
260         other_events = block.loc[
261             block["maintenance_type"].notna() & (block["date"] != event_date), "date"
262         ]
263         overlap = bool(((other_events - event_date).dt.days.between(-7, 30)).any())
264         pre_adjusted = pre["adjusted_performance"].dropna()
265         post_7_adjusted = post_7["adjusted_performance"].dropna()

```

```

262     post_30_adjusted = post_30["adjusted_performance"].dropna()
263     pre_raw = pre["performance_model"].dropna()
264     post_7_raw = post_7["performance_model"].dropna()
265     post_30_raw = post_30["performance_model"].dropna()
266
267     usable_instant = len(pre_adjusted) >= 4 and len(post_7_adjusted) >= 4 and
not overlap
268     usable_retained = len(pre_adjusted) >= 4 and len(post_30_adjusted) >= 10 and
not overlap
269     rows.append(
270         {
271             "asset": asset,
272             "maintenance_date": event_date,
273             "maintenance_type": str(event.maintenance_type),
274             "overlap_with_other_event": overlap,
275             "n_pre": len(pre_adjusted),
276             "n_post_1_7": len(post_7_adjusted),
277             "n_post_8_30": len(post_30_adjusted),
278             "instant_gain_adjusted": (
279                 float(post_7_adjusted.mean() - pre_adjusted.mean()) if usable_instant
280             else np.nan
281         ),
282         "retained_gain_adjusted": (
283             float(post_30_adjusted.mean() - pre_adjusted.mean()) if usable_retained
284             else np.nan
285         ),
286         "instant_gain_raw": (
287             float(post_7_raw.mean() - pre_raw.mean()) if usable_instant else np.nan
288         ),
289         "retained_gain_raw": (
290             float(post_30_raw.mean() - pre_raw.mean()) if usable_retained
291             else np.nan
292         ),
293     )
294     return pd.DataFrame(rows)
295
296 def cluster_bootstrap_ci(
297     frame: pd.DataFrame,
298     value_col: str,
299     repetitions: int = BOOTSTRAP_REPETITIONS,
300     seed: int = BOOTSTRAP_SEED,
301 ) -> Tuple[float, float]:
302     usable = frame.dropna(subset=[value_col]).copy()
303     clusters = usable["asset"].drop_duplicates().tolist()
304     if not clusters:
305         return np.nan, np.nan
306
307     rng = np.random.default_rng(seed)
308     grouped = {asset: usable.loc[usable["asset"] == asset, value_col].to_numpy() for
asset in clusters}
309     estimates = np.empty(repetitions, dtype=float)
310     for index in range(repetitions):
311         sampled_clusters = rng.choice(clusters, size=len(clusters), replace=True)
312         values = np.concatenate([grouped[asset] for asset in sampled_clusters])
313         estimates[index] = np.median(values)
314     low, high = np.quantile(estimates, [0.025, 0.975])
315     return float(low), float(high)
316
317 def summarize_maintenance_effects(detail: pd.DataFrame) -> pd.DataFrame:
318     """维护护类型汇总调整后恢复量，并给出设备族自助置信区间。"""
319
320     rows = []
321     costs = {"medium": BASE_COSTS.medium, "major": BASE_COSTS.major, "minor":
BASE_COSTS.minor}
322     for maintenance_type in ("medium", "major"):
323         group = detail.loc[detail["maintenance_type"] == maintenance_type].copy()
324         instant = group["instant_gain_adjusted"].dropna()
325         retained = group["retained_gain_adjusted"].dropna()
326         instant_low, instant_high = _cluster_bootstrap_ci(group, "instant_gain_adjusted")
327         retained_low, retained_high = _cluster_bootstrap_ci(
328             group,
329             "retained_gain_adjusted",
330             seed=BOOTSTRAP_SEED + 1,
331         )
332         retained_median = float(retained.median()) if len(retained) else np.nan
333         rows.append(
334             {
335                 "maintenance_type": maintenance_type,
336                 "recorded_events": len(group),
337                 "usable_instant_events": len(instant),
338                 "usable_retained_events": len(retained),
339                 "instant_mean": float(instant.mean()) if len(instant) else np.nan,
340                 "instant_median": float(instant.median()) if len(instant) else np.nan,
341                 "instant_ci95_low": instant_low,
342                 "instant_ci95_high": instant_high,
343                 "instant_positive_rate": float((instant > 0).mean()) if len(instant)
344             else np.nan,
345                 "retained_mean": float(retained.mean()) if len(retained) else np.nan,

```

```

345         "retained_median": retained_median,
346         "retained_ci95_low": retained_low,
347         "retained_ci95_high": retained_high,
348         "retained_positive_rate": float((retained > 0).mean()) if len(retained)
else np.nan,
349         "retained_gain_per_10k_cost": retained_median / costs[maintenance_type],
350     }
351 )
352 return pd.DataFrame(rows)
353
354
355 def model_diagnostics(models: Dict[str, Any]) -> pd.DataFrame:
356     rows = []
357     for name in ("trend_season", "full", "month_trend", "month_full"):
358         model = models[name]
359         residual = np.asarray(model.resid, dtype=float)
360         rows.append(
361             {
362                 "model": name,
363                 "nobs": int(model.nobs),
364                 "r_squared": float(model.rsquared),
365                 "adjusted_r_squared": float(model.rsquared_adj),
366                 "rmse": float(np.sqrt(np.mean(np.square(residual)))),
367                 "aic": float(model.aic),
368                 "bic": float(model.bic),
369                 "formula": model.model.formula,
370             }
371         )
372     return pd.DataFrame(rows)
373
374
375 def model_parameters(models: Dict[str, Any]) -> pd.DataFrame:
376     """导出模型系数、聚类稳健标准误和置信区间，保证论文数字可追溯。"""
377
378     rows = []
379     for model_name in ("trend_season", "full", "month_trend", "month_full"):
380         model = models[model_name]
381         intervals = model.conf_int()
382         for term in model.params.index:
383             rows.append(
384                 {
385                     "model": model_name,
386                     "term": term,
387                     "estimate": float(model.params[term]),
388                     "cluster_robust_std_error": float(model.bse[term]),
389                     "ci95_low": float(intervals.loc[term].iloc[0]),
390                     "ci95_high": float(intervals.loc[term].iloc[1]),
391                     "p_value": float(model.pvalues[term]),
392                 }
393             )
394     return pd.DataFrame(rows)
395
396
397 def detect_residual_outliers(
398     panel: pd.DataFrame,
399     full_model: Any,
400     threshold: float = 3.5,
401 ) -> pd.DataFrame:
402     """在拟合季节、趋势和维护周期后，用设备内 MAD 标记残差异常。"""
403
404     work = panel.loc[panel["performance_model"].notna()].copy()
405     work["prediction"] = full_model.predict(work)
406     work["residual"] = work["performance_model"] - work["prediction"]
407     parts = []
408     for _, block in work.groupby("asset", observed=True):
409         center = float(block["residual"].median())
410         mad = float((block["residual"] - center).abs().median())
411         scale = max(1.4826 * mad, 1e-9)
412         out = block.copy()
413         out["robust_z"] = (out["residual"] - center) / scale
414         out["is_residual_outlier"] = out["robust_z"].abs() > threshold
415     parts.append(out)
416     columns = [
417         "asset",
418         "date",
419         "performance_model",
420         "prediction",
421         "residual",
422         "robust_z",
423         "maintenance_type",
424         "was_imputed",
425         "is_residual_outlier",
426     ]
427     return pd.concat(parts, ignore_index=True)[columns]
428
429
430 def configure_plot_style() -> None:
431     font_path = PROJECT_DIR / "paper" / "fonts" / "SimHei.ttf"
432     font_name = "sans-serif"
433     if font_path.is_file():
434         fontManager.addfont(str(font_path))
435         font_name = FontProperties(fname=str(font_path)).get_name()
436     sns.set_theme(style="whitegrid", context="notebook")
437     plt.rcParams.update(
438         {

```

```

439         "font.family": font_name,
440         "axes.unicode_minus": False,
441         "figure.dpi": 120,
442         "savefig.dpi": 240,
443         "axes.titleweight": "bold",
444     }
445 )
446
447
448 def save_figure(fig: Any, name: str) -> Path:
449     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
450     path = FIGURES_DIR / name
451     fig.savefig(path, bbox_inches="tight", facecolor="white")
452     plt.close(fig)
453     return path
454
455
456 def plot_device_overview(panel: pd.DataFrame) -> Path:
457     fig, axes = plt.subplots(5, 2, figsize=(14, 18), sharex=True)
458     for ax, asset in zip(axes.ravel(), ASSETS):
459         block = panel.loc[panel["asset"].astype(str) == asset].sort_values("date")
460         ax.plot(block["date"], block["performance_model"], color="#9aa0a6", lw=0.7,
461               alpha=0.55)
462         rolling = block["performance_model"].rolling(30, min_periods=15).median()
463         ax.plot(block["date"], rolling, color="#2457a7", lw=1.8)
464         for event in block.loc[block["maintenance_type"].notna()].itertuples(index=False):
465             color = "#d95f02" if event.maintenance_type == "major" else "#f2a900"
466             ax.axvline(event.date, color=color, alpha=0.28, lw=0.8)
467         ax.set_title(asset)
468         ax.set_ylabel("透水性")
469         ax.xaxis.set_major_locator(mdates.MonthLocator(interval=4))
470         ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
471     for ax in axes[-1]:
472         ax.tick_params(axis="x", rotation=30)
473     handles = [
474         Line2D([0], [0], color="#2457a7", lw=2, label="30日滚动中位数"),
475         Line2D([0], [0], color="#f2a900", lw=2, label="中维护"),
476         Line2D([0], [0], color="#d95f02", lw=2, label="大维护"),
477     ]
478     fig.suptitle("十台设备日透水性与维护记录", fontsize=18, y=0.995)
479     fig.legend(
480         handles=handles,
481         loc="upper center",
482         bbox_to_anchor=(0.5, 0.978),
483         ncol=3,
484         frameon=False,
485     )
486     fig.tight_layout(rect=(0.0, 0.0, 1.0, 0.95))
487     return _save_figure(fig, "q1_01_device_overview.png")
488
489 def plot_seasonal_curve(seasonal: pd.DataFrame) -> Path:
490     fig, ax = plt.subplots(figsize=(10, 5.5))
491     x = seasonal["day_of_year"].to_numpy()
492     ax.plot(x, seasonal["seasonal_effect"], color="#2457a7", lw=2.2, label="公共季节效应")
493     ax.fill_between(
494         x,
495         seasonal["ci95_low"].to_numpy(),
496         seasonal["ci95_high"].to_numpy(),
497         color="#2457a7",
498         alpha=0.18,
499         label="95%置信区间",
500     )
501     month_starts = pd.date_range("2025-01-01", "2025-12-01", freq="MS")
502     ax.set_xticks(month_starts.dayofyear)
503     ax.set_xticklabels([f"{month}月" for month in range(1, 13)])
504     ax.axhline(0.0, color="black", lw=0.8)
505     ax.set_xlabel("年内日期")
506     ax.set_ylabel("季节调整量")
507     ax.set_title("控制设备趋势与维护周期后的公共季节效应")
508     ax.legend(frameon=False)
509     fig.tight_layout()
510     return _save_figure(fig, "q1_02_seasonal_curve.png")
511
512
513 def plot_maintenance_effects(detail: pd.DataFrame) -> Path:
514     long = detail.melt(
515         id_vars=["asset", "maintenance_type"],
516         value_vars=["instant_gain_adjusted", "retained_gain_adjusted"],
517         var_name="effect_window",
518         value_name="gain",
519     ).dropna(subset=["gain"])
520     long["维护类型"] = long["maintenance_type"].map({"medium": "中维护", "major": "大维护"})
521     long["观察窗口"] = long["effect_window"].map(
522         {"instant_gain_adjusted": "维护后1-7日", "retained_gain_adjusted": "维护后8-30日"}
523     )
524     fig, ax = plt.subplots(figsize=(9, 5.8))
525     sns.boxplot(
526         data=long,
527         x="观察窗口",
528         y="gain",
529         hue="维护类型",
530         showfliers=False,

```



```

531     palette={"中维护": "#f2a900", "大维护": "#d95f02"},
532     ax=ax,
533 )
534 sns.stripplot(
535     data=long,
536     x="观察窗口",
537     y="gain",
538     hue="维护类型",
539     dodge=True,
540     alpha=0.45,
541     size=3,
542     palette={"中维护": "#f2a900", "大维护": "#d95f02"},
543     legend=False,
544     ax=ax,
545 )
546 ax.axhline(0.0, color="black", lw=0.9)
547 ax.set_xlabel("")
548 ax.set_ylabel("去趋势、去季节后的透水性变化")
549 ax.set_title("维护事件调整后恢复量与保持量")
550 ax.legend(title="维护类型", frameon=False)
551 fig.tight_layout()
552 return _save_figure(fig, "q1_03_maintenance_effects.png")
553
554
555 def plot_device_metrics(metrics: pd.DataFrame) -> Path:
556     ordered = metrics.set_index("asset").reindex(ASSETS).reset_index()
557     x = np.arange(len(ordered))
558     fig, axes = plt.subplots(2, 1, figsize=(11, 8), sharex=True)
559     trend = ordered["adjusted_linear_trend_per_year"].to_numpy()
560     low = ordered["trend_ci95_low"].to_numpy()
561     high = ordered["trend_ci95_high"].to_numpy()
562     axes[0].bar(x, trend, color=np.where(trend < 0, "#c44e52", "#4c72b0"), alpha=0.85)
563     axes[0].errorbar(x, trend, yerr=np.vstack([trend - low, high - trend]), fmt="none",
564                     ecolor="black", capsize=3)
565     axes[0].axhline(0.0, color="black", lw=0.8)
566     axes[0].set_ylabel("年变化量")
567     axes[0].set_title("控制季节和维护周期后的设备线性趋势 (95%置信区间)")
568
569     margin = ordered["threshold_margin"].to_numpy()
570     axes[1].bar(x, margin, color="#55a868", alpha=0.85)
571     axes[1].axhline(0.0, color="black", lw=0.8)
572     axes[1].set_ylabel("滚动年均值减37")
573     axes[1].set_title("当前滚动365日性能阈值裕度")
574     axes[1].set_xticks(x)
575     axes[1].set_xticklabels(ordered["asset"])
576     fig.tight_layout()
577     return _save_figure(fig, "q1_04_device_metrics.png")
578
579 def plot_sampling_profile(panel: pd.DataFrame) -> Path:
580     profile = panel.groupby("date", as_index=False)["valid_rows"].median()
581     profile["rolling_14"] = profile["valid_rows"].rolling(I4, min_periods=7).median()
582     fig, ax = plt.subplots(figsize=(11, 4.8))
583     ax.plot(profile["date"], profile["valid_rows"], color="#9aa0a6", alpha=0.35, lw=0.7)
584     ax.plot(profile["date"], profile["rolling_14"], color="#8172b2", lw=2.0)
585     ax.set_ylabel("每日有效样本数中位数")
586     ax.set_xlabel("日期")
587     ax.set_title("采样频率变化: 十台设备每日有效记录数")
588     ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
589     ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
590     ax.tick_params(axis="x", rotation=30)
591     fig.tight_layout()
592     return _save_figure(fig, "q1_05_sampling_profile.png")
593
594
595 def write_key_findings(
596     metrics: pd.DataFrame,
597     seasonal: pd.DataFrame,
598     maintenance: pd.DataFrame,
599     diagnostics: pd.DataFrame,
600     outliers: pd.DataFrame,
601     figure_paths: Iterable[Path],
602 ) -> Dict[str, Any]:
603     peak = seasonal.loc[seasonal["seasonal_effect"].idxmax()]
604     trough = seasonal.loc[seasonal["seasonal_effect"].idxmin()]
605     findings = Dict[str, Any] = {
606         "analysis_version": "q1-formal-v1",
607         "bootstrap_seed": BOOTSTRAP_SEED,
608         "bootstrap_repetitions": BOOTSTRAP_REPETITIONS,
609         "device_count": int(metrics["asset"].nunique()),
610         "annual_seasonal_amplitude": float(metrics["common_seasonal_amplitude"].iloc[0]),
611         "seasonal_peak_day_of_year": int(peak["day_of_year"]),
612         "seasonal_trough_day_of_year": int(trough["day_of_year"]),
613         "full_model_r_squared": float(
614             diagnostics.loc[diagnostics["model"] == "full", "r_squared"].iloc[0]
615         ),
616         "residual_outlier_count": int(outliers["is_residual_outlier"].sum()),
617         "residual_outlier_rate": float(outliers["is_residual_outlier"].mean()),
618         "maintenance_effects": maintenance.to_dict(orient="records"),
619         "figures": [path.name for path in figure_paths],
620         "interpretation_limit": (
621             "维护效果已控制公共季节与设备趋势, 但仍属于观察性事件研究, "
622             "不能排除未记录小维护和指征偏差。"

```

```

623     ),
624 }
625 with (RESULTS_DIR / "q1_key_findings.json").open("w", encoding="utf-8") as handle:
626     json.dump(findings, handle, ensure_ascii=False, indent=2)
627     return findings
628
629
630 def run_q1_formal() -> Dict[str, Any]:
631     """运行问题一正式分析，生成可供论文和问题二复用的结果。"""
632
633     base = run_q1_scaffold()
634     panel = prepare_analysis_panel(base["daily_panel"])
635     models = fit_panel_models(panel)
636     metrics = build_device_metrics(panel, models)
637     event_detail = maintenance_event_effects(panel, models["trend_season"])
638     event_summary = summarize_maintenance_effects(event_detail)
639     month_event_detail = maintenance_event_effects(panel, models["month_trend"])
640     month_event_summary = summarize_maintenance_effects(month_event_detail)
641     event_robustness = pd.concat(
642         [
643             event_summary.assign(season_specification="fourier"),
644             month_event_summary.assign(season_specification="month_fixed_effect"),
645         ],
646         ignore_index=True,
647     )
648     diagnostics = model_diagnostics(models)
649     parameters = model_parameters(models)
650     seasonal = seasonal_curve(models["full"])
651     outliers = detect_residual_outliers(panel, models["full"])
652
653     metrics.to_csv(RERESULTS_DIR / "q1_device_metrics.csv", index=False, date_format="%Y-
654 %m-%d")
655     event_detail.to_csv(
656         RESULTS_DIR / "q1_event_effects_detail.csv",
657         index=False,
658         date_format="%Y-%m-%d",
659     )
660     event_summary.to_csv(RERESULTS_DIR / "q1_maintenance_effects.csv", index=False)
661     event_robustness.to_csv(
662         RESULTS_DIR / "q1_maintenance_effects_robustness.csv",
663         index=False,
664     )
665     diagnostics.to_csv(RERESULTS_DIR / "q1_model_diagnostics.csv", index=False)
666     parameters.to_csv(RERESULTS_DIR / "q1_model_parameters.csv", index=False)
667     seasonal.to_csv(RERESULTS_DIR / "q1_seasonal_curve.csv", index=False)
668     outliers.loc[outliers["is_residual_outlier"]].to_csv(
669         RESULTS_DIR / "q1_residual_outliers.csv",
670         index=False,
671         date_format="%Y-%m-%d",
672     )
673
674     figure_paths = [
675         plot_device_overview(panel),
676         plot_seasonal_curve(seasonal),
677         plot_maintenance_effects(event_detail),
678         plot_device_metrics(metrics),
679         plot_sampling_profile(panel),
680     ]
681     findings = _write_key_findings(
682         metrics,
683         seasonal,
684         event_summary,
685         diagnostics,
686         outliers,
687         figure_paths,
688     )
689     return {
690         **base,
691         "analysis_panel": panel,
692         "models": models,
693         "device_metrics": metrics,
694         "event_detail": event_detail,
695         "maintenance_effects": event_summary,
696         "maintenance_effects_robustness": event_robustness,
697         "model_diagnostics": diagnostics,
698         "model_parameters": parameters,
699         "seasonal_curve": seasonal,
700         "residual_outliers": outliers,
701         "figures": figure_paths,
702         "key_findings": findings,
703     }
704

```


附录 5：问题二完整源程序

src/filter_monitoring/q2_lifetime.py

```
1  """问题二：现行维护规律下的分层退化状态模型与寿命外推。"""
2
3  from __future__ import annotations
4
5  import json
6  import os
7  import tempfile
8  import warnings
9  from dataclasses import dataclass
10 from pathlib import Path
11 from typing import Any, Dict, Iterable, Sequence
12
13 # Matplotlib 必须在导入 pyplot 前指定可写缓存目录。
14 CACHE_ROOT = Path(tempfile.gettempdir()) / "math2026-filter-monitoring"
15 os.environ.setdefault("MPLCONFIGDIR", str(CACHE_ROOT / "matplotlib"))
16 os.environ.setdefault("XDG_CACHE_HOME", str(CACHE_ROOT / "xdg"))
17
18 import matplotlib.dates as mdates
19 import matplotlib.pyplot as plt
20 import numpy as np
21 import pandas as pd
22 import seaborn as sns
23 from scipy.linalg import pinvh
24 from scipy.optimize import lsq_linear
25 from statsmodels.tools.sm_exceptions import ConvergenceWarning, ValueWarning
26 from statsmodels.tsa.statespace.sarimax import SARIMAX
27
28 from config import (
29     ASSETS,
30     COMMISSION_DATE,
31     FIGURES_DIR,
32     LIFETIME_THRESHOLD,
33     PROCESSED_DIR,
34     RESULTS_DIR,
35     ROLLING_YEAR_DAYS,
36     ensure_output_dirs,
37 )
38 from .io import load_maintenance
39 from .ql_analysis import (
40     ANNUAL_PERIOD_DAYS,
41     _configure_plot_style,
42     _save_figure,
43     Fit_panel_models,
44     prepare_analysis_panel,
45     run_ql_scaffold,
46 )
47
48
49 FORECAST_HORIZON_YEARS = 30
50 SIMULATION_PATHS = 2000
51 SIMULATION_SEED = 20260724
52 STATE_SHRINKAGE = 1600.0
53 TRANSIENT_TIME_YEARS = 2.0
54 TRANSIENT_PENALTY = 5.0
55 PRIOR_WEIGHT = 20.0
56 BACKTEST_HORIZONS_DAYS = (270, 180, 90)
57 RECOVERY_LEVELS = (35.0, 37.0, 39.0)
58 MODEL_FEATURES = (
59     "sin_1",
60     "cos_1",
61     "sin_2",
62     "cos_2",
63     "has_maintenance_history",
64     "fouling_0_30",
65     "fouling_30_90",
66 )
67
68
69 @dataclass(frozen=True)
70 class HierarchicalStateModel:
71     """带非负长期退化率、设备部分汇聚和饱和早期漂移的状态模型。"""
72
73     assets: tuple[str, ...]
74     reference_date: pd.Timestamp
75     transient_time_years: float
76     shrinkage: float
77     transient_penalty: float
78     parameter_names: tuple[str, ...]
79     params: np.ndarray
80     covariance: np.ndarray
81     residual_std: float
82     residual_ar1: float
83     nobs: int
84     weighted_rmse: float
85     r_squared: float
86     prior_degradation_per_year: float
```

```

87
88     def parameter(self, name: str) -> float:
89         return float(self.params[self.parameter_names.index(name)])
90
91     def predict(self, frame: pd.DataFrame) -> np.ndarray:
92         design = _state_design(
93             frame,
94             assets=self.assets,
95             reference_date=self.reference_date,
96             transient_time_years=self.transient_time_years,
97             parameter_names=self.parameter_names,
98         )
99         return np.sum(design * self.params[np.newaxis, :], axis=1)
100
101
102     def add_rolling_annual_mean(
103         forecast: pd.DataFrame,
104         value_col: str = "performance_forecast",
105     ) -> pd.DataFrame:
106         """为每台设备的日预测添加滚动 365 日平均值。"""
107
108         required = {"asset", "date", value_col}
109         missing = required - set(forecast.columns)
110         if missing:
111             raise ValueError(f"寿命预测输入缺少字段: {sorted(missing)}")
112         out = forecast.sort_values(["asset", "date"]).copy()
113         out["rolling_annual_mean"] = out.groupby("asset")[value_col].transform(
114             lambda series: series.rolling(
115                 ROLLING_YEAR_DAYS,
116                 min_periods=ROLLING_YEAR_DAYS,
117             ).mean()
118         )
119         return out
120
121
122     def first_threshold_crossing(
123         forecast: pd.DataFrame,
124         threshold: float = LIFETIME_THRESHOLD,
125     ) -> pd.DataFrame:
126         """返回滚动年均首次低于阈值的候选日期。"""
127
128         crossed = forecast.loc[forecast["rolling_annual_mean"] < threshold]
129         return (
130             crossed.groupby("asset", as_index=False)["date"]
131             .min()
132             .rename(columns={"date": "candidate_threshold_date"})
133         )
134
135
136     def _asset_strings(series: pd.Series) -> pd.Series:
137         return series.astype("object").astype(str)
138
139
140     def _ensure_state_features(
141         frame: pd.DataFrame,
142         reference_date: pd.Timestamp,
143     ) -> pd.DataFrame:
144         out = frame.copy()
145         out["date"] = pd.to_datetime(out["date"], errors="coerce")
146         out["asset"] = _asset_strings(out["asset"])
147         out["time_years"] = (
148             (out["date"] - pd.Timestamp(reference_date)).dt.days.astype(float)
149             / ANNUAL_PERIOD_DAYS
150         )
151         phase = 2.0 * np.pi * out["date"].dt.dayofyear.astype(float) / ANNUAL_PERIOD_DAYS
152         out["sin_1"] = np.sin(phase)
153         out["cos_1"] = np.cos(phase)
154         out["sin_2"] = np.sin(2.0 * phase)
155         out["cos_2"] = np.cos(2.0 * phase)
156
157         since = pd.to_numeric(out.get("days_since_maintenance"), errors="coerce")
158         out["has_maintenance_history"] = since.notna().astype(float)
159         since = since.fillna(0.0).clip(lower=0.0)
160         out["fouling_0_30"] = since.clip(upper=30.0) / 30.0
161         out["fouling_30_90"] = (since - 30.0).clip(lower=0.0, upper=60.0) / 60.0
162         return out
163
164
165     def _parameter_names(assets: Sequence[str]) -> tuple[str, ...]:
166         return tuple(
167             [f"intercept:{asset}" for asset in assets]
168             + [f"degradation:{asset}" for asset in assets]
169             + [f"transient:{asset}" for asset in assets]
170             + list(MODEL_FEATURES)
171             + ["common_degradation"]
172         )
173
174
175     def _state_design(
176         frame: pd.DataFrame,
177         assets: Sequence[str],
178         reference_date: pd.Timestamp,
179         transient_time_years: float,
180         parameter_names: Sequence[str] | None = None,
181     ) -> np.ndarray:

```

```

182 work = _ensure_state_features(frame, reference_date)
183 asset_values = work["asset"].to_numpy(dtype=str)
184 time_years = work["time_years"].to_numpy(dtype=float)
185 transient = 1.0 - np.exp(-np.maximum(time_years, 0.0) / transient_time_years)
186 columns: dict[str, np.ndarray] = {}
187 for asset in assets:
188     indicator = (asset_values == asset).astype(float)
189     columns[f"intercept:{asset}"] = indicator
190     columns[f"degradation:{asset}"] = -time_years * indicator
191     columns[f"transient:{asset}"] = transient * indicator
192 for name in MODEL_FEATURES:
193     columns[name] = work[name].to_numpy(dtype=float)
194 columns["common_degradation"] = np.zeros(len(work), dtype=float)
195 names = tuple(parameter_names or _parameter_names(assets))
196 return np.column_stack([columns[name] for name in names])
197
198
199 def _independent_decay_prior(panel: pd.DataFrame) -> float:
200     """用训练期设备斜率的稳健中位数给长期退化率设置弱先验。"""
201
202     try:
203         model = fit_panel_models(panel)["full"]
204         declines = [
205             -float(value)
206             for name, value in model.params.items()
207             if name.endswith(":time_years") and float(value) < 0.0
208         ]
209     except (ValueError, KeyError, np.linalg.LinAlgError):
210         declines = []
211     prior = float(np.median(declines)) if declines else 12.0
212     return float(np.clip(prior, 3.0, 30.0))
213
214
215 def fit_hierarchical_state_model(
216     panel: pd.DataFrame,
217     *
218     shrinkage: float = STATE_SHRINKAGE,
219     transient_time_years: float = TRANSIENT_TIME_YEARS,
220     transient_penalty: float = TRANSIENT_PENALTY,
221     reference_date: pd.Timestamp | None = None,
222 ) -> HierarchicalStateModel:
223     """拟合可长期外推的分层退化状态模型。
224
225     设备长期退化率被限制为非负并向队列均值汇聚；观测期内的阶段性上升或
226     下降由饱和项  $u_i(1 - \exp(-t/\tau))$  吸收，避免把正斜率永久外推。
227
228     """
229     required = {"asset", "date", "performance_model"}
230     missing = required - set(panel.columns)
231     if missing:
232         raise ValueError(f"问题二面板缺少字段: {sorted(missing)}")
233     valid = panel.loc[panel["performance_model"].notna()].copy()
234     if len(valid) < 500 or valid["asset"].nunique() < 2:
235         raise ValueError("分层退化模型至少需要两个设备和 500 个有效设备日")
236     reference = pd.Timestamp(reference_date or pd.to_datetime(valid["date"]).min())
237     valid = _ensure_state_features(valid, reference)
238     assets = tuple(asset for asset in ASSETS if asset in set(valid["asset"]))
239     names = _parameter_names(assets)
240     design = _state_design(valid, assets, reference, transient_time_years, names)
241     response = valid["performance_model"].to_numpy(dtype=float)
242     weights = pd.to_numeric(valid.get("quality_weight", 1.0), errors="coerce")
243     if not isinstance(weights, pd.Series):
244         weights = pd.Series(np.ones(len(valid)), index=valid.index)
245     weights = weights.fillna(0.25).clip(lower=0.05, upper=1.0).to_numpy(dtype=float)
246     root_weight = np.sqrt(weights)
247     weighted_design = design * root_weight[:, np.newaxis]
248     weighted_response = response * root_weight
249
250     prior = _independent_decay_prior(valid)
251     penalty_rows: list[np.ndarray] = []
252     penalty_targets: list[float] = []
253     common_index = names.index("common_degradation")
254     for asset in assets:
255         row = np.zeros(len(names), dtype=float)
256         row[names.index(f"degradation:{asset}")] = np.sqrt(shrinkage)
257         row[common_index] = -np.sqrt(shrinkage)
258         penalty_rows.append(row)
259         penalty_targets.append(0.0)
260     row = np.zeros(len(names), dtype=float)
261     row[common_index] = np.sqrt(PRIOR_WEIGHT)
262     penalty_rows.append(row)
263     penalty_targets.append(np.sqrt(PRIOR_WEIGHT) * prior)
264     for asset in assets:
265         row = np.zeros(len(names), dtype=float)
266         row[names.index(f"transient:{asset}")] = np.sqrt(transient_penalty)
267         penalty_rows.append(row)
268         penalty_targets.append(0.0)
269
270     augmented_design = np.vstack([weighted_design, np.vstack(penalty_rows)])
271     augmented_response = np.concatenate([weighted_response, penalty_targets])
272     lower = np.full(len(names), -np.inf, dtype=float)
273     upper = np.full(len(names), np.inf, dtype=float)
274     for index, name in enumerate(names):
275         if name.startswith("degradation:") or name == "common_degradation":
276             lower[index] = 0.0

```

```

277
278 with warnings.catch_warnings():
279     warnings.simplefilter("ignore", RuntimeWarning)
280     fitted = lsq_linear(
281         augmented_design,
282         augmented_response,
283         bounds=(lower, upper),
284         lsq_solver="lsmr",
285         lsqr_tol="auto",
286         max_iter=500,
287     )
288 if not fitted.success:
289     raise RuntimeError(f"分层退化状态模型未收敛: {fitted.message}")
290
291 prediction = np.sum(design * fitted.x[np.newaxis, :], axis=1)
292 residual = response - prediction
293 weighted_sse = float(np.sum(weights * np.square(residual)))
294 weighted_rmse = float(np.sqrt(weighted_sse / np.sum(weights)))
295 centered = response - np.average(response, weights=weights)
296 weighted_sst = float(np.sum(weights * np.square(centered)))
297 r_squared = 1.0 - weighted_sse / weighted_sst
298
299 ar1_values = []
300 residual_frame = valid[["asset", "date"]].copy()
301 residual_frame["residual"] = residual
302 for _, block in residual_frame.groupby("asset"):
303     ordered = block.sort_values("date")["residual"]
304     correlation = ordered.corr(ordered.shift(1))
305     if pd.notna(correlation):
306         ar1_values.append(float(correlation))
307 residual_ar1 = float(np.clip(np.median(ar1_values), -0.8, 0.9)) if ar1_values else 0.0
308 inflation = float(np.clip((1.0 + residual_ar1) / (1.0 - residual_ar1), 1.0, 9.0))
309 degrees = max(int(np.sum(weights > 0.0)) - len(names), 1)
310 variance = weighted_sse / degrees
311 gram = np.einsum("ni,nj->ij", augmented_design, augmented_design)
312 # 部分 macOS Accelerate/BLAS 组合会在小型伪逆矩阵乘法时发出伪
313 # RuntimeWarning: 结果本身有限, 故在此局部屏蔽并随后显式校验。
314 with warnings.catch_warnings():
315     warnings.simplefilter("ignore", RuntimeWarning)
316     covariance = pinvh(gram + np.eye(len(names)) * 1e-9) * variance * inflation
317 if not np.isfinite(covariance).all():
318     raise RuntimeError("分层退化模型协方差矩阵包含非有限值")
319
320 return HierarchicalStateModel(
321     assets=assets,
322     reference_date=reference,
323     transient_time_years=float(transient_time_years),
324     shrinkage=float(shrinkage),
325     transient_penalty=float(transient_penalty),
326     parameter_names=names,
327     params=np.asarray(fitted.x, dtype=float),
328     covariance=np.asarray(covariance, dtype=float),
329     residual_std=float(np.sqrt(np.mean(np.square(residual)))),
330     residual_ar1=residual_ar1,
331     nobs=len(valid),
332     weighted_rmse=weighted_rmse,
333     r_squared=float(r_squared),
334     prior_degradation_per_year=prior,
335 )
336
337
338 def state_parameter_table(model: HierarchicalStateModel) -> pd.DataFrame:
339     standard_error = np.sqrt(np.maximum(np.diag(model.covariance), 0.0))
340     rows = []
341     for index, name in enumerate(model.parameter_names):
342         estimate = float(model.params[index])
343         low = estimate - 1.96 * standard_error[index]
344         if name.startswith("degradation:") or name == "common_degradation":
345             low = max(low, 0.0)
346         rows.append(
347             {
348                 "parameter": name,
349                 "estimate": estimate,
350                 "approx_std_error_ar1_inflated": float(standard_error[index]),
351                 "approx_ci95_low": float(low),
352                 "approx_ci95_high": float(estimate + 1.96 * standard_error[index]),
353             }
354         )
355     return pd.DataFrame(rows)
356
357
358 def sarimax_prediction(train: pd.DataFrame, test: pd.DataFrame) -> pd.DataFrame:
359     rows = []
360     exogenous = ("time_years",) + MODEL_FEATURES
361     for asset in ASSETS:
362         train_block = train.loc[_asset_strings(train["asset"]) ==
asset].sort_values("date")
363         test_block = test.loc[_asset_strings(test["asset"]) == asset].sort_values("date")
364         if len(train_block) < 300 or test_block.empty:
365             continue
366         x_train = np.column_stack(
367             [np.ones(len(train_block)), train_block.loc[:,
exogenous].to_numpy(dtype=float)]
368         )

```

```

369     x_test = np.column_stack(
370         [np.ones(len(test_block)), test_block.loc[:, exogenous].to_numpy(dtype=float)]
371     )
372     response = pd.Series(train_block["performance_model"].to_numpy(dtype=float))
373     with warnings.catch_warnings():
374         warnings.simplefilter("ignore", ConvergenceWarning)
375         warnings.simplefilter("ignore", ValueWarning)
376         warnings.simplefilter("ignore", FutureWarning)
377         warnings.simplefilter("ignore", RuntimeWarning)
378         result = SARIMAX(
379             response,
380             exog=x_test,
381             order=(1, 0, 0),
382             trend="n",
383             enforce_stationarity=False,
384             enforce_invertibility=False,
385         ).fit(disp=False, maxiter=120)
386         prediction = np.asarray(
387             result.get_forecast(steps=len(test_block), exog=x_test).predicted_mean,
388             dtype=float,
389         )
390     rows.append(
391         pd.DataFrame(
392             {
393                 "asset": asset,
394                 "date": test_block["date"].to_numpy(),
395                 "prediction": prediction,
396             }
397         )
398     )
399     return pd.concat(rows, ignore_index=True) if rows else pd.DataFrame()
400
401 def prediction_metrics(
402     full_panel: pd.DataFrame,
403     test: pd.DataFrame,
404     predictions: pd.DataFrame,
405     cutoff: pd.Timestamp,
406 ) -> dict[str, float | int]:
407     actual = test[["asset", "date", "performance_model", "maintenance_type"]].copy()
408     actual["asset"] = _asset_strings(actual["asset"])
409     merged = actual.merge(predictions, on=["asset", "date"], how="inner")
410     valid = merged.dropna(subset=["performance_model", "prediction"]).copy()
411     error = valid["performance_model"] - valid["prediction"]
412
413     valid["month"] = pd.to_datetime(valid["date"]).dt.to_period("M")
414     monthly = (
415         valid.groupby(["asset", "month"])["performance_model", "prediction"]
416         .mean()
417         .reset_index()
418     )
419     monthly_error = monthly["performance_model"] - monthly["prediction"]
420
421     rolling_errors: list[float] = []
422     gain7_errors: list[float] = []
423     gain30_errors: list[float] = []
424     prediction_lookup = predictions.set_index(["asset", "date"])["prediction"]
425     for asset in ASSETS:
426         block = full_panel.loc[_asset_strings(full_panel["asset"])] ==
427         asset].sort_values("date").copy()
428         block["asset"] = asset
429         stitched = block["performance_model"].copy()
430         test_mask = block["date"] > cutoff
431         keys = pd.MultiIndex.from_arrays(
432             [np.repeat(asset, int(test_mask.sum()))], block.loc[test_mask, "date"]
433         )
434         stitched.loc[test_mask] = prediction_lookup.reindex(keys).to_numpy()
435         actual_rolling = block["performance_model"].rolling(365, min_periods=330).mean()
436         predicted_rolling = stitched.rolling(365, min_periods=330).mean()
437         compare = test_mask & actual_rolling.notna() & predicted_rolling.notna()
438         rolling_errors.extend((actual_rolling.loc[compare] -
439         predicted_rolling.loc[compare]).tolist())
440
441         asset_predictions = predictions.loc[predictions["asset"] ==
442         asset].set_index("date")["prediction"]
443         for event in block.loc[
444             block["maintenance_type"].notna() & (block["date"] > cutoff +
445             pd.Timedelta(days=7))
446             ].itertuples(index=False):
447             delta = (block["date"] - event.date).dt.days
448             pre = block.loc[delta.between(-7, -1)]
449             post7 = block.loc[delta.between(1, 7)]
450             post30 = block.loc[delta.between(8, 30)]
451             if len(pre) >= 4 and len(post7) >= 4:
452                 dates = pd.concat([pre["date"], post7["date"]])
453                 predicted = asset_predictions.reindex(dates)
454                 if predicted.notna().all():
455                     predicted_pre = predicted.iloc[: len(pre)].mean()
456                     predicted_post = predicted.iloc[len(pre) :].mean()

```



```

456         actual_gain = post7["performance_model"].mean() -
pre["performance_model"].mean()
457     gain7_errors.append(float(actual_gain - (predicted_post - predicted_pre)))
458     if len(pre) >= 4 and len(post30) >= 10:
459         dates = pd.concat([pre["date"], post30["date"]])
460         predicted = asset_predictions.reindex(dates)
461         if predicted.notna().all():
462             predicted_pre = predicted.iloc[:len(pre)].mean()
463             predicted_post = predicted.iloc[len(pre):].mean()
464             actual_gain = post30["performance_model"].mean() -
pre["performance_model"].mean()
465         gain30_errors.append(float(actual_gain - (predicted_post - predicted_pre)))
466
467     return {
468         "n_test": len(valid),
469         "mae": float(np.mean(np.abs(error))),
470         "rmse": float(np.sqrt(np.mean(np.square(error)))),
471         "monthly_rmse": float(np.sqrt(np.mean(np.square(monthly_error)))),
472         "rolling_365_rmse": (
473             float(np.sqrt(np.mean(np.square(rolling_errors)))) if rolling_errors else np.nan
474         ),
475         "post_1_7_gain_mae": (
476             float(np.mean(np.abs(gain7_errors))) if gain7_errors else np.nan
477         ),
478         "post_8_30_gain_mae": (
479             float(np.mean(np.abs(gain30_errors))) if gain30_errors else np.nan
480         ),
481     }
482
483
484 def rolling_origin_backtest(panel: pd.DataFrame) -> pd.DataFrame:
485     """比较独立性、SARIMAX 与分层状态模型，不使用随机切分。"""
486
487     work = prepare_analysis_panel(panel)
488     last_date = pd.to_datetime(work["date"]).max()
489     rows = list(dict[str, Any]) = []
490     for horizon in BACKTEST_HORIZONS_DAYS:
491         cutoff = last_date - pd.Timedelta(days=horizon)
492         train = work.loc[work["date"] <= cutoff].copy()
493         test = work.loc[work["date"] > cutoff].copy()
494
495         independent = fit_panel_models(train)["full"]
496         valid_test = test.loc[test["performance_model"].notna()].copy()
497         independent_predictions = pd.DataFrame(
498             {
499                 "asset": asset_strings(valid_test["asset"]),
500                 "date": valid_test["date"].to_numpy(),
501                 "prediction": independent.predict(valid_test),
502             }
503         )
504
505         state_model = fit_hierarchical_state_model(train,
reference_date=work["date"].min())
506         state_predictions = pd.DataFrame(
507             {
508                 "asset": asset_strings(valid_test["asset"]),
509                 "date": valid_test["date"].to_numpy(),
510                 "prediction": state_model.predict(valid_test),
511             }
512         )
513         sarimax_predictions = _sarimax_prediction(train, test)
514
515         for model_name, predictions in (
516             ("independent_linear", independent_predictions),
517             ("sarimax_ar1", sarimax_predictions),
518             ("hierarchical_state", state_predictions),
519         ):
520             metrics = prediction_metrics(work, test, predictions, cutoff)
521             rows.append(
522                 {
523                     "split": f"holdout {horizon}d",
524                     "train_end": cutoff,
525                     "test_end": last_date,
526                     "horizon_days": horizon,
527                     "model": model_name,
528                     **metrics,
529                 }
530             )
531     return pd.DataFrame(rows)
532
533
534 def leave_one_asset_validation(panel: pd.DataFrame, calibration_days: int = 120) ->
pd.DataFrame:
535     """留一设备后，仅用该设备前 120 日校准截距和早期漂移。"""
536
537     work = prepare_analysis_panel(panel)
538     reference = pd.to_datetime(work["date"]).min()
539     rows = []
540     for held_out in ASSETS:
541         training = work.loc[_asset_strings(work["asset"]) != held_out].copy()
542         held = work.loc[_asset_strings(work["asset"]) ==
held_out].sort_values("date").copy()

```

```

543         if held.empty:
544             continue
545         model = fit_hierarchical_state_model(training, reference_date=reference)
546         cutoff = held["date"].min() + pd.Timedelta(days=calibration_days - 1)
547         calibration = held.loc[
548             (held["date"] <= cutoff) & held["performance_model"].notna()
549         ].copy()
550         test = held.loc[(held["date"] > cutoff) & held["performance_model"].notna()].copy()
551         shared_names = list(MODEL_FEATURES)
552         shared = np.column_stack(
553             [_ensure_state_features(calibration, reference)[name] for name in shared_names]
554         )
555         shared_beta = np.array([model.parameter(name) for name in shared_names])
556         common_decay = model.parameter("common_degradation")
557         time_cal = (
558             (calibration["date"] - reference).dt.days.to_numpy(dtype=float) /
ANNUAL_PERIOD_DAYS
559         )
560         transient_cal = 1.0 - np.exp(-np.maximum(time_cal, 0.0) /
model.transient_time_years)
561         target = (
562             calibration["performance_model"].to_numpy(dtype=float)
563             - np.sum(shared * shared_beta[np.newaxis, :], axis=1)
564             + common_decay * time_cal
565         )
566         calibration_design = np.column_stack([np.ones(len(calibration)), transient_cal])
567         intercept, transient = np.linalg.lstsq(calibration_design, target, rcond=None)[0]
568
569         test_features = _ensure_state_features(test, reference)
570         shared_test = test_features.loc[:, shared_names].to_numpy(dtype=float)
571         time_test = test_features["time_years"].to_numpy(dtype=float)
572         transient_test = 1.0 - np.exp(-np.maximum(time_test, 0.0) /
model.transient_time_years)
573         prediction = (
574             intercept
575             - common_decay * time_test
576             + transient * transient_test
577             + np.sum(shared_test * shared_beta[np.newaxis, :], axis=1)
578         )
579         error = test["performance_model"].to_numpy(dtype=float) - prediction
580         rows.append(
581             {
582                 "asset": held.out,
583                 "calibration_days": calibration_days,
584                 "n_test": len(test),
585                 "mae": float(np.mean(np.abs(error))),
586                 "rmse": float(np.sqrt(np.mean(np.square(error)))),
587                 "borrowed_common_degradation_per_year": common_decay,
588             }
589         )
590     return pd.DataFrame(rows)
591
592
593 def estimate_fixed_schedule(maintenance: pd.DataFrame) -> pd.DataFrame:
594     """把附件维护记录转为逐设备固定间隔和大维护频率。"""
595
596     ordered = maintenance.sort_values(["asset", "maintenance_date"]).copy()
597     global_major_events = int((ordered["maintenance_type"] == "major").sum())
598     global_major_every = max(int(round(len(ordered) / max(global_major_events, 1))), 2)
599     rows = []
600     for asset in ASSETS:
601         block = ordered.loc[ordered["asset"] == asset].reset_index(drop=True)
602         intervals = block["maintenance_date"].diff().dt.days.dropna()
603         interval_days = int(round(float(intervals.median()))) if len(intervals) else 57
604         major_positions = block.index[block["maintenance_type"] == "major"].tolist()
605         if major_positions:
606             major_every = max(int(round(len(block) / len(major_positions))), 2)
607             major_source = "asset_history"
608             events_since_major = len(block) - 1 - major_positions[-1]
609         else:
610             major_every = global_major_every
611             major_source = "cohort_imputed_no_recorded_major"
612             events_since_major = 0
613         rows.append(
614             {
615                 "asset": asset,
616                 "observed_events": len(block),
617                 "observed_major_events": len(major_positions),
618                 "interval_days": interval_days,
619                 "interval_q25_days": float(intervals.quantile(0.25)) if len(intervals)
620                 else np.nan,
621                 "interval_q75_days": float(intervals.quantile(0.75)) if len(intervals)
622                 else np.nan,
623                 "major_every_events": major_every,
624                 "major_frequency_source": major_source,
625                 "events_since_major_at_origin": events_since_major,
626                 "last_recorded_event_date": block["maintenance_date"].max(),
627                 "last_recorded_event_type": block.iloc[-1]["maintenance_type"],
628             }
629         )
630     return pd.DataFrame(rows)

```

```

630
631 def build_future_frame(
632     policy: pd.DataFrame,
633     start_date: pd.Timestamp,
634     end_date: pd.Timestamp,
635     reference_date: pd.Timestamp,
636 ) -> pd.DataFrame:
637     """按设备固定规律生成未来维护日历和状态特征。"""
638
639     parts = []
640     for row in policy.itertuples(index=False):
641         dates = pd.date_range(pd.Timestamp(start_date) + pd.Timedelta(days=1), end_date,
642                                freq="D")
643         events: dict[pd.Timestamp, str] = {}
644         event_date = pd.Timestamp(row.last_recorded_event_date)
645         count_since_major = int(row.events_since_major_at_origin)
646         while event_date <= end_date:
647             event_date += pd.Timedelta(days=int(row.interval_days))
648             if count_since_major + 1 >= int(row.major_every_events):
649                 event_type = "major"
650                 count_since_major = 0
651             else:
652                 event_type = "medium"
653                 count_since_major += 1
654             if event_date > start_date and event_date <= end_date:
655                 events[event_date] = event_type
656         block = pd.DataFrame({"asset": row.asset, "date": dates})
657         block["maintenance_type"] = block["date"].map(events)
658         block["last_maintenance_date"] = block["date"].where(
659             block["maintenance_type"].notna()
660         )
661         seed_last = pd.Timestamp(row.last_recorded_event_date)
662         if len(block):
663             block.loc[block.index[0], "last_maintenance_date"] = {
664                 block.loc[block.index[0], "last_maintenance_date"]
665                 if pd.notna(block.loc[block.index[0], "last_maintenance_date"])
666                 else seed_last
667             }
668         block["last_maintenance_date"] = block["last_maintenance_date"].ffill()
669         block["days_since_maintenance"] = {
670             block["date"] - block["last_maintenance_date"]
671         }.dt.days
672         parts.append(block)
673         future = pd.concat(parts, ignore_index=True)
674         return _ensure_state_features(future, reference_date)
675
676 def attach_historical_rolling(
677     history: pd.DataFrame,
678     future: pd.DataFrame,
679     value_col: str = "performance_forecast",
680 ) -> pd.DataFrame:
681     parts = []
682     for asset in ASSETS:
683         past = history.loc[_asset_strings(history["asset"])] ==
684         asset.sort_values("date").copy()
685         forecast = future.loc[_asset_strings(future["asset"])] ==
686         asset.sort_values("date").copy()
687         past_values = past["performance_model"].interpolate(limit_direction="both")
688         combined = pd.concat(
689             [
690                 pd.DataFrame({"date": past["date"], "value": past_values}),
691                 pd.DataFrame({"date": forecast["date"], "value": forecast[value_col]}),
692             ],
693             ignore_index=True,
694             rolling = combined["value"].rolling(ROLLING_YEAR_DAYS,
695             min_periods=ROLLING_YEAR_DAYS).mean()
696         )
697         forecast["rolling_annual_mean"] = rolling.iloc[-len(forecast) :].to_numpy()
698         parts.append(forecast)
699         return pd.concat(parts, ignore_index=True)
700
701 def joint_failure_dates(
702     forecast: pd.DataFrame,
703     recovery_level: float = LIFETIME_THRESHOLD,
704 ) -> pd.DataFrame:
705     """联合判据：年均低于 37，且随后大维护 8-30 日均值仍低于恢复线。"""
706
707     rows = []
708     for asset, block in forecast.groupby("asset"):
709         ordered = block.sort_values("date").reset_index(drop=True)
710         crossing = ordered.index[ordered["rolling_annual_mean"] < LIFETIME_THRESHOLD]
711         threshold_date = pd.NaT
712         diagnostic_major_date = pd.NaT
713         post_major_mean = np.nan
714         failure_date = pd.NaT
715         if len(crossing):
716             first_index = int(crossing[0])
717             threshold_date = ordered.loc[first_index, "date"]
718             major_indices = ordered.index[
719                 (ordered.index >= first_index) & (ordered["maintenance_type"] == "major")

```



```

719         for major_index in major_indices:
720             post = ordered.loc[
721                 (ordered.index >= major_index + 8) & (ordered.index <= major_index
+ 30),
722                 "performance_forecast",
723             ]
724             if len(post) >= 20 and float(post.mean()) < recovery_level:
725                 diagnostic_major_date = ordered.loc[major_index, "date"]
726                 post_major_mean = float(post.mean())
727                 failure_date = ordered.loc[major_index + 30, "date"]
728                 break
729             rows.append(
730                 {
731                     "asset": asset,
732                     "candidate_threshold_date": threshold_date,
733                     "diagnostic_major_date": diagnostic_major_date,
734                     "post_major_8_30_mean": post_major_mean,
735                     "joint_failure_date": failure_date,
736                     "recovery_level": recovery_level,
737                 }
738             )
739         return pd.DataFrame(rows)
740
741
742 def structural_sensitivity_models(panel: pd.DataFrame) ->
tuple[list[HierarchicalStateModel], pd.DataFrame]:
743     """用可解释的超参数组构造结构不确定性集合。"""
744
745     specifications = (
746         (400.0, 1.0, 5.0),
747         (400.0, 2.0, 5.0),
748         (400.0, 4.0, 5.0),
749         (1600.0, 1.0, 2.0),
750         (1600.0, 2.0, 5.0),
751         (1600.0, 4.0, 10.0),
752         (5000.0, 1.0, 5.0),
753         (5000.0, 2.0, 5.0),
754         (5000.0, 4.0, 5.0),
755     )
756     reference = pd.to_datetime(panel["date"]).min()
757     models = []
758     rows = []
759     for model_id, (shrinkage, transient_years, transient_penalty) in
enumerate(specifications):
760         model = fit_hierarchical_state_model(
761             panel,
762             shrinkage=shrinkage,
763             transient_time_years=transient_years,
764             transient_penalty=transient_penalty,
765             reference_date=reference,
766         )
767         models.append(model)
768         for asset in ASSETS:
769             rows.append(
770                 {
771                     "model_id": model_id,
772                     "asset": asset,
773                     "shrinkage": shrinkage,
774                     "transient_time_years": transient_years,
775                     "transient_penalty": transient_penalty,
776                     "degradation_per_year": model.parameter(f"degradation:{asset}"),
777                     "common_degradation_per_year": model.parameter("common_degradation"),
778                     "weighted_rmse": model.weighted_rmse,
779                     "r_squared": model.r_squared,
780                 }
781             )
782     return models, pd.DataFrame(rows)
783
784
785 def residual_outlier_sensitivity(
786     panel: pd.DataFrame,
787     main_model: HierarchicalStateModel,
788     residual_outliers: pd.DataFrame,
789 ) -> tuple[HierarchicalStateModel, pd.DataFrame]:
790     """比较保留与排除问题一残差异常日时的长期退化率。"""
791
792     flagged = residual_outliers.copy()
793     flagged["date"] = pd.to_datetime(flagged["date"], errors="coerce")
794     if "is_residual_outlier" in flagged:
795         flag = flagged["is_residual_outlier"]
796         if flag.dtype == object:
797             flag = flag.astype(str).str.lower().isin({"true", "1", "yes"})
798         flagged = flagged.loc[flag.astype(bool)]
799     keys = pd.MultiIndex.from_frame(flagged[["asset", "date"]].drop_duplicates())
800     work = panel.copy()
801     work["asset"] = asset_strings(work["asset"])
802     work["date"] = pd.to_datetime(work["date"])
803     panel_keys = pd.MultiIndex.from_frame(work[["asset", "date"]])
804     clean = work.loc[~panel_keys.isin(keys)].copy()
805     clean_model = fit_hierarchical_state_model(
806         clean,
807         reference_date=main_model.reference_date,
808         shrinkage=main_model.shrinkage,

```

```

809     transient_time_years=main_model.transient_time_years,
810     transient_penalty=main_model.transient_penalty,
811 )
812 rows = []
813 for asset in ASSETS:
814     main_rate = main_model.parameter(f"degradation:{asset}")
815     clean_rate = clean_model.parameter(f"degradation:{asset}")
816     rows.append(
817         {
818             "asset": asset,
819             "flagged_days_removed": int(
820                 ((flagged["asset"] == asset) & flagged["date"].notna()).sum()
821             ),
822             "degradation_with_outliers": main_rate,
823             "degradation_without_outliers": clean_rate,
824             "absolute_difference": clean_rate - main_rate,
825             "relative_difference": (
826                 (clean_rate - main_rate) / main_rate if main_rate > 0.0 else np.nan
827             ),
828         }
829     )
830 return clean_model, pd.DataFrame(rows)
831
832
833 def _censored_quantile_index(indices: np.ndarray, horizon: int, probability: float) ->
834 int | None:
835     filled = np.where(indices >= 0, indices, horizon)
836     result = int(np.quantile(filled, probability, method="higher"))
837     return None if result >= horizon else result
838
839 def simulate_lifetime_distribution(
840     history: pd.DataFrame,
841     future: pd.DataFrame,
842     scenario_models: Sequence[HierarchicalStateModel],
843     paths: int = SIMULATION_PATHS,
844     seed: int = SIMULATION_SEED,
845 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
846     """在结构模型集合上做蒙特卡洛外推，返回寿命、判据和预测带。"""
847
848     rng = np.random.default_rng(seed)
849     lifetime_rows = []
850     sensitivity_rows = []
851     band_rows = []
852     commission = pd.Timestamp(COMMISSION_DATE)
853     forecast_origin = pd.to_datetime(history["date"]).max()
854
855     for asset in ASSETS:
856         block = future.loc[_asset_strings(future["asset"]) ==
857 asset].sort_values("date").reset_index(drop=True)
858         dates = pd.DatetimeIndex(block["date"])
859         horizon = len(block)
860         elapsed_years = (dates - forecast_origin).days.to_numpy(dtype=float) /
861 ANNUAL_PERIOD_DAYS
862         scenario_paths = np.vstack([model.predict(block) for model in scenario_models])
863         scenario_degradation = np.array(
864             [model.parameter(f"degradation:{asset}") for model in scenario_models],
865             dtype=float
866         )
867         scenario_residual = np.array([model.residual_std for model in scenario_models],
868             dtype=float)
869         choices = rng.integers(0, len(scenario_models), size=paths)
870         degradation_multiplier = np.exp(rng.normal(-0.5 * 0.10**2, 0.10, size=paths))
871         chosen_degradation = scenario_degradation[choices]
872         degradation_draw = chosen_degradation * degradation_multiplier
873         level_shift = rng.normal(0.0, scenario_residual[choices] / np.sqrt(60.0),
874             size=paths)
875
876         past = history.loc[_asset_strings(history["asset"]) == asset].sort_values("date")
877         past_values = past.values
878         past["performance_model"].interpolate(limit_direction="both").to_numpy(dtype=float)
879         past_values = past_values[-(ROLLING_YEAR_DAYS - 1) :]
880         if len(past_values) < ROLLING_YEAR_DAYS - 1:
881             raise ValueError(f"设备 {asset} 缺少滚动年均所需历史窗口")
882
883         failure_by_level = {
884             level: np.full(paths, -1, dtype=int) for level in RECOVERY_LEVELS
885         }
886         crossing_indices = np.full(paths, -1, dtype=int)
887         month_indices = np.flatnonzero(dates.is_month_end)
888         monthly_samples = np.empty((paths, len(month_indices)), dtype=float)
889         major_indices = np.flatnonzero(block["maintenance_type"].eq("major").to_numpy())
890
891         chunk_size = 100
892         for start in range(0, paths, chunk_size):
893             stop = min(start + chunk_size, paths)
894             selected = choices[start:stop]
895             prediction = scenario_paths[selected].copy()
896             delta = degradation_draw[start:stop] - chosen_degradation[start:stop]
897             prediction -= delta[:, np.newaxis] * elapsed_years[np.newaxis, :]
898             prediction += level_shift[start:stop, np.newaxis]

```

```

893 historical = np.broadcast_to(past_values, (stop - start, len(past_values)))
894 combined = np.concatenate([historical, prediction], axis=1)
895 cumulative = np.cumsum(
896     np.concatenate([np.zeros((stop - start, 1)), combined], axis=1),
897     axis=1,
898 )
899 rolling = (
900     cumulative[:, ROLLING_YEAR_DAYS:]
901     - cumulative[:, :-ROLLING_YEAR_DAYS]
902 ) / ROLLING_YEAR_DAYS
903 monthly_samples[start:stop] = rolling[:, month_indices]
904 below = rolling < LIFETIME_THRESHOLD
905 has_crossing = below.any(axis=1)
906 local_crossing = np.where(has_crossing, below.argmax(axis=1), -1)
907 crossing_indices[start:stop] = local_crossing
908
909 for local_index, crossing in enumerate(local_crossing):
910     if crossing < 0:
911         continue
912     candidate_majors = major_indices[major_indices >= crossing]
913     for level in RECOVERY_LEVELS:
914         for major_index in candidate_majors:
915             if major_index + 30 >= horizon:
916                 break
917             post_mean = float(prediction[local_index, major_index + 8 :
918 major_index + 31].mean())
919             if post_mean < level:
920                 failure_by_level[level][start + local_index] = major_index
921 + 30
922                 break
923
924 for month_column, date_index in enumerate(month_indices):
925     values = monthly_samples[:, month_column]
926     band_rows.append(
927         {
928             "asset": asset,
929             "date": dates[date_index],
930             "rolling_p10": float(np.quantile(values, 0.10)),
931             "rolling_p50": float(np.quantile(values, 0.50)),
932             "rolling_p90": float(np.quantile(values, 0.90)),
933         }
934     )
935
936 nominal = failure_by_level[LIFETIME_THRESHOLD]
937 quantile_indices = {
938     "p025": _censored_quantile_index(nominal, horizon, 0.025),
939     "p10": _censored_quantile_index(nominal, horizon, 0.10),
940     "p50": _censored_quantile_index(nominal, horizon, 0.50),
941     "p90": _censored_quantile_index(nominal, horizon, 0.90),
942     "p975": _censored_quantile_index(nominal, horizon, 0.975),
943 }
944
945 def date_at(index: int | None) -> pd.Timestamp:
946     return pd.NaT if index is None else pd.Timestamp(dates[index])
947
948 median_date = date_at(quantile_indices["p50"])
949 lifetime_rows.append(
950     {
951         "asset": asset,
952         "forecast_origin": forecast_origin,
953         "simulation_paths": paths,
954         "horizon_end": dates[-1],
955         "median_failure_date": median_date,
956         "p10_failure_date": date_at(quantile_indices["p10"]),
957         "p90_failure_date": date_at(quantile_indices["p90"]),
958         "p025_failure_date": date_at(quantile_indices["p025"]),
959         "p975_failure_date": date_at(quantile_indices["p975"]),
960         "median_total_lifetime_years": (
961             float((median_date - commission).days / ANNUAL_PERIOD_DAYS)
962             if pd.notna(median_date)
963             else np.nan
964         ),
965         "median_remaining_years": (
966             float((median_date - forecast_origin).days / ANNUAL_PERIOD_DAYS)
967             if pd.notna(median_date)
968             else np.nan
969         ),
970         "censored_rate_at_horizon": float(np.mean(nominal < 0)),
971         "median_threshold_crossing_date": date_at(
972             _censored_quantile_index(crossing_indices, horizon, 0.50)
973         ),
974     }
975 )
976 for level, failure_indices in failure_by_level.items():
977     median_index = _censored_quantile_index(failure_indices, horizon, 0.50)
978     median_failure = date_at(median_index)
979     sensitivity_rows.append(
980         {
981             "asset": asset,
982             "post_major_recovery_level": level,
983             "median_failure_date": median_failure,
984             "median_total_lifetime_years": (
985                 float((median_failure - commission).days / ANNUAL_PERIOD_DAYS)

```

```

985         if pd.notna(median_failure)
986         else np.nan
987     },
988     "censored_rate_at_horizon": float(np.mean(failure_indices < 0)),
989 }
990 )
991
992 return (
993     pd.DataFrame(lifetime_rows),
994     pd.DataFrame(sensitivity_rows),
995     pd.DataFrame(band_rows),
996 )
997
998
999 def plot_backtest(backtest: pd.DataFrame) -> Path:
1000     summary = backtest.groupby("model", as_index=False)["rmse"].mean()
1001     labels = {
1002         "independent linear": "设备独立线性",
1003         "sarimax ar1": "SARIMAX-AR(1)",
1004         "hierarchical_state": "分层退化状态",
1005     }
1006     summary["模型"] = summary["model"].map(labels)
1007     fig, ax = plt.subplots(figsize=(8.6, 5.0))
1008     sns.barplot(data=summary, x="模型", y="rmse", color="#3f77b5", ax=ax)
1009     ax.set_ylabel("滚动留出日预测 RMSE")
1010     ax.set_xlabel("")
1011     ax.set_title("问题二候选模型时间留出比较")
1012     for patch, value in zip(ax.patches, summary["rmse"]):
1013         ax.text(patch.get_x() + patch.get_width() / 2, value + 0.2, f"{value:.2f}",
1014             ha="center")
1015     return _save_figure(fig, "q2_01_backtest.png")
1016
1017 def plot_degradation_sensitivity(
1018     parameters: pd.DataFrame,
1019     main_model: HierarchicalStateModel,
1020 ) -> Path:
1021     grouped = parameters.groupby("asset")["degradation_per_year"]
1022     summary = grouped.quantile([0.1, 0.9]).unstack()
1023     summary.columns = ["low", "high"]
1024     summary["main"] = [main_model.parameter(f"degradation:{asset}") for asset in
1025         summary.index]
1026     summary = summary.reindex(ASSETS)
1027     x = np.arange(len(summary))
1028     fig, ax = plt.subplots(figsize=(10.5, 5.2))
1029     ax.errorbar(
1030         x,
1031         summary["main"],
1032         yerr=np.vstack([summary["main"] - summary["low"], summary["high"] -
1033             summary["main"]]),
1034         fmt="o",
1035         color="#2457a7",
1036         ecolor="#7ba3d1",
1037         capsize=4,
1038     )
1039     ax.set_xticks(x, summary.index)
1040     ax.set_ylabel("长期不可逆退化率 (透速率单位/年)")
1041     ax.set_title("分层状态模型的设备退化率与结构敏感区间")
1042     return _save_figure(fig, "q2_02_degradation_rates.png")
1043
1044 def plot_forecast_bands(bands: pd.DataFrame, lifetimes: pd.DataFrame) -> Path:
1045     fig, axes = plt.subplots(5, 2, figsize=(14, 18), sharex=True, sharey=True)
1046     lifetime_lookup = lifetimes.set_index("asset")
1047     display_end = pd.to_datetime(lifetimes["p90_failure_date"]).max() +
1048         pd.DateOffset(years=2)
1049     for ax, asset in zip(axes.ravel(), ASSETS):
1050         block = bands.loc[
1051             (bands["asset"] == asset) & (pd.to_datetime(bands["date"]) <= display_end)
1052             ].sort_values("date")
1053         x = pd.to_datetime(block["date"])
1054         ax.fill_between(
1055             mdates.date2num(x),
1056             block["rolling_p10"].clip(lower=0.0).to_numpy(),
1057             block["rolling_p90"].clip(lower=0.0).to_numpy(),
1058             color="#8db8df",
1059             alpha=0.35,
1060         )
1061         ax.plot(x, block["rolling_p50"].clip(lower=0.0), color="#2457a7", lw=1.4)
1062         ax.axhline(LIFETIME_THRESHOLD, color="#c43c39", ls="--", lw=1.0)
1063         failure = lifetime_lookup.loc[asset, "median_failure_date"]
1064         if pd.notna(failure):
1065             ax.axvline(pd.Timestamp(failure), color="#d95f02", ls=":", lw=1.2)
1066         ax.set_title(asset)
1067         ax.set_ylim(0.0, 125.0)
1068         ax.xaxis.set_major_locator(mdates.YearLocator(3))
1069         ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
1070     for ax in axes[-1]:
1071         ax.tick_params(axis="x", rotation=30)
1072     fig.suptitle("现行维护规律下滚动年均透速率预测 (失效判断附近)", fontsize=18, y=0.995)
1073     from matplotlib.lines import Line2D
1074     from matplotlib.patches import Patch

```

```

1073 handles = [
1074     Patch(facecolor="#8db8df", alpha=0.35, label="80%区间"),
1075     Line2D([0], [0], color="#2457a7", lw=1.5, label="中位路径"),
1076     Line2D([0], [0], color="#c43c39", ls="--", label="阈值37"),
1077     Line2D([0], [0], color="#d95f02", ls=":", label="寿命中位数"),
1078 ]
1079 fig.legend(
1080     handles=handles,
1081     loc="upper center",
1082     ncol=4,
1083     bbox_to_anchor=(0.5, 0.978),
1084     frameon=False,
1085 )
1086 fig.tight_layout(rect=(0, 0, 1, 0.95))
1087 return _save_figure(fig, "q2_03_lifetime_forecast.png")
1088
1089
1090
1091 def plot_lifetime_summary(lifetimes: pd.DataFrame) -> Path:
1092     commission = pd.Timestamp(COMMISSION_DATE)
1093     work = lifetimes.copy().set_index("asset").reindex(ASSETS)
1094     for name in ("p10", "p50", "p90"):
1095         column = "median_failure_date" if name == "p50" else f"{name}_failure_date"
1096         work[name] = (
1097             pd.to_datetime(work[column]) - commission
1098             ).dt.days / ANNUAL_PERIOD_DAYS
1099     y = np.arange(len(work))
1100     fig, ax = plt.subplots(figsize=(10.0, 6.0))
1101     ax.hlines(y, work["p10"], work["p90"], color="#7ba3d1", lw=5, alpha=0.7)
1102     ax.scatter(work["p50"], y, color="#2457a7", s=50, zorder=3)
1103     ax.set_yticks(y, work.index)
1104     ax.invert_yaxis()
1105     ax.set_xlabel("自 2022-04-01 投用起的总寿命 (年)")
1106     ax.set_title("十台设备寿命中位数与80%预测区间")
1107     return _save_figure(fig, "q2_04_lifetime_summary.png")
1108
1109
1110 def run_q2_formal(
1111     *,
1112     simulation_paths: int = SIMULATION_PATHS,
1113     forecast_horizon_years: int = FORECAST_HORIZON_YEARS,
1114 ) -> Dict[str, Any]:
1115     """运行问题二完整流程并保存可追溯结果、图形和关键结论。"""
1116
1117     ensure_output_dirs()
1118     panel_path = PROCESSED_DIR / "daily_panel.csv"
1119     if not panel_path.is_file():
1120         run_q1_scaffold()
1121     raw_panel = pd.read_csv(panel_path, parse_dates=["date", "last_maintenance_date"])
1122     panel = prepare_analysis_panel(raw_panel)
1123     maintenance = load_maintenance()
1124
1125     backtest = rolling_origin_backtest(raw_panel)
1126     leave_one = leave_one_asset_validation(raw_panel)
1127     main_model = fit_hierarchical_state_model(panel, reference_date=panel["date"].min())
1128     parameters = state_parameter_table(main_model)
1129     outlier_path = RESULTS_DIR / "q1_residual_outliers.csv"
1130     residual_outliers = (
1131         pd.read_csv(outlier_path, parse_dates=["date"])
1132         if outlier_path.is_file()
1133         else pd.DataFrame(columns=["asset", "date", "is_residual_outlier"])
1134     )
1135     clean_model, outlier_sensitivity = residual_outlier_sensitivity(
1136         panel,
1137         main_model,
1138         residual_outliers,
1139     )
1140     policy = estimate_fixed_schedule(maintenance)
1141     forecast_origin = pd.to_datetime(panel["date"].max())
1142     forecast_end = forecast_origin + pd.DateOffset(years=forecast_horizon_years)
1143     future = build_future_frame(policy, forecast_origin, forecast_end,
1144     main_model.reference_date)
1145     future["performance_forecast"] = main_model.predict(future)
1146     point_forecast = attach_historical_rolling(panel, future)
1147     point_failures = joint_failure_dates(point_forecast)
1148     scenario_models, structural = structural_sensitivity_models(panel)
1149     scenario_models.append(clean_model)
1150     lifetimes, lifetime_sensitivity, forecast_bands = simulate_lifetime_distribution(
1151         panel,
1152         future,
1153         scenario_models,
1154         paths=simulation_paths,
1155     )
1156     lifetimes = lifetimes.merge(
1157         point_failures[
1158             ["asset", "candidate_threshold_date", "joint_failure_date",
1159             "post_major_8_30_mean"]
1160         ],
1161         on="asset",
1162         how="left",
1163     ).merge(
1164         policy[

```



```

1164         [
1165             "asset",
1166             "interval_days",
1167             "major_every_events",
1168             "major_frequency_source",
1169         ],
1170     ],
1171     on="asset",
1172     how="left",
1173 )
1174
1175 output_tables = {
1176     "q2_model_backtest.csv": backtest,
1177     "q2_leave_one_asset_validation.csv": leave_one,
1178     "q2_state_parameters.csv": parameters,
1179     "q2_structural_sensitivity.csv": structural,
1180     "q2_outlier_sensitivity.csv": outlier_sensitivity,
1181     "q2_fixed_schedule.csv": policy,
1182     "q2_forecast_paths.csv": point_forecast,
1183     "q2_forecast_bands.csv": forecast_bands,
1184     "q2_lifetime_summary.csv": lifetimes,
1185     "q2_lifetime_sensitivity.csv": lifetime_sensitivity,
1186 }
1187 for name, frame in output_tables.items():
1188     frame.to_csv(RESULTS_DIR / name, index=False, date_format="%Y-%m-%d")
1189
1190 configure_plot_style()
1191 figures = [
1192     plot_backtest(backtest),
1193     plot_degradation_sensitivity(structural, main_model),
1194     plot_forecast_bands(forecast_bands, lifetimes),
1195     plot_lifetime_summary(lifetimes),
1196 ]
1197
1198 backtest_summary = backtest.groupby("model")["rmse"].mean().to_dict()
1199 finite_lifetimes = lifetimes.dropna(subset=["median_total_lifetime_years"])
1200 key_findings = {
1201     "forecast_origin": str(forecast_origin.date()),
1202     "observation_start": str(pd.to_datetime(panel["date"]).min().date()),
1203     "observation_end": str(forecast_origin.date()),
1204     "lifetime_definition": (
1205         "rolling 365-day mean below 37 and the following major maintenance "
1206         "has post-day-8-to-30 mean below 37"
1207     ),
1208     "main_model": "nonnegative hierarchical degradation with saturating transient
1209 drift",
1210     "state_model_r_squared": main_model.r_squared,
1211     "state_model_weighted_rmse": main_model.weighted_rmse,
1212     "common_degradation_per_year": main_model.parameter("common_degradation"),
1213     "mean_backtest_rmse_by_model": backtest_summary,
1214     "leave_one_asset_mean_rmse": float(leave_one["rmse"].mean()),
1215     "max_outlier_exclusion_rate_change": float(
1216         outlier_sensitivity["relative_difference"].abs().max()
1217     ),
1218     "simulation_paths": simulation_paths,
1219     "forecast_horizon_years": forecast_horizon_years,
1220     "earliest_median_asset": (
1221         finite_lifetimes.sort_values("median_total_lifetime_years").iloc[0]["asset"]
1222         if len(finite_lifetimes)
1223         else None
1224     ),
1225     "earliest_median_total_lifetime_years": (
1226         float(finite_lifetimes["median_total_lifetime_years"].min())
1227         if len(finite_lifetimes)
1228         else None
1229     ),
1230     "latest_median_asset": (
1231         finite_lifetimes.sort_values("median_total_lifetime_years").iloc[-1]["asset"]
1232         if len(finite_lifetimes)
1233         else None
1234     ),
1235     "latest_median_total_lifetime_years": (
1236         float(finite_lifetimes["median_total_lifetime_years"].max())
1237         if len(finite_lifetimes)
1238         else None
1239     ),
1240     "limitations": [
1241         "no observed lifetime termination; results are degradation-threshold
1242         extrapolations",
1243         "A4 and A8 have no recorded major maintenance; cohort major frequency
1244         is imputed",
1245         "small maintenance is unrecorded and absorbed into the background state",
1246         "prediction intervals include structural model choices but cannot cover
1247         unknown regime changes",
1248     ],
1249 }
1250 with (RESULTS_DIR / "q2_key_findings.json").open("w", encoding="utf-8") as file:
1251     json.dump(key_findings, file, ensure_ascii=False, indent=2)
1252
1253 return {
1254     "backtest": backtest,
1255     "leave_one_asset": leave_one,
1256     "main_model": main_model,

```

```

1253         "parameters": parameters,
1254         "policy": policy,
1255         "point_forecast": point_forecast,
1256         "lifetimes": lifetimes,
1257         "lifetime_sensitivity": lifetime_sensitivity,
1258         "forecast_bands": forecast_bands,
1259         "figures": figures,
1260         "key_findings": key_findings,
1261     }
1262
1263
1264 def fit_and_forecast_fixed_schedule(*args: Any, **kwargs: Any) -> pd.DataFrame:
1265     """兼容早期接口：返回现行维护规律下的日级点预测。"""
1266
1267     outputs = run_g2_formal(*args, **kwargs)
1268     return outputs["point_forecast"]
1269

```

附录 6：问题三完整源程序

src/filter_monitoring/q3_optimization.py

```
1 """问题三：分类维护响应、可解释策略搜索与全生命周期年均成本。
2
3 问题二的状态模型提供长期退化、季节和“任意一次有记录维护”后的
4 通用污染重置。本模块在其上增加中/大维护的类型修正、维护损伤情景和
5 任意维护日历，先用点估计搜索策略，再对前沿方案做蒙特卡复验。
6 """
7
8 from __future__ import annotations
9
10 import json
11 import os
12 import tempfile
13 from dataclasses import asdict, dataclass, replace
14 from pathlib import Path
15 from typing import Any, Dict, Iterable, Mapping, Sequence
16
17 # Matplotlib 必须在导入 pyplot 前指定可写缓存目录。
18 CACHE_ROOT = Path(tempfile.gettempdir()) / "math2026-filter-monitoring"
19 os.environ.setdefault("MPLCONFIGDIR", str(CACHE_ROOT / "matplotlib"))
20 os.environ.setdefault("XDG_CACHE_HOME", str(CACHE_ROOT / "xdg"))
21
22 import matplotlib.dates as mdates
23 import matplotlib.pyplot as plt
24 import numpy as np
25 import pandas as pd
26
27 from config import (
28     ASSETS,
29     BASE_COSTS,
30     COMMISSION_DATE,
31     LIFETIME_THRESHOLD,
32     PROCESSED_DIR,
33     RESULTS_DIR,
34     ROLLING_YEAR_DAYS,
35     CostConfig,
36     ensure_output_dirs,
37 )
38 from .io import load_maintenance
39 from .q1_analysis import (
40     ANNUAL_PERIOD_DAYS,
41     _configure_plot_style,
42     _save_figure,
43     prepare_analysis_panel,
44     run_q1_formal,
45     run_q1_scaffold,
46 )
47 from .q2_lifetime import (
48     FORECAST_HORIZON_YEARS,
49     HierarchicalStateModel,
50     ensure_state_features,
51     build_future_frame,
52     estimate_fixed_schedule,
53     fit_hierarchical_state_model,
54     residual_outlier_sensitivity,
55     structural_sensitivity_models,
56 )
57
58 SEARCH_SEED = 20260725
59 VALIDATION_SEED = 20260726
60 SCREENING_PATHS = 200
61 FINAL_PATHS = 2000
62 VALIDATION_PATHS = 1000
63 RESPONSE_SHRINKAGE_EVENTS = 20.0
64 DAMAGE_SHARES = (0.0, 0.15, 0.30)
65 MAIN_DAMAGE_SHARE = 0.15
66 MAJOR_DAMAGE_MULTIPLIER = 4.0
67 RESPONSE_SCENARIOS = ("shrunk", "empirical", "major_equal")
68
69
70 @dataclass(frozen=True)
71 class PolicyOutcome:
72     """一条更新周期内的寿命和维护次数。"""
73
74     lifetime_days: float
75     medium_events: float
76     major_events: float
77     minor_events: float = 0.0
78
79
80 @dataclass(frozen=True)
81 class PolicySpec:
82     """可解释维护策略。"""
83
84     policy_id: str
```



```

86 policy_kind: str
87 medium_interval_days: int | None = None
88 major_interval_days: int | None = None
89 trigger_level: float | None = None
90 cooldown_days: int | None = None
91 description: str = ""
92
93
94 def annualized_lifecycle_cost(
95     outcome: PolicyOutcome,
96     costs: CostConfig = BASE_COSTS,
97 ) -> float:
98     """计算全生命周期年均成本，单位为万元/年。"""
99
100     if outcome.lifetime_days <= 0:
101         raise ValueError("设备寿命必须大于 0 天")
102     lifecycle_cost = (
103         costs.purchase
104         + costs.medium * outcome.medium_events
105         + costs.major * outcome.major_events
106         + costs.minor * outcome.minor_events
107     )
108     return lifecycle_cost / (outcome.lifetime_days / 365.0)
109
110
111 def build_policy_space() -> tuple[PolicySpec, ...]:
112     """构造周期规则和状态触发规则的搜索空间。"""
113
114     specs: list[PolicySpec] = [
115         PolicySpec(
116             policy_id="current",
117             policy_kind="current",
118             description="附件维护间隔中位数与历史大维护比例",
119         )
120     ]
121     # 搜索必须跨过现行 54-60 日区间且不在 120 天人为截断；
122     # 365 天与每年一次大维护组合时等价于不做额外中维护。
123     medium_intervals = (35, 45, 55, 65, 75, 90, 120, 150, 180, 240, 365)
124     major_intervals = (91, 122, 183, 365)
125     for medium in medium_intervals:
126         for major in major_intervals:
127             description = (
128                 "仅每年一次大维护，不安排额外中维护"
129                 if medium >= major == 365
130                 else f"每 {medium} 天检修，大维护最大间隔 {major} 天"
131             )
132             specs.append(
133                 PolicySpec(
134                     policy_id=f"periodic_m{medium}_b{major}",
135                     policy_kind="periodic",
136                     medium_interval_days=medium,
137                     major_interval_days=major,
138                     cooldown_days=min(medium, 30),
139                     description=description,
140                 )
141             )
142     for trigger in (40.0, 45.0, 50.0, 55.0, 60.0, 65.0, 70.0):
143         for cooldown in (30, 45, 60):
144             for major in major_intervals:
145                 specs.append(
146                     PolicySpec(
147                         policy_id=f"trigger_t{int(trigger)}_c{cooldown}_b{major}",
148                         policy_kind="trigger",
149                         major_interval_days=major,
150                         trigger_level=trigger,
151                         cooldown_days=cooldown,
152                         description=(
153                             f"未来30日预测低于 {trigger:.0f} 时中维护，"
154                             f"冷却 {cooldown} 天，大维护最大间隔 {major} 天"
155                         ),
156                     )
157                 )
158     return tuple(specs)
159
160
161 def policy_space_table(specs: Iterable[PolicySpec] | None = None) -> pd.DataFrame:
162     rows = [asdict(spec) for spec in (specs or build_policy_space())]
163     frame = pd.DataFrame(rows)
164     frame["feasible_main"] = True
165     frame["major frequency per year"] = np.where(
166         frame["major_interval_days"].notna(),
167         ANNUAL_PERIOD_DAYS / frame["major_interval_days"],
168         np.nan,
169     )
170     return frame
171
172
173 def estimate_maintenance_response(
174     effects: pd.DataFrame,
175     prior_events: float = RESPONSE_SHRINKAGE_EVENTS,
176 ) -> pd.DataFrame:
177     """将中/大维护事件效果向总体部分汇聚。"""
178
179     required = {

```

```

180         "maintenance_type",
181         "recorded_events",
182         "instant_mean",
183         "instant_ci95_low",
184         "instant_ci95_high",
185         "retained_mean",
186         "retained_ci95_low",
187         "retained_ci95_high",
188     }
189     missing = required - set(effects.columns)
190     if missing:
191         raise ValueError(f"维护效果表缺少字段: {sorted(missing)}")
192     work = effects.loc[effects["maintenance_type"].isin(["medium", "major"])]
193     if set(work["maintenance_type"]) != {"medium", "major"}:
194         raise ValueError("维护效果表必须同时包含中维护和大维护")
195     weights = work["recorded_events"].to_numpy(dtype=float)
196     pooled_instant = float(np.average(work["instant_mean"], weights=weights))
197     pooled_retained = float(np.average(work["retained_mean"], weights=weights))
198     rows = []
199     for row in work.itertuples(index=False):
200         n = float(row.recorded_events)
201         instant = float(row.instant_mean)
202         retained = float(row.retained_mean)
203         instant_shrunk = (n * instant + prior_events * pooled_instant) / (n + prior_events)
204         retained_shrunk = (n * retained + prior_events * pooled_retained) / (n
+ prior_events)
205     rows.append(
206         {
207             "maintenance_type": row.maintenance_type,
208             "recorded_events": int(row.recorded_events),
209             "instant_empirical": instant,
210             "retained_empirical": retained,
211             "instant_shrunk": instant_shrunk,
212             "retained_shrunk": retained_shrunk,
213             "instant_standard_error": (
214                 float(row.instant_ci95_high) - float(row.instant_ci95_low)
215             )
216             / (2.0 * 1.96),
217             "retained_standard_error": (
218                 float(row.retained_ci95_high) - float(row.retained_ci95_low)
219             )
220             / (2.0 * 1.96),
221             "pooled_instant": pooled_instant,
222             "pooled_retained": pooled_retained,
223             "prior_events": prior_events,
224         }
225     )
226     return pd.DataFrame(rows).sort_values("maintenance_type").reset_index(drop=True)
227
228 def validate_maintenance_response(
229     event_detail: pd.DataFrame,
230     prior_events: float = RESPONSE_SHRINKAGE_EVENTS,
231 ) -> pd.DataFrame:
232     """留一设备检查类型效果向新设备的迁移误差。"""
233
234     rows: list[dict[str, Any]] = []
235     data = event_detail.copy()
236     for held_asset in ASSETS:
237         train = data.loc[data["asset"].astype(str) != held_asset]
238         test = data.loc[data["asset"].astype(str) == held_asset]
239         for maintenance_type in ("medium", "major"):
240             for metric in ("instant_gain_adjusted", "retained_gain_adjusted"):
241                 train_metric = train.loc[
242                     train["maintenance_type"].eq(maintenance_type), metric
243                 ].dropna()
244                 pooled = train_metric.dropna()
245                 actual = test.loc[
246                     test["maintenance_type"].eq(maintenance_type), metric
247                 ].dropna()
248                 if len(actual) == 0 or len(train_metric) == 0 or len(pooled) == 0:
249                     continue
250                 prediction = (
251                     len(train_metric) * float(train_metric.mean())
252                     + prior_events * float(pooled.mean())
253                 ) / (len(train_metric) + prior_events)
254                 for value in actual:
255                     rows.append(
256                         {
257                             "held_asset": held_asset,
258                             "maintenance_type": maintenance_type,
259                             "metric": metric,
260                             "actual": float(value),
261                             "prediction": prediction,
262                             "error": float(value) - prediction,
263                         }
264                     )
265     detail = pd.DataFrame(rows)
266     if detail.empty:
267         return detail
268     summary = (
269         detail.groupby(["maintenance_type", "metric"], as_index=False)
270         .agg(
271             events=("error", "size"),
272

```

```

273         mae=("error", lambda x: float(np.mean(np.abs(x)))),
274         rmse=("error", lambda x: float(np.sqrt(np.mean(np.square(x))))),
275         mean_error=("error", "mean"),
276     )
277 )
278 return summary
279
280
281 def historical_maintenance_rates(
282     maintenance: pd.DataFrame,
283     forecast_origin: pd.Timestamp,
284 ) -> pd.DataFrame:
285     """计算观测期年频率和投用至预测原点的等价历史次数。"""
286
287     observation_start = pd.to_datetime(maintenance["maintenance_date"]).min()
288     observed_years = max(
289         float((pd.Timestamp(forecast_origin) - observation_start).days /
290             ANNUAL_PERIOD_DAYS),
291         1.0,
292     )
293     elapsed_years = float(
294         (pd.Timestamp(forecast_origin) - pd.Timestamp(COMMISSION_DATE)).days
295         / ANNUAL_PERIOD_DAYS
296     )
297     rows = []
298     global_major_rate = float(
299         maintenance["maintenance_type"].eq("major").sum()
300         / (len(ASSETS) * observed_years)
301     )
302     for asset in ASSETS:
303         block = maintenance.loc[maintenance["asset"].astype(str) == asset]
304         medium_rate = float(block["maintenance_type"].eq("medium").sum() / observed_years)
305         major_count = int(block["maintenance_type"].eq("major").sum())
306         # A4/A8 无大维护记录，成本与损伤基准频率与问题二一样用队列补充。
307         major_rate = float(major_count / observed_years) if major_count else
308         global_major_rate
309     rows.append(
310         {
311             "asset": asset,
312             "observation_years": observed_years,
313             "elapsed_years_at_origin": elapsed_years,
314             "medium_rate_per_year": medium_rate,
315             "major_rate_per_year": major_rate,
316             "historical_medium_events_equiv": medium_rate * elapsed_years,
317             "historical_major_events_equiv": major_rate * elapsed_years,
318             "major_rate_source": "asset_history" if major_count else "cohort_imputed",
319         }
320     )
321     return pd.DataFrame(rows)
322
323 def _event_type_series(
324     maintenance_type: pd.Series,
325     seed_type: str,
326 ) -> pd.Series:
327     out = maintenance_type.copy().astype("object")
328     if len(out):
329         out.iloc[0] = out.iloc[0] if pd.notna(out.iloc[0]) else seed_type
330     return out.ffill().fillna(seed_type)
331
332 def _finalize_policy_frame(
333     frame: pd.DataFrame,
334     seed_last_date: pd.Timestamp,
335     seed_last_type: str,
336     reference_date: pd.Timestamp,
337 ) -> pd.DataFrame:
338     out = frame.sort_values("date").reset_index(drop=True).copy()
339     out["last_maintenance_date"] = out["date"].where(out["maintenance_type"].notna())
340     if len(out) and pd.isna(out.loc[0, "last_maintenance_date"]):
341         out.loc[0, "last_maintenance_date"] = pd.Timestamp(seed_last_date)
342     out["last_maintenance_date"] = out["last_maintenance_date"].ffill()
343     out["days_since_maintenance"] = (
344         out["date"] - out["last_maintenance_date"]
345     ).dt.days.astype(int)
346     out["last_maintenance_type"] = _event_type_series(
347         out["maintenance_type"], seed_last_type
348     )
349     return _ensure_state_features(out, reference_date)
350
351
352 def build_periodic_policy_frame(
353     spec: PolicySpec,
354     asset: str,
355     start_date: pd.Timestamp,
356     end_date: pd.Timestamp,
357     reference_date: pd.Timestamp,
358     maintenance: pd.DataFrame,
359 ) -> pd.DataFrame:
360     """按中维护与大维护最大间隔生成日历，同日以大维护覆盖。"""
361
362     if spec.medium_interval_days is None or spec.major_interval_days is None:
363         raise ValueError("周期策略必须给出中维护和大维护间隔")
364     history = maintenance.loc[maintenance["asset"].astype(str) == asset].sort_values(

```

```

365     "maintenance_date"
366 )
367 last_event = history.iloc[-1]
368 major_history = history.loc[history["maintenance_type"].eq("major")]
369 last_any_date = pd.Timestamp(last_event["maintenance_date"])
370 last_major_date = (
371     pd.Timestamp(major_history.iloc[-1]["maintenance_date"])
372     if len(major_history)
373     else last_any_date
374 )
375 last_type = str(last_event["maintenance_type"])
376 events: list[tuple[pd.Timestamp, str]] = []
377 minimum_gap = int(spec.cooldown_days or min(int(spec.medium_interval_days), 30))
378 for date in pd.date_range(pd.Timestamp(start_date) + pd.Timedelta(days=1), end_date,
freq="D"):
379     action: str | None = None
380     days_since_any = (date - last_any_date).days
381     days_since_major = (date - last_major_date).days
382     days_to_major = int(spec.major_interval_days) - days_since_major
383     if (
384         days_since_major >= int(spec.major_interval_days)
385         and days_since_any >= minimum_gap
386     ):
387         action = "major"
388     elif (
389         days_since_any >= int(spec.medium_interval_days)
390         and days_to_major > minimum_gap
391     ):
392         action = "medium"
393     if action is not None:
394         events.append((date, action))
395         last_any_date = date
396         last_type = action
397         if action == "major":
398             last_major_date = date
399     event_map = dict(events)
400     frame = pd.DataFrame(
401         {
402             "asset": asset,
403             "date": pd.date_range(pd.Timestamp(start_date) + pd.Timedelta(days=1),
end_date),
404         }
405     )
406     frame["maintenance_type"] = frame["date"].map(event_map)
407     seed = history.iloc[-1]
408     return finalize_policy_frame(
409         frame,
410         pd.Timestamp(seed["maintenance_date"]),
411         str(seed["maintenance_type"]),
412         reference_date,
413     )
414
415 def _response_parameters(
416     response: pd.DataFrame,
417     scenario: str,
418 ) -> dict[str, tuple[float, float]]:
419     lookup = response.set_index("maintenance_type")
420     if scenario not in RESPONSE_SCENARIOS:
421         raise ValueError(f"未知维护效果情景: {scenario}")
422     if scenario == "empirical":
423         medium = (
424             float(lookup.loc["medium", "instant_empirical"]),
425             float(lookup.loc["medium", "retained_empirical"]),
426         )
427         major = (
428             float(lookup.loc["major", "instant_empirical"]),
429             float(lookup.loc["major", "retained_empirical"]),
430         )
431     else:
432         medium = (
433             float(lookup.loc["medium", "instant_shrunk"]),
434             float(lookup.loc["medium", "retained_shrunk"]),
435         )
436         major = (
437             float(lookup.loc["major", "instant_shrunk"]),
438             float(lookup.loc["major", "retained_shrunk"]),
439         )
440         if scenario == "major_equal":
441             major = medium
442     if scenario == "empirical":
443         pooled = (
444             float(lookup["pooled_instant"].iloc[0]),
445             float(lookup["pooled_retained"].iloc[0]),
446         )
447     else:
448         event_weights = lookup["recorded_events"].to_numpy(dtype=float)
449         pooled = (
450             float(np.average(lookup["instant_shrunk"], weights=event_weights)),
451             float(np.average(lookup["retained_shrunk"], weights=event_weights)),
452         )
453     return {
454         "medium": (medium[0] - pooled[0], medium[1] - pooled[1]),
455         "major": (major[0] - pooled[0], major[1] - pooled[1]),
456     }

```

```

457     }
458
459
460 def typed_response_profile(
461     frame: pd.DataFrame,
462     response: pd.DataFrame,
463     scenario: str = "shrunk",
464 ) -> np.ndarray:
465     """在问题二通用维护重置上添加中/大维护类型差异。"""
466
467     params = _response_parameters(response, scenario)
468     age = pd.to_numeric(frame["days_since_maintenance"],
469 errors="coerce").fillna(9999).to_numpy()
470 event_type = frame["last_maintenance_type"].astype(str).to_numpy()
471 profile = np.zeros(len(frame), dtype=float)
472 for maintenance_type in ("medium", "major"):
473     instant_delta, retained_delta = params[maintenance_type]
474     mask = event_type == maintenance_type
475     profile[mask & (age <= 7)] = instant_delta
476     profile[mask & (age >= 8) & (age <= 30)] = retained_delta
477     taper = mask & (age >= 31) & (age <= 60)
478     profile[taper] = retained_delta * (60.0 - age[taper]) / 30.0
479
480
481 def maintenance_damage_correction(
482     frame: pd.DataFrame,
483     degradation_per_year: float,
484     medium_rate_per_year: float,
485     major_rate_per_year: float,
486     forecast_origin: pd.Timestamp,
487     damage_share: float = MAIN_DAMAGE_SHARE,
488 ) -> np.ndarray:
489     """将问题二总退化拆为自然退化与随政策变化的维护损伤。
490
491     在历史年频率下，期望修正为 0，因此现行策略能复现问题二；
492     改变频率时，按中维护 1、大维护 4 的相对强度累计损伤。
493     """
494
495     if not 0.0 <= damage_share <= 1.0:
496         raise ValueError("维护损伤份额必须位于 [0,1]")
497     intensity = float(
498         medium_rate_per_year + MAJOR_DAMAGE_MULTIPLIER * major_rate_per_year
499     )
500     if intensity <= 0.0 or damage_share == 0.0:
501         return np.zeros(len(frame), dtype=float)
502     elapsed = (
503         pd.to_datetime(frame["date"]) - pd.Timestamp(forecast_origin)
504     ).dt.days.to_numpy(dtype=float) / ANNUAL_PERIOD_DAYS
505     event_weight = np.where(
506         frame["maintenance_type"].eq("medium"),
507         1.0,
508         np.where(
509             frame["maintenance_type"].eq("major"),
510             MAJOR_DAMAGE_MULTIPLIER,
511             0.0,
512         ),
513     )
514     cumulative_weight = np.cumsum(event_weight)
515     return damage_share * float(degradation_per_year) * (
516         elapsed - cumulative_weight / intensity
517     )
518
519
520 def point_policy_prediction(
521     frame: pd.DataFrame,
522     model: HierarchicalStateModel,
523     response: pd.DataFrame,
524     rates: Mapping[str, Any],
525     forecast_origin: pd.Timestamp,
526     *,
527     damage_share: float = MAIN_DAMAGE_SHARE,
528     response_scenario: str = "shrunk",
529 ) -> np.ndarray:
530     asset = str(frame["asset"].iloc[0])
531     base = model.predict(frame)
532     correction = maintenance_damage_correction(
533         frame,
534         model.parameter(f"degradation:{asset}"),
535         float(rates["medium_rate_per_year"]),
536         float(rates["major_rate_per_year"]),
537         forecast_origin,
538         damage_share,
539     )
540     return base + typed_response_profile(frame, response, response_scenario) + correction
541
542
543 def _generic_fouling_effect(model: HierarchicalStateModel, ages: np.ndarray) ->
544 np.ndarray:
545     ages = np.maximum(np.asarray(ages, dtype=float), 0.0)
546     return (
547         model.parameter("fouling_0_30") * np.minimum(ages, 30.0) / 30.0
548         + model.parameter("fouling_30_90")
549         * np.minimum(np.maximum(ages - 30.0, 0.0), 60.0)

```

```

549         / 60.0
550     )
551
552
553 def build_trigger_policy_frame(
554     spec: PolicySpec,
555     asset: str,
556     start_date: pd.Timestamp,
557     end_date: pd.Timestamp,
558     reference_date: pd.Timestamp,
559     maintenance: pd.DataFrame,
560     history_panel: pd.DataFrame,
561     model: HierarchicalStateModel,
562     response: pd.DataFrame,
563     rates: Mapping[str, Any],
564     damage_share: float = MAIN_DAMAGE_SHARE,
565 ) -> pd.DataFrame:
566     """根据未来 30 日预测余量生成状态触发日历。
567
568     触发日历使用点估计状态生成，后续蒙特卡评估时把该日历视为
569     一套现场可执行的规则结果，而不是预知未来噪声的完美闭环控制。
570     """
571
572     if (
573         spec.trigger_level is None
574         or spec.cooldown_days is None
575         or spec.major_interval_days is None
576     ):
577         raise ValueError("状态触发策略缺少触发线、冷却期或大维护间隔")
578
579     asset_history = maintenance.loc[
580         maintenance["asset"].astype(str) == asset
581     ].sort_values("maintenance_date")
582     last_event = asset_history.iloc[-1]
583     major_history = asset_history.loc[asset_history["maintenance_type"].eq("major")]
584     last_any_date = pd.Timestamp(last_event["maintenance_date"])
585     last_type = str(last_event["maintenance_type"])
586     last_major_date = (
587         pd.Timestamp(major_history.iloc[-1]["maintenance_date"])
588         if len(major_history)
589         else last_any_date
590     )
591
592     dates = pd.date_range(pd.Timestamp(start_date) + pd.Timedelta(days=1), end_date,
593 freq="D")
594     clean_frame = pd.DataFrame(
595         {
596             "asset": asset,
597             "date": dates,
598             "days_since_maintenance": np.zeros(len(dates), dtype=int),
599         }
600     )
601     clean_base = model.predict(_ensure_state_features(clean_frame, reference_date))
602     elapsed = (dates - pd.Timestamp(start_date)).days.to_numpy(dtype=float) /
603
604     ANNUAL PERIOD DAYS
605     degradation = model.parameter(f"degradation:{asset}")
606     intensity = float(
607         rates["medium_rate_per_year"]
608         + MAJOR_DAMAGE_MULTIPLIER * rates["major_rate_per_year"]
609     )
610     history_values = (
611         history_panel.loc[history_panel["asset"].astype(str) == asset]
612         .sort_values("date")["performance_model"]
613         .interpolate(limit_direction="both")
614         .dropna()
615         .to_numpy(dtype=float)
616     )
617     rolling_values = list(history_values[-(ROLLING_YEAR_DAYS - 1) :])
618     events: dict[pd.Timestamp, str] = {}
619     cumulative_damage_weight = 0.0
620     diagnostic_due = False
621
622     # 类型修正只与距上次维护时间有关，在循环中使用小向量评估。
623     response_params = _response_parameters(response, "shrunk")
624
625     def type_adjustment(event_type: str, ages: np.ndarray) -> np.ndarray:
626         instant_delta, retained_delta = response_params[event_type]
627         ages = np.asarray(ages, dtype=float)
628         values = np.zeros(len(ages), dtype=float)
629         values[ages <= 7] = instant_delta
630         values[(ages >= 8) & (ages <= 30)] = retained_delta
631         taper = (ages >= 31) & (ages <= 60)
632         values[taper] = retained_delta * (60.0 - ages[taper]) / 30.0
633         return values
634
635     for index, date in enumerate(dates):
636         action: str | None = None
637         if diagnostic_due:
638             action = "major"
639         elif (date - last_major_date).days >= int(spec.major_interval_days):
640             action = "major"
641         elif (
642             index % 7 == 0
643             and (date - last_any_date).days >= int(spec.cooldown_days)

```



```

641 ):
642     stop = min(index + 31, len(dates))
643     offsets = np.arange(stop - index, dtype=float)
644     future_ages = (date - last_any_date).days + offsets
645     fouling = generic_fouling_effect(model, future_ages)
646     response_adjustment = type_adjustment(last_type, future_ages)
647     if intensity > 0.0:
648         damage = damage_share * degradation * (
649             elapsed[index:stop] - cumulative_damage_weight / intensity
650         )
651     else:
652         damage = np.zeros(stop - index, dtype=float)
653     no_action_forecast = (
654         clean_base[index:stop] + fouling + response_adjustment + damage
655     )
656     if float(np.min(no_action_forecast)) < float(spec.trigger_level):
657         action = "medium"
658
659     if action is not None:
660         events[date] = action
661         last_any_date = date
662         last_type = action
663         cumulative_damage_weight += (
664             MAJOR_DAMAGE_MULTIPLIER if action == "major" else 1.0
665         )
666         if action == "major":
667             last_major_date = date
668             diagnostic_due = False
669
670     age = float((date - last_any_date).days)
671     point = clean_base[index] + float(generic_fouling_effect(model, np.array([age])))
672
673     point += float(type_adjustment(last_type, np.array([age]))[0])
674     if intensity > 0.0:
675         point += damage_share * degradation * (
676             elapsed[index] - cumulative_damage_weight / intensity
677         )
678     rolling_values.append(point)
679     if len(rolling_values) >= ROLLING_YEAR_DAYS:
680         rolling_mean = float(np.mean(rolling_values[-ROLLING_YEAR_DAYS:]))
681         if rolling_mean < LIFETIME_THRESHOLD and action != "major":
682             diagnostic_due = True
683
684     frame = pd.DataFrame({"asset": asset, "date": dates})
685     frame["maintenance_type"] = frame["date"].map(events)
686     return finalize_policy_frame(
687         frame,
688         pd.Timestamp(last_event["maintenance_date"]),
689         str(last_event["maintenance_type"]),
690         reference_date,
691     )
692
693 def build_current_policy_frame(
694     asset: str,
695     start_date: pd.Timestamp,
696     end_date: pd.Timestamp,
697     reference_date: pd.Timestamp,
698     maintenance: pd.DataFrame,
699     fixed_schedule: pd.DataFrame,
700 ) -> pd.DataFrame:
701     full = build_future_frame(fixed_schedule, start_date, end_date, reference_date)
702     frame = full.loc[full["asset"].astype(str) == asset].copy()
703     asset_history = maintenance.loc[
704         maintenance["asset"].astype(str) == asset
705     ].sort_values("maintenance_date")
706     last_event = asset_history.iloc[-1]
707     frame["last_maintenance_type"] = event_type_series(
708         frame["maintenance_type"], str(last_event["maintenance_type"])
709     )
710     return frame.reset_index(drop=True)
711
712 def build_policy_frame(
713     spec: PolicySpec,
714     asset: str,
715     start_date: pd.Timestamp,
716     end_date: pd.Timestamp,
717     reference_date: pd.Timestamp,
718     maintenance: pd.DataFrame,
719     fixed_schedule: pd.DataFrame,
720     history_panel: pd.DataFrame,
721     model: HierarchicalStateModel,
722     response: pd.DataFrame,
723     rates: Mapping[str, Any],
724 ) -> pd.DataFrame:
725     if spec.policy_kind == "current":
726         return build_current_policy_frame(
727             asset,
728             start_date,
729             end_date,
730             reference_date,
731             maintenance,
732             fixed_schedule,
733         )

```

```

735 if spec.policy_kind == "periodic":
736     return build_periodic_policy_frame(
737         spec,
738         asset,
739         start_date,
740         end_date,
741         reference_date,
742         maintenance,
743     )
744 if spec.policy_kind == "trigger":
745     return build_trigger_policy_frame(
746         spec,
747         asset,
748         start_date,
749         end_date,
750         reference_date,
751         maintenance,
752         history_panel,
753         model,
754         response,
755         rates,
756     )
757 raise ValueError(f"未知策略类型: {spec.policy_kind}")
758
759 def _rolling_forecast(
760     asset_history: pd.DataFrame,
761     prediction: np.ndarray,
762 ) -> np.ndarray:
763     historical = (
764         asset_history.sort_values("date")["performance_model"]
765         .interpolate(limit_direction="both")
766         .dropna()
767         .to_numpy(dtype=float)
768     )
769     historical = historical[-(ROLLING_YEAR_DAYS - 1) :]
770     if len(historical) < ROLLING_YEAR_DAYS - 1:
771         raise ValueError("历史日数不足以计算滚动 365 日均值")
772     combined = np.concatenate([historical, np.asarray(prediction, dtype=float)])
773     cumulative = np.cumsum(np.concatenate([[0.0], combined]))
774     return (
775         cumulative[ROLLING_YEAR_DAYS:] - cumulative[:-ROLLING_YEAR_DAYS]
776     ) / ROLLING_YEAR_DAYS
777
778 def joint_failure_index(
779     prediction: np.ndarray,
780     rolling: np.ndarray,
781     maintenance_type: Sequence[Any],
782     recovery_level: float = LIFETIME_THRESHOLD,
783 ) -> tuple[int | None, int | None]:
784     """返回年均阈值下穿与联合寿命终点下标。"""
785     below = np.flatnonzero(np.asarray(rolling) < LIFETIME_THRESHOLD)
786     if len(below) == 0:
787         return None, None
788     crossing = int(below[0])
789     types = np.asarray(maintenance_type, dtype=object)
790     major_indices = np.flatnonzero(types == "major")
791     for major_index in major_indices[major_indices >= crossing]:
792         if major_index + 30 >= len(prediction):
793             break
794         post = np.asarray(prediction)[major_index + 8 : major_index + 31]
795         if len(post) >= 20 and float(np.mean(post)) < recovery_level:
796             return crossing, int(major_index + 30)
797     return crossing, None
798
799 def _historical_cost(rates: Mapping[str, Any], costs: CostConfig) -> float:
800     return (
801         costs.medium * float(rates["historical_medium_events_equiv"])
802         + costs.major * float(rates["historical_major_events_equiv"])
803     )
804
805 def evaluate_point_policy(
806     spec: PolicySpec,
807     asset: str,
808     frame: pd.DataFrame,
809     history_panel: pd.DataFrame,
810     model: HierarchicalStateModel,
811     response: pd.DataFrame,
812     rates: Mapping[str, Any],
813     forecast_origin: pd.Timestamp,
814     costs: CostConfig = BASE_COSTS,
815     *,
816     damage_share: float = MAIN_DAMAGE_SHARE,
817     response_scenario: str = "shrunk",
818 ) -> dict[str, Any]:
819     prediction = point_policy_prediction(
820         frame,
821         model,
822         response,
823         rates,
824         forecast_origin,

```



```

830     damage_share=damage_share,
831     response_scenario=response_scenario,
832 )
833 asset_history = history_panel.loc[history_panel["asset"].astype(str) == asset]
834 rolling = rolling_forecast(asset_history, prediction)
835 crossing, failure = joint_failure_index(
836     prediction,
837     rolling,
838     frame["maintenance_type"].to_numpy(),
839 )
840 endpoint = len(frame) - 1 if failure is None else int(failure)
841 dates = pd.DatetimeIndex(frame["date"])
842 endpoint_date = pd.Timestamp(dates[endpoint])
843 total_lifetime_days = float((endpoint_date - pd.Timestamp(COMMISSION_DATE)).days)
844 medium_count = int(frame.iloc[: endpoint + 1][["maintenance_type"].eq("medium").sum()})
845 major_count = int(frame.iloc[: endpoint + 1][["maintenance_type"].eq("major").sum()})
846 lifecycle_cost = (
847     costs.purchase
848     + historical_cost(rates, costs)
849     + costs.medium * medium_count
850     + costs.major * major_count
851 )
852 annual_cost = lifecycle_cost / (total_lifetime_days / ANNUAL_PERIOD_DAYS)
853 future_years = max(float((endpoint_date - pd.Timestamp(forecast_origin)).days /
ANNUAL_PERIOD_DAYS), 1e-9)
854 return {
855     "asset": asset,
856     "policy_id": spec.policy_id,
857     "policy_kind": spec.policy_kind,
858     "evaluation_stage": "point",
859     "simulation_paths": 1,
860     "mean annual cost": annual_cost,
861     "median annual cost": annual_cost,
862     "p10 annual cost": annual_cost,
863     "p90 annual cost": annual_cost,
864     "cvar90 annual cost": annual_cost,
865     "mean total lifetime years": total_lifetime_days / ANNUAL_PERIOD_DAYS,
866     "median total lifetime years": total_lifetime_days / ANNUAL_PERIOD_DAYS,
867     "p10 total lifetime years": total_lifetime_days / ANNUAL_PERIOD_DAYS,
868     "p90 total lifetime years": total_lifetime_days / ANNUAL_PERIOD_DAYS,
869     "mean future medium events": float(medium_count),
870     "mean future major events": float(major_count),
871     "mean medium events per future year": medium_count / future_years,
872     "mean major events per future year": major_count / future_years,
873     "median failure date": endpoint_date,
874     "censored rate": float(failure is None),
875     "threshold crossing date": (
876         pd.NaT if crossing is None else pd.Timestamp(dates[crossing])
877     ),
878     "damage_share": damage_share,
879     "response_scenario": response_scenario,
880 }
881
882
883 def response_uncertainty_bases(
884     frame: pd.DataFrame,
885 ) -> dict[str, np.ndarray]:
886     age = pd.to_numeric(frame["days_since_maintenance"],
errors="coerce").fillna(9999).to_numpy()
887     event_type = frame["last_maintenance_type"].astype(str).to_numpy()
888     bases: dict[str, np.ndarray] = {}
889     for maintenance_type in ("medium", "major"):
890         mask = event_type == maintenance_type
891         instant = (mask & (age <= 7)).astype(float)
892         retained = (mask & (age >= 8) & (age <= 30)).astype(float)
893         taper = mask & (age >= 31) & (age <= 60)
894         retained[taper] = (60.0 - age[taper]) / 30.0
895         bases[f"{maintenance_type}_instant"] = instant
896         bases[f"{maintenance_type}_retained"] = retained
897     return bases
898
899
900 def simulate_policy_asset(
901     spec: PolicySpec,
902     asset: str,
903     frame: pd.DataFrame,
904     history_panel: pd.DataFrame,
905     scenario_models: Sequence[HierarchicalStateModel],
906     response: pd.DataFrame,
907     rates: Mapping[str, Any],
908     forecast_origin: pd.Timestamp,
909     *,
910     paths: int,
911     seed: int,
912     costs: CostConfig = BASE_COSTS,
913     evaluation_stage: str = "final",
914 ) -> tuple[dict[str, Any], pd.DataFrame]:
915     """用公共随机数评估单设备、单策略的寿命与年均成本。"""
916
917     if paths <= 0:
918         raise ValueError("仿真路径数必须为正")
919     if not scenario_models:
920         raise ValueError("至少需要一个退化情景模型")
921

```

```

922     rng = np.random.default_rng(seed + ASSETS.index(asset) * 1000)
923     dates = pd.DatetimeIndex(frame["date"])
924     horizon = len(frame)
925     elapsed = (dates - pd.Timestamp(forecast_origin)).days.to_numpy(dtype=float) /
ANNUAL_PERIOD_DAYS
926     scenario_predictions = np.vstack([model.predict(frame) for model in scenario_models])
927     scenario_degradation = np.array(
928         [model.parameter(f"degradation:{asset}") for model in scenario_models],
929         dtype=float,
930     )
931     scenario_residual = np.array(
932         [model.residual_std for model in scenario_models], dtype=float
933     )
934     model_choices = rng.integers(0, len(scenario_models), size=paths)
935     degradation_multiplier = np.exp(
936         rng.normal(-0.5 * 0.10**2, 0.10, size=paths)
937     )
938     chosen_degradation = scenario_degradation[model_choices]
939     degradation_draw = chosen_degradation * degradation_multiplier
940     level_shift = rng.normal(
941         0.0, scenario_residual[model_choices] / np.sqrt(60.0), size=paths
942     )
943     damage_choices = rng.choice(
944         np.asarray(DAMAGE_SHARES, dtype=float),
945         size=paths,
946         p=np.array([0.2, 0.6, 0.2]),
947     )
948     response_choices = rng.choice(
949         np.arange(len(RESPONSE_SCENARIOS)),
950         size=paths,
951         p=np.array([0.6, 0.2, 0.2]),
952     )
953     response_profiles = np.vstack(
954         [typed_response_profile(frame, response, scenario) for scenario in
RESPONSE_SCENARIOS]
955     )
956     response_bases = _response_uncertainty_bases(frame)
957     response_lookup = response.set_index("maintenance_type")
958
959     intensity = float(
960         rates["medium_rate_per_year"]
961         + MAJOR_DAMAGE_MULTIPLIER * rates["major_rate_per_year"]
962     )
963     event_weight = np.where(
964         frame["maintenance_type"].eq("medium"),
965         1.0,
966         np.where(
967             frame["maintenance_type"].eq("major"),
968             MAJOR_DAMAGE_MULTIPLIER,
969             0.0,
970         ),
971     )
972     cumulative_weight = np.cumsum(event_weight)
973     damage_gap = (
974         elapsed - cumulative_weight / intensity
975         if intensity > 0.0
976         else np.zeros(horizon, dtype=float)
977     )
978
979     asset_history = history_panel.loc[
980         history_panel["asset"].astype(str) == asset
981     ].sort_values("date")
982     past = (
983         asset_history["performance_model"]
984         .interpolate(limit_direction="both")
985         .dropna()
986         .to_numpy(dtype=float)
987     )[-(ROLLING_YEAR_DAYS - 1) : ]
988     if len(past) < ROLLING_YEAR_DAYS - 1:
989         raise ValueError(f"设备 {asset} 缺少滚动年均历史窗口")
990
991     medium_cumulative
992     = np.cumsum(frame["maintenance_type"].eq("medium").to_numpy(dtype=int))
993     major_cumulative
994     = np.cumsum(frame["maintenance_type"].eq("major").to_numpy(dtype=int))
995     major_indices = np.flatnonzero(frame["maintenance_type"].eq("major").to_numpy())
996     annual_costs = np.empty(paths, dtype=float)
997     lifetime_years = np.empty(paths, dtype=float)
998     future_medium = np.empty(paths, dtype=float)
999     future_major = np.empty(paths, dtype=float)
1000     failure_indices = np.full(paths, -1, dtype=int)
1001     crossing_indices = np.full(paths, -1, dtype=int)
1002
1003     chunk_size = 100
1004     for start in range(0, paths, chunk_size):
1005         stop = min(start + chunk_size, paths)
1006         selected = model_choices[start:stop]
1007         prediction = scenario_predictions[selected].copy()
1008         delta = degradation_draw[start:stop] - chosen_degradation[start:stop]
1009         prediction -= delta[:, np.newaxis] * elapsed[np.newaxis, :]
1010         prediction += level_shift[start:stop, np.newaxis]
1011         prediction += response_profiles[response_choices[start:stop]]
1012         prediction += (

```

```

1011         damage_choices[start:stop, np.newaxis]
1012         * degradation_draw[start:stop, np.newaxis]
1013         * damage_gap[np.newaxis, :]
1014     )
1015
1016     # 事件效果的均值不确定性按路径抽样, 避免把 17 次
1017     # 大维护的样本误差忽略掉。
1018     for maintenance_type in ("medium", "major"):
1019         instant_sd = float(
1020             response_lookup.loc[maintenance_type, "instant_standard_error"]
1021         )
1022         retained_sd = float(
1023             response_lookup.loc[maintenance_type, "retained_standard_error"]
1024         )
1025         instant_draw = rng.normal(0.0, instant_sd, size=stop - start)
1026         retained_draw = rng.normal(0.0, retained_sd, size=stop - start)
1027         prediction += (
1028             instant_draw[:, np.newaxis]
1029             * response_bases[f"{maintenance_type}_instant"][np.newaxis, :]
1030             + retained_draw[:, np.newaxis]
1031             * response_bases[f"{maintenance_type}_retained"][np.newaxis, :]
1032         )
1033
1034     historical = np.broadcast_to(past, (stop - start, len(past)))
1035     combined = np.concatenate([historical, prediction], axis=1)
1036     cumulative = np.cumsum(
1037         np.concatenate([np.zeros((stop - start, 1)), combined], axis=1), axis=1
1038     )
1039     rolling = (
1040         cumulative[:, ROLLING_YEAR_DAYS:]
1041         - cumulative[:, :-ROLLING_YEAR_DAYS]
1042     ) / ROLLING_YEAR_DAYS
1043     below = rolling < LIFETIME_THRESHOLD
1044     has_crossing = below.any(axis=1)
1045     local_crossings = np.where(has_crossing, below.argmax(axis=1), -1)
1046     crossing_indices[start:stop] = local_crossings
1047
1048     for local_index, crossing in enumerate(local_crossings):
1049         endpoint = horizon - 1
1050         if crossing >= 0:
1051             for major_index in major_indices[major_indices >= crossing]:
1052                 if major_index + 30 >= horizon:
1053                     break
1054                 post = prediction[local_index, major_index + 8 : major_index + 31]
1055                 if float(np.mean(post)) < LIFETIME_THRESHOLD:
1056                     endpoint = int(major_index + 30)
1057                 failure_indices[start + local_index] = endpoint
1058                 break
1059         endpoint_date = pd.Timestamp(dates[endpoint])
1060         years = float(
1061             (endpoint_date - pd.Timestamp(COMMISSION_DATE)).days
1062             / ANNUAL_PERIOD_DAYS
1063         )
1064         med = float(medium_cumulative[endpoint])
1065         maj = float(major_cumulative[endpoint])
1066         total_cost = (
1067             costs.purchase
1068             + historical_cost(rates, costs)
1069             + costs.medium * med
1070             + costs.major * maj
1071         )
1072         lifetime_years[start + local_index] = years
1073         future_medium[start + local_index] = med
1074         future_major[start + local_index] = maj
1075         annual_costs[start + local_index] = total_cost / years
1076
1077     worst_count = max(int(np.ceil(paths * 0.10)), 1)
1078     worst_costs = np.sort(annual_costs)[-worst_count:]
1079
1080     def q(values: np.ndarray, probability: float) -> float:
1081         return float(np.quantile(values, probability, method="linear"))
1082
1083     endpoint_indices = np.where(failure_indices >= 0, failure_indices, horizon - 1)
1084     median_endpoint = int(np.quantile(endpoint_indices, 0.50, method="higher"))
1085     future_years = np.maximum(
1086         lifetime_years
1087         - float(
1088             (pd.Timestamp(forecast_origin) - pd.Timestamp(COMMISSION_DATE)).days
1089             / ANNUAL_PERIOD_DAYS
1090         ),
1091         1e-9,
1092     )
1093     summary = {
1094         "asset": asset,
1095         "policy_id": spec.policy_id,
1096         "policy_kind": spec.policy_kind,
1097         "evaluation_stage": evaluation_stage,
1098         "simulation_paths": paths,
1099         "mean annual cost": float(np.mean(annual_costs)),
1100         "median annual cost": q(annual_costs, 0.50),
1101         "p10 annual cost": q(annual_costs, 0.10),
1102         "p90 annual cost": q(annual_costs, 0.90),
1103         "cvar90 annual cost": float(np.mean(worst_costs)),
1104         "mean total lifetime years": float(np.mean(lifetime_years)),
1105         "median total lifetime years": q(lifetime_years, 0.50),

```

```

1106         "p10_total_lifetime_years": q(lifetime_years, 0.10),
1107         "p90_total_lifetime_years": q(lifetime_years, 0.90),
1108         "mean_future_medium_events": float(np.mean(future_medium)),
1109         "mean_future_major_events": float(np.mean(future_major)),
1110         "mean_medium_events_per_future_year": float(np.mean(future_medium /
future_years)),
1111         "mean_major_events_per_future_year": float(np.mean(future_major / future_years)),
1112         "median_failure_date": pd.Timestamp(dates[median_endpoint]),
1113         "censored_rate": float(np.mean(failure_indices < 0)),
1114         "threshold_crossing_date": (
1115             pd.Nat
1116             if np.all(crossing_indices < 0)
1117             else pd.Timestamp(
1118                 dates[
1119                     int(
1120                         np.quantile(
1121                             np.where(crossing_indices >= 0, crossing_indices, horizon
- 1),
1122                             0.50,
1123                             method="higher",
1124                         )
1125                     )
1126                 ]
1127             ),
1128         ),
1129         "damage_share": MAIN_DAMAGE_SHARE,
1130         "response_scenario": "mixture",
1131     }
1132     distributions = pd.DataFrame(
1133     {
1134         "asset": asset,
1135         "policy_id": spec.policy_id,
1136         "path_id": np.arange(paths, dtype=int),
1137         "annual_cost": annual_costs,
1138         "total_lifetime_years": lifetime_years,
1139         "future_medium_events": future_medium,
1140         "future_major_events": future_major,
1141         "failure_index": failure_indices,
1142         "damage_share": damage_choices,
1143         "response_scenario": [RESPONSE_SCENARIOS[i] for i in response_choices],
1144     }
1145 )
1146     return summary, distributions
1147
1148 def select_with_tie_break(evaluations: pd.DataFrame) -> pd.Series:
1149     """成本近似相同时，仅接受同时“维护更少且寿命不短”的替代。"""
1150
1151     if evaluations.empty:
1152         raise ValueError("没有可选策略")
1153     work = evaluations.copy()
1154     work["mean_total_events"] = (
1155         work["mean_future_medium_events"] + work["mean_future_major_events"]
1156     )
1157     minimum_row = work.sort_values(["mean_annual_cost", "policy_id"]).iloc[0]
1158     candidates = work.loc[
1159         (work["mean_annual_cost"] <= float(minimum_row["mean_annual_cost"]) * 1.01)
1160         & (work["mean_total_events"] <= float(minimum_row["mean_total_events"]))
1161         & (
1162             work["median_total_lifetime_years"]
1163             >= float(minimum_row["median_total_lifetime_years"])
1164         )
1165     ].copy()
1166     if candidates.empty:
1167         return minimum_row
1168     return candidates.sort_values(
1169         ["mean_annual_cost", "mean_total_events", "median_total_lifetime_years",
"policy_id"],
1170         ascending=[True, True, False, True],
1171     ).iloc[0]
1172
1173 def plot_cost_frontier(
1174     point_evaluations: pd.DataFrame,
1175     uniform_policy_id: str,
1176 ) -> Path:
1177     aggregate = (
1178         point_evaluations.groupby(["policy_id", "policy_kind"], as_index=False)
1179         .agg(
1180             fleet_annual_cost=("mean_annual_cost", "sum"),
1181             mean_lifetime=("median_total_lifetime_years", "mean"),
1182         )
1183     )
1184     colors = {"current": "#d95f02", "periodic": "#2457a7", "trigger": "#2b8c6b"}
1185     fig, ax = plt.subplots(figsize=(9.5, 6.0))
1186     for kind, block in aggregate.groupby("policy_kind"):
1187         ax.scatter(
1188             block["mean_lifetime"],
1189             block["fleet_annual_cost"],
1190             s=28,
1191             alpha=0.65,
1192             label={
1193                 "current": "现行", "periodic": "周期", "trigger": "状态触发"
1194             }.get(kind, kind),

```

```

1195         color=colors.get(kind, "#777777"),
1196     )
1197     highlight = aggregate.loc[aggregate["policy_id"].eq(uniform_policy_id)]
1198     if len(highlight):
1199         ax.scatter(
1200             highlight["mean_lifetime"],
1201             highlight["fleet_annual_cost"],
1202             s=150,
1203             facecolor="none",
1204             edgecolor="#111111",
1205             linewidth=1.8,
1206             label="全厂统一优选",
1207             zorder=5,
1208         )
1209     ax.set_xlabel("10台设备平均总寿命 (年)")
1210     ax.set_ylabel("全厂全生命周期年均成本 (万元/年)")
1211     ax.set_title("候选维护策略的寿命-成本前沿")
1212     ax.legend(frameon=False, ncol=2)
1213     return _save_figure(fig, "q3_01_cost_frontier.png")
1214
1215
1216 def plot_cost_comparison(comparison: pd.DataFrame) -> Path:
1217     work = comparison.set_index("asset").reindex(ASSETS)
1218     x = np.arange(len(work))
1219     width = 0.38
1220     fig, ax = plt.subplots(figsize=(10.5, 5.8))
1221     ax.bar(
1222         x - width / 2,
1223         work["current_mean_annual_cost"],
1224         width,
1225         label="现行策略",
1226         color="#9aa0a6",
1227     )
1228     ax.bar(
1229         x + width / 2,
1230         work["optimal_mean_annual_cost"],
1231         width,
1232         label="优选策略",
1233         color="#2457a7",
1234     )
1235     ax.set_xticks(x, work.index)
1236     ax.set_ylabel("年均成本 (万元/年)")
1237     ax.set_title("各设备现行与优选维护策略成本对比")
1238     ax.legend(frameon=False)
1239     return _save_figure(fig, "q3_02_current_vs_optimal.png")
1240
1241
1242 def plot_recommended_calendar(calendar: pd.DataFrame) -> Path:
1243     fig, ax = plt.subplots(figsize=(12.0, 6.4))
1244     y_lookup = {asset: index for index, asset in enumerate(ASSETS)}
1245     styles = {
1246         "medium": ("f2a900", "o", "中维护"),
1247         "major": ("d95f02", "s", "大维护"),
1248     }
1249     for maintenance_type, (color, marker, label) in styles.items():
1250         block = calendar.loc[calendar["maintenance_type"].eq(maintenance_type)]
1251         ax.scatter(
1252             pd.to_datetime(block["date"]),
1253             block["asset"].map(y_lookup),
1254             color=color,
1255             marker=marker,
1256             s=32 if maintenance_type == "medium" else 45,
1257             label=label,
1258             alpha=0.85,
1259         )
1260     ax.set_yticks(np.arange(len(ASSETS)), ASSETS)
1261     ax.invert_yaxis()
1262     ax.xaxis.set_major_locator(mdates.MonthLocator(interval=4))
1263     ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
1264     ax.set_xlabel("建议日期")
1265     ax.set_title("预测原点后三年推荐维护日历")
1266     ax.legend(frameon=False, ncol=2)
1267     fig.autofmt_xdate(rotation=30)
1268     return _save_figure(fig, "q3_03_recommended_calendar.png")
1269
1270
1271 def plot_savings_uncertainty(comparison: pd.DataFrame) -> Path:
1272     work = comparison.set_index("asset").reindex(ASSETS).copy()
1273     y = np.arange(len(work))
1274     mean = work["mean_savings_percent"].to_numpy(dtype=float)
1275     lower = work["p10_savings_percent"].to_numpy(dtype=float)
1276     upper = work["p90_savings_percent"].to_numpy(dtype=float) - mean
1277     fig, ax = plt.subplots(figsize=(9.5, 6.0))
1278     ax.errorbar(
1279         mean,
1280         y,
1281         xerr=np.vstack([np.maximum(lower, 0.0), np.maximum(upper, 0.0)]),
1282         fmt="o",
1283         color="#2457a7",
1284         ecolor="#7ba3d1",
1285         capsize=4,
1286     )
1287     ax.axvline(0.0, color="#d95f02", ls="--", lw=1.2)
1288     ax.set_yticks(y, work.index)

```



```

1289 ax.invert_yaxis()
1290 ax.set_xlabel("相对现行策略的年均成本节省率 (%) ")
1291 ax.set_title("优选策略节省率与80%仿真区间")
1292 return _save_figure(fig, "q3_04_savings_uncertainty.png")
1293
1294
1295 def load_q3_inputs() -> dict[str, Any]:
1296     ensure_output_dirs()
1297     panel_path = PROCESSED_DIR / "daily_panel.csv"
1298     if not panel_path.is_file():
1299         run_q1_scaffold()
1300     effects_path = RESULTS_DIR / "q1_maintenance_effects.csv"
1301     detail_path = RESULTS_DIR / "q1_event_effects_detail.csv"
1302     if not effects_path.is_file() or not detail_path.is_file():
1303         run_q1_formal()
1304     raw_panel = pd.read_csv(panel_path, parse_dates=["date", "last_maintenance_date"])
1305     panel = prepare_analysis_panel(raw_panel)
1306     maintenance = load_maintenance()
1307     effects = pd.read_csv(effects_path)
1308     event_detail = pd.read_csv(detail_path, parse_dates=["maintenance_date"])
1309     return {
1310         "raw_panel": raw_panel,
1311         "panel": panel,
1312         "maintenance": maintenance,
1313         "effects": effects,
1314         "event_detail": event_detail,
1315     }
1316
1317
1318 def run_q3_formal(
1319     *,
1320     screening_paths: int = SCREENING_PATHS,
1321     final_paths: int = FINAL_PATHS,
1322     validation_paths: int = VALIDATION_PATHS,
1323     forecast_horizon_years: int = FORECAST_HORIZON_YEARS,
1324     costs: CostConfig = BASE_COSTS,
1325 ) -> Dict[str, Any]:
1326     """运行问题三正式策略搜索、蒙特卡复验和图表输出。"""
1327
1328     inputs = load_q3_inputs()
1329     panel: pd.DataFrame = inputs["panel"]
1330     maintenance: pd.DataFrame = inputs["maintenance"]
1331     response = estimate_maintenance_response(inputs["effects"])
1332     response_validation = validate_maintenance_response(inputs["event_detail"])
1333     main_model = fit_hierarchical_state_model(panel, reference_date=panel["date"].min())
1334     outlier_path = RESULTS_DIR / "q1_residual_outliers.csv"
1335     residual_outliers = (
1336         pd.read_csv(outlier_path, parse_dates=["date"])
1337         if outlier_path.is_file()
1338         else pd.DataFrame(columns=["asset", "date", "is_residual_outlier"])
1339     )
1340     clean_model, _ = residual_outlier_sensitivity(panel, main_model, residual_outliers)
1341     scenario_models, _ = structural_sensitivity_models(panel)
1342     scenario_models = list(scenario_models) + [clean_model]
1343     fixed_schedule = estimate_fixed_schedule(maintenance)
1344     forecast_origin = pd.to_datetime(panel["date"].max())
1345     forecast_end = forecast_origin + pd.DateOffset(years=forecast_horizon_years)
1346     rates = historical_maintenance_rates(maintenance, forecast_origin)
1347     rates_lookup = rates.set_index("asset").to_dict(orient="index")
1348     specs = build_policy_space()
1349     spec_lookup = {spec.policy_id: spec for spec in specs}
1350
1351     def make_frame(asset: str, policy_id: str) -> pd.DataFrame:
1352         return build_policy_frame(
1353             spec_lookup[policy_id],
1354             asset,
1355             forecast_origin,
1356             forecast_end,
1357             main_model.reference_date,
1358             maintenance,
1359             fixed_schedule,
1360             panel,
1361             main_model,
1362             response,
1363             rates_lookup[asset],
1364         )
1365
1366     # 第一层：用主结构点估计评估全部规则。
1367     point_rows: list[dict[str, Any]] = []
1368     for asset in ASSETS:
1369         for spec in specs:
1370             frame = make_frame(asset, spec.policy_id)
1371             point_rows.append(
1372                 evaluate_point_policy(
1373                     spec,
1374                     asset,
1375                     frame,
1376                     panel,
1377                     main_model,
1378                     response,
1379                     rates_lookup[asset],
1380                     forecast_origin,
1381                     costs,
1382                 )
1383             )

```

```

1384 point_evaluations = pd.DataFrame(point_rows)
1385
1386 # 全厂统一策略必须对 10 台设备使用同一组规则参数。
1387 uniform_point = (
1388     point_evaluations.loc[~point_evaluations["policy_id"].eq("current")]
1389     .groupby(["policy_id", "policy_kind"], as_index=False)
1390     .agg(
1391         mean_annual_cost=("mean_annual_cost", "sum"),
1392         median_total_lifetime_years=("median_total_lifetime_years", "mean"),
1393         mean_future_medium_events=("mean_future_medium_events", "sum"),
1394         mean_future_major_events=("mean_future_major_events", "sum"),
1395     )
1396 )
1397 uniform_policy_id = str(_select_with_tie_break(uniform_point)["policy_id"])
1398
1399 # 第二层：每台选点估计前 5 名，并强制包含现行、最优周期和最优触发规则。
1400 screening_ids: dict[str, list[str]] = {}
1401 for asset in ASSETS:
1402     block = point_evaluations.loc[point_evaluations["asset"].eq(asset)]
1403     ids = block.nsmallest(5, "mean_annual_cost")["policy_id"].tolist()
1404     ids.append("current")
1405     ids.append(str(block.loc[block["policy_kind"].eq("periodic")].nsmallest(1,
1406 "mean_annual_cost").iloc[0]["policy_id"]))
1407     ids.append(str(block.loc[block["policy_kind"].eq("trigger")].nsmallest(1,
1408 "mean_annual_cost").iloc[0]["policy_id"]))
1409     screening_ids[asset] = list(dict.fromkeys(ids))
1410
1411 screening_rows: list[dict[str, Any]] = []
1412 for asset in ASSETS:
1413     for policy_id in screening_ids[asset]:
1414         summary = simulate_policy_asset(
1415             spec_lookup[policy_id],
1416             asset,
1417             make_frame(asset, policy_id),
1418             panel,
1419             scenario_models,
1420             response,
1421             rates_lookup[asset],
1422             forecast_origin,
1423             paths=screening_paths,
1424             seed=SEARCH_SEED,
1425             costs=costs,
1426             evaluation_stage="screening",
1427         )
1428     screening_rows.append(summary)
1429 screening_evaluations = pd.DataFrame(screening_rows)
1430
1431 # 第三层：每台对筛选第 1 名、现行和统一策略做 2,000 路径复验。
1432 final_ids: dict[str, list[str]] = {}
1433 for asset in ASSETS:
1434     block = screening_evaluations.loc[screening_evaluations["asset"].eq(asset)]
1435     ids = block.nsmallest(1, "mean_annual_cost")["policy_id"].tolist()
1436     ids.extend(["current", uniform_policy_id])
1437     final_ids[asset] = list(dict.fromkeys(ids))
1438
1439 final_rows: list[dict[str, Any]] = []
1440 distributions: dict[tuple[str, str], pd.DataFrame] = {}
1441 for asset in ASSETS:
1442     for policy_id in final_ids[asset]:
1443         summary, distribution = simulate_policy_asset(
1444             spec_lookup[policy_id],
1445             asset,
1446             make_frame(asset, policy_id),
1447             panel,
1448             scenario_models,
1449             response,
1450             rates_lookup[asset],
1451             forecast_origin,
1452             paths=final_paths,
1453             seed=SEARCH_SEED,
1454             costs=costs,
1455             evaluation_stage="final",
1456         )
1457     final_rows.append(summary)
1458     distributions[(asset, policy_id)] = distribution
1459 final_evaluations = pd.DataFrame(final_rows)
1460
1461 # 检查新的分类维护层在现行日历下与问题二寿命的衔接程度。
1462 q2_lifetime_path = RESULTS_DIR / "q2_lifetime_summary.csv"
1463 if q2_lifetime_path.is_file():
1464     q2_lifetime = pd.read_csv(q2_lifetime_path)
1465     q2_bridge = q2_lifetime[
1466         ["asset", "median_total_lifetime_years"]
1467     ].rename(columns={"median_total_lifetime_years": "q2_median_lifetime_years"})
1468     q3_current = final_evaluations.loc[
1469         final_evaluations["policy_id"].eq("current"),
1470         ["asset", "median_total_lifetime_years"],
1471     ].rename(columns={"median_total_lifetime_years":
1472 "q3_current_median_lifetime_years"})
1473     baseline_check = q2_bridge.merge(q3_current, on="asset", how="inner")
1474     baseline_check["difference_years"] = (
1475         baseline_check["q3_current_median_lifetime_years"]

```



```

1474         - baseline_check["q2_median_lifetime_years"]
1475     )
1476     baseline_check["absolute_difference_years"] = baseline_check[
1477         "difference_years"
1478     ].abs()
1479 else:
1480     baseline_check = pd.DataFrame(
1481         columns=[
1482             "asset",
1483             "q2_median_lifetime_years",
1484             "q3_current_median_lifetime_years",
1485             "difference_years",
1486             "absolute_difference_years",
1487         ]
1488     )
1489
1490     selected_rows: list[pd.Series] = []
1491     for asset in ASSETS:
1492         block = final_evaluations.loc[final_evaluations["asset"].eq(asset)]
1493         selected_rows.append(select_with_tie_break(block))
1494     selected = pd.DataFrame(selected_rows).reset_index(drop=True)
1495     selected["selection"] = "asset_specific_optimal"
1496     optimal_policy = selected.merge(
1497         policy_space_table(specs).drop(columns=["feasible_main"]),
1498         on=["policy_id", "policy_kind"],
1499         how="left",
1500     )
1501
1502     comparison_rows: list[dict[str, Any]] = []
1503     for asset in ASSETS:
1504         optimal_row = selected.loc[selected["asset"].eq(asset)].iloc[0]
1505         optimal_id = str(optimal_row["policy_id"])
1506         current_row = final_evaluations.loc[
1507             final_evaluations["asset"].eq(asset)
1508             & final_evaluations["policy_id"].eq("current")
1509         ].iloc[0]
1510         current_dist = distributions[(asset, "current")].sort_values("path_id")
1511         optimal_dist = distributions[(asset, optimal_id)].sort_values("path_id")
1512         savings = current_dist["annual_cost"].to_numpy() -
1513         optimal_dist["annual_cost"].to_numpy()
1514         savings_percent = savings / current_dist["annual_cost"].to_numpy() * 100.0
1515         comparison_rows.append(
1516             {
1517                 "asset": asset,
1518                 "optimal_policy_id": optimal_id,
1519                 "optimal_policy_kind": optimal_row["policy_kind"],
1520                 "current_mean_annual_cost": float(current_row["mean_annual_cost"]),
1521                 "optimal_mean_annual_cost": float(optimal_row["mean_annual_cost"]),
1522                 "mean_annual_saving": float(np.mean(savings)),
1523                 "mean_savings_percent": float(np.mean(savings_percent)),
1524                 "p10_savings_percent": float(np.quantile(savings_percent, 0.10)),
1525                 "p90_savings_percent": float(np.quantile(savings_percent, 0.90)),
1526                 "probability_of_positive_saving": float(np.mean(savings > 0.0)),
1527                 "current_median_lifetime_years":
1528                 float(current_row["median_total_lifetime_years"]),
1529                 "optimal_median_lifetime_years":
1530                 float(optimal_row["median_total_lifetime_years"]),
1531                 "current_medium_events_per_year":
1532                 float(current_row["mean_medium_events_per_future_year"]),
1533                 "optimal_medium_events_per_year":
1534                 float(optimal_row["mean_medium_events_per_future_year"]),
1535                 "current_major_events_per_year":
1536                 float(current_row["mean_major_events_per_future_year"]),
1537                 "optimal_major_events_per_year":
1538                 float(optimal_row["mean_major_events_per_future_year"]),
1539             }
1540         )
1541     comparison = pd.DataFrame(comparison_rows)
1542     comparison["recommendation_confidence"] = np.select(
1543         [
1544             comparison["p10_savings_percent"] > 0.0,
1545             comparison["probability_of_positive_saving"] >= 0.75,
1546         ],
1547         ["high", "medium"],
1548         default="low",
1549     )
1550     comparison["risk_note"] = np.where(
1551         comparison["recommendation_confidence"].eq("low"),
1552         "风险厌恶时建议暂维持现行策略并继续观测",
1553         "按期望年均成本执行优选策略",
1554     )
1555
1556     # 生成未来三年可执行日历。
1557     calendar_rows: list[pd.DataFrame] = []
1558     calendar_end = forecast_origin + pd.DateOffset(years=3)
1559     for asset in ASSETS:
1560         policy_id = str(selected.loc[selected["asset"].eq(asset), "policy_id"].iloc[0])
1561         frame = make_frame(asset, policy_id)
1562         prediction = point_policy_prediction(
1563             frame,
1564             main_model,

```

```

1558         response,
1559         rates_lookup[asset],
1560         forecast_origin,
1561     )
1562     frame = frame.copy()
1563     frame["expected_performance"] = prediction
1564     events = frame.loc[
1565         frame["maintenance_type"].notna()
1566         & (frame["date"] <= calendar_end),
1567         [
1568             "asset",
1569             "date",
1570             "maintenance_type",
1571             "days_since_maintenance",
1572             "expected_performance",
1573         ],
1574     ].copy()
1575     events["policy_id"] = policy_id
1576     calendar_rows.append(events)
1577     recommended_calendar = pd.concat(calendar_rows, ignore_index=True)
1578
1579     # 独立随机种子验证与参数情景。
1580     robustness_rows: list[dict[str, Any]] = []
1581     for asset in ASSETS:
1582         optimal_id = str(selected.loc[selected["asset"].eq(asset), "policy_id"].iloc[0])
1583         validation_distributions: dict[str, pd.DataFrame] = {}
1584         for policy_id in list(dict.fromkeys(["current", optimal_id])):
1585             summary, distribution = simulate_policy_asset(
1586                 spec_lookup[policy_id],
1587                 asset,
1588                 make_frame(asset, policy_id),
1589                 panel,
1590                 scenario_models,
1591                 response,
1592                 rates_lookup[asset],
1593                 forecast_origin,
1594                 paths=validation_paths,
1595                 seed=VALIDATION_SEED,
1596                 costs=costs,
1597                 evaluation_stage="validation",
1598             )
1599             validation_distributions[policy_id] = distribution
1600             robustness_rows.append(
1601                 {
1602                     "asset": asset,
1603                     "policy_id": policy_id,
1604                     "robustness_type": "independent_seed",
1605                     "scenario": f"seed {VALIDATION_SEED}",
1606                     "mean_annual_cost": summary["mean_annual_cost"],
1607                     "median_total_lifetime_years": summary["median_total_lifetime_years"],
1608                     "savings_percent_vs_current": np.nan,
1609                 }
1610             )
1611             current_validation = validation_distributions["current"]["annual_cost"].to_numpy()
1612             optimal_validation = validation_distributions[optimal_id]
1613             ["annual_cost"].to_numpy()
1614             saving_pct = (current_validation - optimal_validation) / current_validation
1615             * 100.0
1616             robustness_rows.append(
1617                 {
1618                     "asset": asset,
1619                     "policy_id": optimal_id,
1620                     "robustness_type": "independent_seed_saving",
1621                     "scenario": f"seed {VALIDATION_SEED}",
1622                     "mean_annual_cost": float(np.mean(optimal_validation)),
1623                     "median_total_lifetime_years": float(
1624                         validation_distributions[optimal_id]["total_lifetime_years"].median()
1625                     ),
1626                     "savings_percent_vs_current": float(np.mean(saving_pct)),
1627                 }
1628             )
1629         for damage_share in DAMAGE_SHARES:
1630             for response_scenario in RESPONSE_SCENARIOS:
1631                 scenario_costs: dict[str, float] = {}
1632                 for policy_id in list(dict.fromkeys(["current", optimal_id])):
1633                     point = evaluate_point_policy(
1634                         spec_lookup[policy_id],
1635                         asset,
1636                         make_frame(asset, policy_id),
1637                         panel,
1638                         main_model,
1639                         response,
1640                         rates_lookup[asset],
1641                         forecast_origin,
1642                         costs,
1643                         damage_share=damage_share,
1644                         response_scenario=response_scenario,
1645                     )
1646                     scenario_costs[policy_id] = float(point["mean_annual_cost"])
1647                 saving = (
1648                     scenario_costs["current"] - scenario_costs[optimal_id]
1649                 ) / scenario_costs["current"] * 100.0
1650                 robustness_rows.append(

```

```

1650         {
1651             "asset": asset,
1652             "policy_id": optimal_id,
1653             "robustness_type": "physical_scenario",
1654             "scenario": f"damage {damage_share:.2f} {response_scenario}",
1655             "mean_annual_cost": scenario_costs[optimal_id],
1656             "median_total_lifetime_years": np.nan,
1657             "savings_percent_vs_current": saving,
1658         }
1659     )
1660
1661     # 题目描述的现行工艺为每年 1-4 次大维护。将最大间隔
1662     # 放宽到 1.5/2 年，只作“非现行工艺”理论对照，不进入主方案。
1663     base_id = optimal_id if optimal_id != "current" else uniform_policy_id
1664     base_spec = spec_lookup[base_id]
1665     for relaxed_interval in (548, 730):
1666         relaxed_spec = replace(
1667             base_spec,
1668             policy_id=f"{base_id}_relaxed_b{relaxed_interval}",
1669             major_interval_days=relaxed_interval,
1670             description=base_spec.description + f"; 放宽大维护间隔至 {relaxed_interval} 天",
1671         )
1672         relaxed_frame = build_policy_frame(
1673             relaxed_spec,
1674             asset,
1675             forecast_origin,
1676             forecast_end,
1677             main_model.reference_date,
1678             maintenance,
1679             fixed_schedule,
1680             panel,
1681             main_model,
1682             response,
1683             rates_lookup[asset],
1684         )
1685         relaxed_point = evaluate_point_policy(
1686             relaxed_spec,
1687             asset,
1688             relaxed_frame,
1689             panel,
1690             main_model,
1691             response,
1692             rates_lookup[asset],
1693             forecast_origin,
1694             costs,
1695         )
1696         current_point = evaluate_point_policy(
1697             spec_lookup["current"],
1698             asset,
1699             make_frame(asset, "current"),
1700             panel,
1701             main_model,
1702             response,
1703             rates_lookup[asset],
1704             forecast_origin,
1705             costs,
1706         )
1707         robustness_rows.append(
1708             {
1709                 "asset": asset,
1710                 "policy_id": relaxed_spec.policy_id,
1711                 "robustness_type": "constraint_relaxation",
1712                 "scenario": f"major_interval {relaxed_interval} days",
1713                 "mean_annual_cost": relaxed_point["mean_annual_cost"],
1714                 "median_total_lifetime_years": relaxed_point[
1715                     "median_total_lifetime_years"
1716                 ],
1717                 "savings_percent_vs_current": (
1718                     current_point["mean_annual_cost"] - relaxed_point["mean_annual_cost"]
1719                     / current_point["mean_annual_cost"]
1720                     * 100.0
1721                 ),
1722             }
1723         )
1724     robustness = pd.DataFrame(robustness_rows)
1725
1726     policy_evaluations = pd.concat(
1727         [point_evaluations, screening_evaluations, final_evaluations],
1728         ignore_index=True,
1729     )
1730     output_tables = {
1731         "q3_policy_space.csv": policy_space_table(specs),
1732         "q3_maintenance_response.csv": response,
1733         "q3_response_validation.csv": response_validation,
1734         "q3_q2_baseline_check.csv": baseline_check,
1735         "q3_policy_evaluation.csv": policy_evaluations,
1736         "q3_optimal_policy_by_asset.csv": optimal_policy,
1737         "q3_current_vs_optimal.csv": comparison,
1738         "q3_recommended_calendar.csv": recommended_calendar,
1739         "q3_robustness.csv": robustness,
1740     }
1741     for name, table in output_tables.items():
1742         table.to_csv(RESULTS_DIR / name, index=False, date_format="%Y-%m-%d")
1743
1744     _configure_plot_style()

```

```

1745     figures = [
1746         plot_cost_frontier(point_evaluations, uniform_policy_id),
1747         plot_cost_comparison(comparison),
1748         plot_recommended_calendar(recommended_calendar),
1749         plot_savings_uncertainty(comparison),
1750     ]
1751
1752     current_fleet_cost = float(comparison["current_mean_annual_cost"].sum())
1753     optimal_fleet_cost = float(comparison["optimal_mean_annual_cost"].sum())
1754     uniform_final = final_evaluations.loc[
1755         final_evaluations["policy_id"].eq(uniform_policy_id)
1756     ]
1757     uniform_fleet_cost = float(uniform_final["mean_annual_cost"].sum())
1758     key_findings = {
1759         "analysis_version": "q3-formal-v1",
1760         "forecast_origin": str(pd.Timestamp(forecast_origin).date()),
1761         "candidate_policies": len(specs),
1762         "screening_paths": screening_paths,
1763         "final_paths": final_paths,
1764         "validation_paths": validation_paths,
1765         "objective": "expected pathwise lifecycle annual cost in 10k CNY per year",
1766         "current_fleet_annual_cost": current_fleet_cost,
1767         "optimized_fleet_annual_cost": optimal_fleet_cost,
1768         "fleet_savings_percent": (
1769             (current_fleet_cost - optimal_fleet_cost) / current_fleet_cost * 100.0
1770         ),
1771         "uniform_policy_id": uniform_policy_id,
1772         "uniform_fleet_annual_cost": uniform_fleet_cost,
1773         "uniform_gap_to_asset_specific_percent": (
1774             (uniform_fleet_cost - optimal_fleet_cost) / optimal_fleet_cost * 100.0
1775         ),
1776         "optimal_policy_by_asset": dict(
1777             zip(comparison["asset"], comparison["optimal_policy_id"])
1778         ),
1779         "mean_probability_of_positive_saving": float(
1780             comparison["probability_of_positive_saving"].mean()
1781         ),
1782         "low_confidence_assets": comparison.loc[
1783             comparison["recommendation_confidence"].eq("low"), "asset"
1784         ].tolist(),
1785         "maintenance_damage_share_main": MAIN_DAMAGE_SHARE,
1786         "maintenance_damage_share_sensitivity": list(DAMAGE_SHARES),
1787         "q2_bridge_mean_absolute_lifetime_difference_years": (
1788             float(baseline_check["absolute_difference_years"].mean())
1789             if len(baseline_check)
1790             else None
1791         ),
1792         "q2_bridge_max_absolute_lifetime_difference_years": (
1793             float(baseline_check["absolute_difference_years"].max())
1794             if len(baseline_check)
1795             else None
1796         ),
1797         "limitations": [
1798             "major maintenance has only 17 historical events and suffers indication bias",
1799             "maintenance damage share is scenario-identified rather than point-identified",
1800             "small maintenance is unrecorded and held fixed as a background process",
1801             "no downtime or parallel-capacity constraint was supplied by the problem",
1802             "state-triggered calendars use the estimated median state rather than future
1803             noise foresight",
1804         ],
1805         "figures": [path.name for path in figures],
1806     }
1807     with (RESULTS_DIR / "q3_key_findings.json").open("w", encoding="utf-8") as file:
1808         json.dump(key_findings, file, ensure_ascii=False, indent=2)
1809
1810     return {
1811         "policy_space": output_tables["q3_policy_space.csv"],
1812         "maintenance_response": response,
1813         "response_validation": response_validation,
1814         "baseline_check": baseline_check,
1815         "policy_evaluation": policy_evaluations,
1816         "optimal_policy": optimal_policy,
1817         "comparison": comparison,
1818         "recommended_calendar": recommended_calendar,
1819         "robustness": robustness,
1820         "figures": figures,
1821         "key_findings": key_findings,
1822     }
1823
1824 def optimize_maintenance_policy(*args: Any, **kwargs: Any) -> pd.DataFrame:
1825     """兼容早期接口：返回逐设备优选策略。"""
1826
1827     return run_q3_formal(*args, **kwargs)["optimal_policy"]
1828

```

附录 7：问题四完整源程序

src/filter_monitoring/q4_sensitivity.py

```
1  """问题四：价格波动下的策略稳定区、切换边界与鲁棒方案。
2
3  问题三已经把每条物理路径的寿命和中/大维护次数固定下来。价格变化
4  不会改变这些物理量，因此本模块先把路径级年均成本拆成购置、中维护、
5  大维护三个线性系数，再在同一批公共随机路径上精确重定价。这样既避免
6  为每个价格点重复拟合物理模型，也保证价格敏感性与问题三口径一致。
7  """
8
9  from __future__ import annotations
10
11 import json
12 import os
13 import re
14 import tempfile
15 from collections import Counter
16 from collections.abc import Iterable, Mapping, Sequence
17 from dataclasses import dataclass
18 from pathlib import Path
19 from typing import Any
20
21 # Matplotlib 缓存必须在导入 pyplot 前指向可写临时目录。
22 CACHE_ROOT = Path(tempfile.gettempdir()) / "math2026-filter-monitoring"
23 os.environ.setdefault("MPLCONFIGDIR", str(CACHE_ROOT / "matplotlib"))
24 os.environ.setdefault("XDG_CACHE_HOME", str(CACHE_ROOT / "xdg"))
25
26 import matplotlib.pyplot as plt
27 import numpy as np
28 import pandas as pd
29 from matplotlib.colors import ListedColormap
30 from scipy.optimize import Bounds, LinearConstraint, milp
31
32 from config import (
33     ASSETS,
34     BASE_COSTS,
35     FIGURES_DIR,
36     RESULTS_DIR,
37     CostConfig,
38     ensure_output_dirs,
39 )
40 from .q1_analysis import configure_plot_style, _save_figure
41 from .q2_lifetime import (
42     FORECAST_HORIZON_YEARS,
43     estimate_fixed_schedule,
44     fit_hierarchical_state_model,
45     residual_outlier_sensitivity,
46     structural_sensitivity_models,
47 )
48 from .q3_optimization import (
49     SEARCH_SEED,
50     PolicySpec,
51     _load_q3_inputs,
52     _select_with_tie_break,
53     build_policy_frame,
54     build_policy_space,
55     estimate_maintenance_response,
56     historical_maintenance_rates,
57     run_q3_formal,
58     simulate_policy_asset,
59 )
60
61 Q4_PATHS = 2000
62 Q4_SEED = SEARCH_SEED
63 NEAR_OPTIMAL_TOLERANCE = 0.01
64 COMMON_PRICE_RATIOS = tuple(np.round(np.arange(0.60, 1.4001, 0.05), 2))
65 SPLIT_MAINTENANCE_RATIOS = tuple(np.round(np.arange(0.50, 2.0001, 0.10), 2))
66 ONE_FACTOR_RATIOS = tuple(np.round(np.arange(0.25, 3.0001, 0.01), 2))
67
68
69 @dataclass(frozen=True)
70 class PriceScenario:
71     """一个可直接用于路径重定价的价格场景，单位为万元。"""
72
73     scenario_id: str
74     purchase_ratio: float
75     medium_ratio: float
76     major_ratio: float
77     purchase_cost: float
78     medium_cost: float
79     major_cost: float
80
81
82 def build_price_grid(
83     purchase_ratios: Iterable[float] = (0.8, 0.9, 1.0, 1.1, 1.2),
84     maintenance_ratios: Iterable[float] = (0.8, 0.9, 1.0, 1.1, 1.2),

```

```

86     base: CostConfig = BASE_COSTS,
87 ) -> pd.DataFrame:
88     """生成购置价与中/大维护共同倍率的二维网格。"""
89
90     rows: list[dict[str, Any]] = []
91     for purchase_ratio in purchase_ratios:
92         for maintenance_ratio in maintenance_ratios:
93             pr = float(purchase_ratio)
94             mr = float(maintenance_ratio)
95             rows.append(
96                 {
97                     "scenario_id": f"purchase_{pr:.2f}_maintenance_{mr:.2f}",
98                     "purchase_ratio": pr,
99                     "maintenance_ratio": mr,
100                     "medium_ratio": mr,
101                     "major_ratio": mr,
102                     "purchase_cost": base.purchase * pr,
103                     "medium_cost": base.medium * mr,
104                     "major_cost": base.major * mr,
105                 }
106             )
107     return pd.DataFrame(rows)
108
109 def build_split_maintenance_grid(
110     medium_ratios: Iterable[float] = SPLIT_MAINTENANCE_RATIOS,
111     major_ratios: Iterable[float] = SPLIT_MAINTENANCE_RATIOS,
112     base: CostConfig = BASE_COSTS,
113 ) -> pd.DataFrame:
114     """在购置价固定时分别扰动中维护与大维护价格。"""
115
116     rows: list[dict[str, Any]] = []
117     for medium_ratio in medium_ratios:
118         for major_ratio in major_ratios:
119             mr = float(medium_ratio)
120             br = float(major_ratio)
121             rows.append(
122                 {
123                     "scenario_id": f"medium_{mr:.2f}_major_{br:.2f}",
124                     "purchase_ratio": 1.0,
125                     "maintenance_ratio": np.nan,
126                     "medium_ratio": mr,
127                     "major_ratio": br,
128                     "purchase_cost": base.purchase,
129                     "medium_cost": base.medium * mr,
130                     "major_cost": base.major * br,
131                 }
132             )
133     return pd.DataFrame(rows)
134
135 def validate_coefficients(coefficients: pd.DataFrame) -> None:
136     required = {
137         "asset",
138         "policy_id",
139         "purchase annuity factor",
140         "medium annuity factor",
141         "major annuity factor",
142         "mean future medium events",
143         "mean future major events",
144         "median total lifetime years",
145     }
146     missing = required - set(coefficients.columns)
147     if missing:
148         raise ValueError(f"价格系数表缺少字段: {sorted(missing)}")
149     factors = coefficients[
150         [
151             "purchase annuity factor",
152             "medium annuity factor",
153             "major annuity factor",
154         ]
155     ]
156     if factors.isna().any().any() or (factors < 0.0).any().any():
157         raise ValueError("价格年化系数必须为非负有限数")
158
159 def reprice_policy_coefficients(
160     coefficients: pd.DataFrame,
161     purchase_cost: float,
162     medium_cost: float,
163     major_cost: float,
164 ) -> pd.DataFrame:
165     """不改变寿命和维护次数，只重算期望年均成本。"""
166
167     validate_coefficients(coefficients)
168     if min(purchase_cost, medium_cost, major_cost) < 0.0:
169         raise ValueError("价格不得为负")
170     out = coefficients.copy()
171     out["mean annual cost"] = (
172         float(purchase_cost) * out["purchase annuity factor"]
173         + float(medium_cost) * out["medium annuity factor"]
174         + float(major_cost) * out["major annuity factor"]
175     )
176     return out
177
178
179
180

```



```

181
182 def _plan_signature(rows: pd.DataFrame) -> str:
183     mapping = rows.set_index("asset")["policy_id"].astype(str).to_dict()
184     return ",".join(f"{asset}={mapping[asset]}" for asset in ASSETS)
185
186
187 def _mapping_signature(mapping: Mapping[str, str]) -> str:
188     return ",".join(f"{asset}={mapping[asset]}" for asset in ASSETS)
189
190
191 def _select_asset_rows(scored: pd.DataFrame) -> pd.DataFrame:
192     selected: list[pd.Series] = []
193     for asset in ASSETS:
194         block = scored.loc[scored["asset"].astype(str).eq(asset)]
195         if block.empty:
196             raise ValueError(f"价格候选中缺少设备 {asset}")
197         selected.append(_select_with_tie_break(block))
198     return pd.DataFrame(selected).reset_index(drop=True)
199
200
201 def _uniform_scores(scored: pd.DataFrame) -> pd.DataFrame:
202     eligible = scored.loc[~scored["policy_id"].eq("current")].copy()
203     counts = eligible.groupby("policy_id")["asset"].nunique()
204     complete = counts.loc[counts.eq(len(ASSETS))].index
205     eligible = eligible.loc[eligible["policy_id"].isin(complete)]
206     if eligible.empty:
207         raise ValueError("没有覆盖全部设备的统一候选策略")
208     return (
209         eligible.groupby(["policy_id", "policy_kind"], as_index=False)
210         .agg(
211             mean_annual_cost=("mean_annual_cost", "sum"),
212             mean_future_medium_events=("mean_future_medium_events", "sum"),
213             mean_future_major_events=("mean_future_major_events", "sum"),
214             median_total_lifetime_years=("median_total_lifetime_years", "mean"),
215             operational_feasible=("operational_feasible", "all"),
216         )
217     )
218
219
220 def _evaluate_price_grid(
221     coefficients: pd.DataFrame,
222     price_grid: pd.DataFrame,
223     baseline_plan: Mapping[str, str],
224     uniform_baseline_policy: str,
225 ) -> tuple[pd.DataFrame, pd.DataFrame]:
226     validate_coefficients(coefficients)
227     baseline_signature = _mapping_signature(baseline_plan)
228     summary_rows: list[dict[str, Any]] = []
229     selection_rows: list[pd.DataFrame] = []
230
231     for scenario in price_grid.to_dict(orient="records"):
232         scored = reprice_policy_coefficients(
233             coefficients,
234             float(scenario["purchase_cost"]),
235             float(scenario["medium_cost"]),
236             float(scenario["major_cost"]),
237         )
238         selected = _select_asset_rows(scored)
239         selected_signature = _plan_signature(selected)
240         theoretical_min = float(
241             scored.groupby("asset", sort=False)["mean_annual_cost"].min().sum()
242         )
243         selected_cost = float(selected["mean_annual_cost"].sum())
244         baseline_cost = 0.0
245         for asset in ASSETS:
246             policy_id = str(baseline_plan[asset])
247             row = scored.loc[
248                 scored["asset"].astype(str).eq(asset)
249                 & scored["policy_id"].astype(str).eq(policy_id)
250             ]
251             if len(row) != 1:
252                 raise ValueError(f"候选表缺少基准策略 {asset}/{policy_id}")
253             baseline_cost += float(row.iloc[0]["mean_annual_cost"])
254
255         uniform = _uniform_scores(scored)
256         uniform_minimum = float(uniform["mean_annual_cost"].min())
257         uniform_selected = _select_with_tie_break(uniform)
258         uniform_policy_id = str(uniform_selected["policy_id"])
259         uniform_selected_cost = float(uniform_selected["mean_annual_cost"])
260         uniform_baseline = uniform.loc[
261             uniform["policy_id"].astype(str).eq(uniform_baseline_policy)
262         ]
263         if len(uniform_baseline) != 1:
264             raise ValueError(f"缺少全厂基准策略 {uniform_baseline_policy}")
265         uniform_baseline_cost = float(uniform_baseline.iloc[0]["mean_annual_cost"])
266
267         row = dict(scenario)
268         row.update(
269             {
270                 "optimal_plan_signature": selected_signature,
271                 "q3_plan_signature": baseline_signature,
272                 "theoretical_min_fleet_cost": theoretical_min,
273                 "recommended_optimal_fleet_cost": selected_cost,
274                 "recommended_rule_regret_percent": (
275                     (selected_cost - theoretical_min) / theoretical_min * 100.0

```



```

276         ),
277         "q3_plan_fleet_cost": baseline_cost,
278         "q3_plan_regret_percent": (
279             (baseline_cost - theoretical_min) / theoretical_min * 100.0
280         ),
281         "q3_plan_exactly_selected": selected_signature == baseline_signature,
282         "q3_plan_within_1pct": baseline_cost
283         <= theoretical_min * (1.0 + NEAR_OPTIMAL_TOLERANCE),
284         "selected_mean_lifetime_years": float(
285             selected["median_total_lifetime_years"].mean()
286         ),
287         "selected_operational_feasible": bool(
288             selected["operational_feasible"].all()
289         ),
290         "q3_plan_operational_feasible": True,
291         "uniform_optimal_policy_id": uniform_policy_id,
292         "uniform_theoretical_min_cost": uniform_minimum,
293         "uniform_recommended_cost": uniform_selected_cost,
294         "uniform_q3_policy_id": uniform_baseline_policy,
295         "uniform_q3_cost": uniform_baseline_cost,
296         "uniform_q3_regret_percent": (
297             (uniform_baseline_cost - uniform_minimum)
298             / uniform_minimum
299             * 100.0
300         ),
301         "uniform_q3_exactly_selected": uniform_policy_id
302         == uniform_baseline_policy,
303         "uniform_q3_within_1pct": uniform_baseline_cost
304         <= uniform_minimum * (1.0 + NEAR_OPTIMAL_TOLERANCE),
305     }
306 )
307 summary_rows.append(row)
308 selected_out = selected[
309     [
310         "asset",
311         "policy_id",
312         "policy_kind",
313         "mean_annual_cost",
314         "median_total_lifetime_years",
315         "mean_future_medium_events",
316         "mean_future_major_events",
317         "operational_feasible",
318     ]
319 ].copy()
320 selected_out.insert(0, "scenario_id", str(scenario["scenario_id"]))
321 selection_rows.append(selected_out)
322
323 return pd.DataFrame(summary_rows), pd.concat(selection_rows, ignore_index=True)
324
325
326 def evaluate_optimal_policy_grid(
327     coefficients: pd.DataFrame,
328     price_grid: pd.DataFrame,
329     baseline_plan: Mapping[str, str],
330     uniform_baseline_policy: str,
331 ) -> pd.DataFrame:
332     """公开入口：返回逐价格场景重新选择后的全厂摘要。"""
333
334     summary, _ = evaluate_price_grid(
335         coefficients,
336         price_grid,
337         baseline_plan,
338         uniform_baseline_policy,
339     )
340     return summary
341
342
343 def contiguous_interval_containing_one(
344     factors: Sequence[float],
345     stable: Sequence[bool],
346 ) -> tuple[float, float]:
347     """返回包含倍率 1 的连续稳定区间；基准点不稳定则返回 NaN。"""
348
349     values = np.asarray(factors, dtype=float)
350     mask = np.asarray(stable, dtype=bool)
351     if len(values) != len(mask) or len(values) == 0:
352         raise ValueError("倍率和稳定标记必须等长且非空")
353     order = np.argsort(values)
354     values = values[order]
355     mask = mask[order]
356     center = int(np.argmin(np.abs(values - 1.0)))
357     if not np.isclose(values[center], 1.0, atol=1e-8) or not mask[center]:
358         return np.nan, np.nan
359     left = center
360     right = center
361     while left > 0 and mask[left - 1]:
362         left -= 1
363     while right + 1 < len(mask) and mask[right + 1]:
364         right += 1
365     return float(values[left]), float(values[right])
366
367
368 def point_cost_coefficients() -> pd.DataFrame:
369     evaluation_path = RESULTS_DIR / "q3_policy_evaluation.csv"
370     key_path = RESULTS_DIR / "q3_key_findings.json"

```

```

371 if not evaluation_path.is_file() or not key_path.is_file():
372     run_q3_formal()
373 evaluations = pd.read_csv(evaluation_path)
374 point = evaluations.loc[evaluations["evaluation_stage"].eq("point")].copy()
375 if len(point) != len(ASSETS) * 129:
376     raise ValueError("问题三点估计策略表不完整, 需先重跑 q3")
377 inputs = load_q3_inputs()
378 forecast_origin = pd.to_datetime(inputs["panel"]["date"]).max()
379 rates = historical_maintenance_rates(inputs["maintenance"], forecast_origin)
380 rates = rates[
381     [
382         "asset",
383         "historical_medium_events_equiv",
384         "historical_major_events_equiv",
385     ]
386 ]
387 out = point.merge(rates, on="asset", how="left", validate="many_to_one")
388 years = out["mean_total_lifetime_years"].to_numpy(dtype=float)
389 if np.any(years <= 0.0):
390     raise ValueError("问题三策略寿命必须为正")
391 out["purchase annuity factor"] = 1.0 / years
392 out["medium annuity factor"] = (
393     out["historical_medium_events_equiv"]
394     + out["mean_future_medium_events"]
395 ) / years
396 out["major annuity factor"] = (
397     out["historical_major_events_equiv"]
398     + out["mean_future_major_events"]
399 ) / years
400 repriced = reprice_policy_coefficients(
401     out, BASE_COSTS.purchase, BASE_COSTS.medium, BASE_COSTS.major
402 )
403 error = np.max(
404     np.abs(
405         repriced["mean_annual_cost"].to_numpy(dtype=float)
406         - point["mean_annual_cost"].to_numpy(dtype=float)
407     )
408 )
409 if error > 1e-8:
410     raise AssertionError(f"问题三基准成本分解误差过大: {error}")
411 out["operational_feasible"] = True
412 return out
413
414
415 def _one_factor_price_table(base: CostConfig = BASE_COSTS) -> pd.DataFrame:
416     rows: list[dict[str, Any]] = []
417     for factor_name in ("purchase", "maintenance", "medium", "major"):
418         for ratio in ONE_FACTOR_RATIOS:
419             pr = ratio if factor_name == "purchase" else 1.0
420             mr = ratio if factor_name in ("maintenance", "medium") else 1.0
421             br = ratio if factor_name in ("maintenance", "major") else 1.0
422             rows.append(
423                 {
424                     "scenario_id": f"factor_{factor_name}_{ratio:.2f}",
425                     "factor_name": factor_name,
426                     "factor_ratio": ratio,
427                     "purchase_ratio": pr,
428                     "maintenance_ratio": ratio
429                     if factor_name == "maintenance"
430                     else np.nan,
431                     "medium_ratio": mr,
432                     "major_ratio": br,
433                     "purchase_cost": base.purchase * pr,
434                     "medium_cost": base.medium * mr,
435                     "major_cost": base.major * br,
436                 }
437             )
438     return pd.DataFrame(rows)
439
440
441 def _screen_candidate_pairs(
442     point_coefficients: pd.DataFrame,
443     price_tables: Sequence[pd.DataFrame],
444     baseline_plan: Mapping[str, str],
445     uniform_baseline_policy: str,
446 ) -> set[tuple[str, str]]:
447     """用全部129组点估计保留每个价格点前两名, 再做路径复验。"""
448
449     pairs: set[tuple[str, str]] = {
450         (asset, str(baseline_plan[asset])) for asset in ASSETS
451     }
452     pairs.update((asset, "current") for asset in ASSETS)
453     uniform_candidates = {uniform_baseline_policy}
454
455     for table in price_tables:
456         for scenario in table.to_dict(orient="records"):
457             scored = reprice_policy_coefficients(
458                 point_coefficients,
459                 float(scenario["purchase_cost"]),
460                 float(scenario["medium_cost"]),
461                 float(scenario["major_cost"]),
462             )
463             for asset, block in scored.groupby("asset", sort=False):
464                 pairs.update(
465                     (str(asset), str(policy_id))

```

```

466         for policy_id in block.nsmallest(2, "mean_annual_cost")[
467             "policy_id"
468         ]
469     )
470     uniform = _uniform_scores(scored)
471     uniform_candidates.add(
472         str(uniform.nsmallest(1, "mean_annual_cost").iloc[0]["policy_id"])
473     )
474
475     pairs.update(
476         (asset, policy_id)
477         for policy_id in uniform_candidates
478         for asset in ASSETS
479     )
480     return pairs
481
482
483 def build_physical_context(
484     forecast_horizon_years: int = FORECAST_HORIZON_YEARS,
485 ) -> dict[str, Any]:
486     inputs = load_q3_inputs()
487     panel = inputs["panel"]
488     maintenance = inputs["maintenance"]
489     response = estimate_maintenance_response(inputs["effects"])
490     main_model = fit_hierarchical_state_model(
491         panel, reference_date=panel["date"].min()
492     )
493     outlier_path = RESULTS_DIR / "q1_residual_outliers.csv"
494     residual_outliers = (
495         pd.read_csv(outlier_path, parse_dates=["date"])
496         if outlier_path.is_file()
497         else pd.DataFrame(columns=["asset", "date", "is_residual_outlier"])
498     )
499     clean_model, _ = residual_outlier_sensitivity(
500         panel, main_model, residual_outliers
501     )
502     scenario_models, _ = structural_sensitivity_models(panel)
503     scenario_models = list(scenario_models) + [clean_model]
504     fixed_schedule = estimate_fixed_schedule(maintenance)
505     forecast_origin = pd.to_datetime(panel["date"]).max()
506     forecast_end = forecast_origin + pd.DateOffset(years=forecast_horizon_years)
507     rates = historical_maintenance_rates(maintenance, forecast_origin)
508     rates_lookup = rates.set_index("asset").to_dict(orient="index")
509     specs = build_policy_space()
510     spec_lookup = {spec.policy_id: spec for spec in specs}
511     frame_cache: dict[tuple[str, str], pd.DataFrame] = {}
512
513     def make_frame(asset: str, policy_id: str) -> pd.DataFrame:
514         key = (asset, policy_id)
515         if key not in frame_cache:
516             frame_cache[key] = build_policy_frame(
517                 spec_lookup[policy_id],
518                 asset,
519                 forecast_origin,
520                 forecast_end,
521                 main_model.reference_date,
522                 maintenance,
523                 fixed_schedule,
524                 panel,
525                 main_model,
526                 response,
527                 rates_lookup[asset],
528             )
529         return frame_cache[key]
530
531     return {
532         "panel": panel,
533         "maintenance": maintenance,
534         "response": response,
535         "main_model": main_model,
536         "scenario_models": scenario_models,
537         "forecast_origin": forecast_origin,
538         "rates": rates,
539         "rates_lookup": rates_lookup,
540         "spec_lookup": spec_lookup,
541         "make_frame": make_frame,
542     }
543
544
545 def simulate_price_coefficients(
546     candidate_pairs: set[tuple[str, str]],
547     *,
548     paths: int,
549     seed: int,
550 ) -> tuple[pd.DataFrame, dict[tuple[str, str], pd.DataFrame]]:
551     context = build_physical_context()
552     panel = context["panel"]
553     response = context["response"]
554     scenario_models = context["scenario_models"]
555     forecast_origin = context["forecast_origin"]
556     rates_lookup = context["rates_lookup"]
557     spec_lookup: Mapping[str, PolicySpec] = context["spec_lookup"]
558     make_frame = context["make_frame"]
559     distributions: dict[tuple[str, str], pd.DataFrame] = {}
560     rows: list[dict[str, Any]] = []

```

```

561 asset_order = {asset: index for index, asset in enumerate(ASSETS)}
562
563 for asset, policy_id in sorted(
564     candidate_pairs, key=lambda item: (asset_order[item[0]], item[1])
565 ):
566     spec = spec_lookup[policy_id]
567     summary, distribution = simulate_policy_asset(
568         spec,
569         asset,
570         make_frame(asset, policy_id),
571         panel,
572         scenario_models,
573         response,
574         rates_lookup[asset],
575         forecast_origin,
576         paths=paths,
577         seed=seed,
578         costs=BASE_COSTS,
579         evaluation_stage="q4_price_repricing",
580     )
581     years = distribution["total_lifetime_years"].to_numpy(dtype=float)
582     med_events = (
583         float(rates_lookup[asset]["historical_medium_events_equiv"])
584         + distribution["future_medium_events"].to_numpy(dtype=float)
585     )
586     major_events = (
587         float(rates_lookup[asset]["historical_major_events_equiv"])
588         + distribution["future_major_events"].to_numpy(dtype=float)
589     )
590     factor_distribution = pd.DataFrame(
591         {
592             "path_id": distribution["path_id"].to_numpy(dtype=int),
593             "purchase_annuity_factor": 1.0 / years,
594             "medium_annuity_factor": med_events / years,
595             "major_annuity_factor": major_events / years,
596             "total_lifetime_years": years,
597         }
598     )
599     distributions[(asset, policy_id)] = factor_distribution
600     row = dict(summary)
601     row.update(
602         {
603             "purchase_annuity_factor": float(
604                 factor_distribution["purchase_annuity_factor"].mean()
605             ),
606             "medium_annuity_factor": float(
607                 factor_distribution["medium_annuity_factor"].mean()
608             ),
609             "major_annuity_factor": float(
610                 factor_distribution["major_annuity_factor"].mean()
611             ),
612             "base_repriced_mean_annual_cost": float(
613                 BASE_COSTS.purchase
614                 * factor_distribution["purchase_annuity_factor"].mean()
615                 + BASE_COSTS.medium
616                 * factor_distribution["medium_annuity_factor"].mean()
617                 + BASE_COSTS.major
618                 * factor_distribution["major_annuity_factor"].mean()
619             ),
620             "operational_feasible": True,
621             "policy_description": spec.description,
622         }
623     )
624     rows.append(row)
625
626 coefficients = pd.DataFrame(rows)
627 max_error = float(
628     np.max(
629         np.abs(
630             coefficients["mean_annual_cost"]
631             - coefficients["base_repriced_mean_annual_cost"]
632         )
633     )
634 )
635 if max_error > 1e-8:
636     raise AssertionError(f"路径重定价未复现基准成本: {max_error}")
637 return coefficients, distributions
638
639
640 def factor_sensitivity(
641     coefficients: pd.DataFrame,
642     factors: pd.DataFrame,
643     baseline_plan: Mapping[str, str],
644     uniform_baseline_policy: str,
645 ) -> pd.DataFrame:
646     rows: list[pd.DataFrame] = []
647     for factor_name, table in factors.groupby("factor_name", sort=False):
648         summary, = evaluate_price_grid(
649             coefficients,
650             table,
651             baseline_plan,
652             uniform_baseline_policy,
653         )
654         summary["factor_name"] = factor_name
655         summary["factor_ratio"] = table["factor_ratio"].to_numpy(dtype=float)

```

```

656     rows.append(summary)
657     return pd.concat(rows, ignore_index=True)
658
659
660 def _switch_interval_table(one_factor: pd.DataFrame) -> pd.DataFrame:
661     rows: list[dict[str, Any]] = []
662     for factor_name, block in one_factor.groupby("factor_name", sort=False):
663         block = block.sort_values("factor_ratio")
664         factors = block["factor_ratio"].to_numpy(dtype=float)
665         for plan_scope, exact_col, near_col in (
666             (
667                 "asset_specific_q3_plan",
668                 "q3_plan_exactly_selected",
669                 "q3_plan_within_1pct",
670             ),
671             (
672                 "uniform_q3_plan",
673                 "uniform_q3_exactly_selected",
674                 "uniform_q3_within_1pct",
675             ),
676         ):
677             exact_low, exact_high = contiguous_interval_containing_one(
678                 factors, block[exact_col].to_numpy(dtype=bool)
679             )
680             near_low, near_high = contiguous_interval_containing_one(
681                 factors, block[near_col].to_numpy(dtype=bool)
682             )
683             rows.append(
684                 {
685                     "factor_name": factor_name,
686                     "plan_scope": plan_scope,
687                     "exact_optimal_low_ratio": exact_low,
688                     "exact_optimal_high_ratio": exact_high,
689                     "within_1pct_low_ratio": near_low,
690                     "within_1pct_high_ratio": near_high,
691                     "scan_low_ratio": float(factors.min()),
692                     "scan_high_ratio": float(factors.max()),
693                     "exact_low_truncated": bool(
694                         np.isfinite(exact_low) and exact_low == factors.min()
695                     ),
696                     "exact_high_truncated": bool(
697                         np.isfinite(exact_high) and exact_high == factors.max()
698                     ),
699                     "near_low_truncated": bool(
700                         np.isfinite(near_low) and near_low == factors.min()
701                     ),
702                     "near_high_truncated": bool(
703                         np.isfinite(near_high) and near_high == factors.max()
704                     ),
705                 }
706             )
707     return pd.DataFrame(rows)
708
709
710 def _minimax_asset_plan(
711     coefficients: pd.DataFrame,
712     price_grid: pd.DataFrame,
713 ) -> tuple[dict[str, str], float, float]:
714     """用混合整数线性规划求跨价格场景的全厂最小最大后悔方案。"""
715
716     candidate_rows = coefficients[["asset",
717 "policy_id"]].drop_duplicates().reset_index(drop=True)
718     scenario_costs: list[np.ndarray] = []
719     optimums: list[float] = []
720     for scenario in price_grid.to_dict(orient="records"):
721         scored = reprice_policy_coefficients(
722             coefficients,
723             float(scenario["purchase_cost"]),
724             float(scenario["medium_cost"]),
725             float(scenario["major_cost"]),
726         )
727         lookup = scored.set_index(["asset", "policy_id"])["mean_annual_cost"]
728         scenario_costs.append(
729             np.array([lookup.loc[(row.asset, row.policy_id)] for row in
730 candidate_rows.itertuples()],
731 dtype=float,
732 )
733 )
734     optimums.append(
735         float(scored.groupby("asset")["mean_annual_cost"].min().sum())
736     )
737     cost_matrix = np.vstack(scenario_costs)
738     optimum = np.asarray(optimums, dtype=float)
739     if not np.isfinite(cost_matrix).all() or not np.isfinite(optimum).all():
740         raise ValueError("鲁棒优化成本矩阵含非有限数")
741     n_candidates = len(candidate_rows)
742     n_variables = n_candidates + 1
743     objective = np.zeros(n_variables, dtype=float)
744     objective[-1] = 1.0
745     integrality = np.zeros(n_variables, dtype=int)
746     integrality[:n_candidates] = 1
747     lower_bounds = np.zeros(n_variables, dtype=float)
748     upper_bounds = np.ones(n_variables, dtype=float)

```

```

748 upper_bounds[-1] = np.inf
749
750 rows: list[np.ndarray] = []
751 lower: list[float] = []
752 upper: list[float] = []
753 for asset in ASSETS:
754     equation = np.zeros(n_variables, dtype=float)
755     asset_mask = candidate_rows["asset"].astype(str).eq(asset).to_numpy()
756     equation[:n_candidates][asset_mask] = 1.0
757     rows.append(equation)
758     lower.append(1.0)
759     upper.append(1.0)
760 for index, scenario_cost in enumerate(cost_matrix):
761     inequality = np.zeros(n_variables, dtype=float)
762     inequality[:n_candidates] = scenario_cost
763     inequality[-1] = -optimum[index]
764     rows.append(inequality)
765     lower.append(-np.inf)
766     upper.append(optimum[index])
767 result = milp(
768     c=objective,
769     integrality=integrality,
770     bounds=Bounds(lower_bounds, upper_bounds),
771     constraints=LinearConstraint(
772         np.vstack(rows), np.asarray(lower), np.asarray(upper)
773     ),
774     options={"time_limit": 60.0},
775 )
776 if not result.success or result.x is None:
777     raise RuntimeError(f"鲁棒方案混合整数规划失败: {result.message}")
778 chosen = candidate_rows.loc[result.x[:n_candidates] > 0.5]
779 mapping = dict(zip(chosen["asset"].astype(str), chosen["policy_id"].astype(str)))
780 if set(mapping) != set(ASSETS):
781     raise AssertionError("鲁棒方案没有为每台设备选择一个策略")
782 chosen_pairs = set(zip(chosen.asset, chosen.policy_id))
783 selected_indices = [
784     index
785     for index, row in enumerate(candidate_rows.itertuples())
786     if (row.asset, row.policy_id) in chosen_pairs
787 ]
788 plan_cost = cost_matrix[:, selected_indices].sum(axis=1)
789 regret = (plan_cost - optimum) / optimum * 100.0
790 return mapping, float(np.max(regret)), float(np.mean(regret))
791
792 def minimax_uniform_policy(
793     coefficients: pd.DataFrame,
794     price_grid: pd.DataFrame,
795 ) -> tuple[str, float, float]:
796     policy_ids = sorted(
797         policy_id
798         for policy_id, block in coefficients.groupby("policy_id")
799         if policy_id != "current" and block["asset"].nunique() == len(ASSETS)
800     )
801     regrets: dict[str, list[float]] = {policy_id: [] for policy_id in policy_ids}
802     for scenario in price_grid.to_dict(orient="records"):
803         scored = reprice_policy_coefficients(
804             coefficients,
805             float(scenario["purchase_cost"]),
806             float(scenario["medium_cost"]),
807             float(scenario["major_cost"]),
808         )
809         fleet = scored.loc[scored["policy_id"].isin(policy_ids)].groupby(
810             "policy_id"
811         )["mean_annual_cost"].sum()
812         minimum = float(fleet.min())
813         for policy_id in policy_ids:
814             regrets[policy_id].append(
815                 (float(fleet.loc[policy_id]) - minimum) / minimum * 100.0
816             )
817     table = pd.DataFrame(
818         {
819             "policy_id": policy_ids,
820             "max_regret": [max(regrets[p]) for p in policy_ids],
821             "mean_regret": [np.mean(regrets[p]) for p in policy_ids],
822         }
823     ).sort_values(["max_regret", "mean_regret", "policy_id"])
824     row = table.iloc[0]
825     return str(row["policy_id"]), float(row["max_regret"]), float(row["mean_regret"])
826
827 def short_policy(policy_id: str) -> str:
828     if policy_id == "current":
829         return "现行"
830     match = re.fullmatch(r"periodic_m(\d+)_b(\d+)", policy_id)
831     if match:
832         medium, major = map(int, match.groups())
833         if medium >= major == 365:
834             return "年1次"
835         return f"中{medium}日/大{major}日"
836     match = re.fullmatch(r"trigger_t(\d+)_c(\d+)_b(\d+)", policy_id)
837     if match:
838         trigger, cooldown, major = match.groups()
839         return f"触发{trigger}/冷却{cooldown}/大{major}"

```



```

842     return policy_id
843
844
845 def _describe_signature(signature: str) -> str:
846     mapping = dict(part.split("=", 1) for part in signature.split(";"))
847     groups: dict[str, list[str]] = {}
848     for asset in ASSETS:
849         groups.setdefault(mapping[asset], []).append(asset)
850     parts = []
851     for policy_id, assets in sorted(groups.items(), key=lambda item: item[0]):
852         parts.append(f"{'.'.join(assets)}: {_short_policy(policy_id)}")
853     return "; ".join(parts)
854
855
856 def _attach_plan_codes(
857     common: pd.DataFrame,
858     split: pd.DataFrame,
859     one_factor: pd.DataFrame,
860     baseline_signature: str,
861 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
862     signatures = list(common["optimal_plan_signature"])
863     signatures += list(split["optimal_plan_signature"])
864     signatures += list(one_factor["optimal_plan_signature"])
865     counts = Counter(signatures)
866     ordered = [baseline_signature]
867     ordered.extend(
868         signature
869         for signature, _ in counts.most_common()
870         if signature != baseline_signature
871     )
872     code_map = {signature: f"P{index}" for index, signature in enumerate(ordered)}
873     for table in (common, split, one_factor):
874         table["optimal_plan_code"] = table["optimal_plan_signature"].map(code_map)
875     catalog = pd.DataFrame(
876         [
877             {
878                 "plan_code": code_map[signature],
879                 "plan_signature": signature,
880                 "plan_description": _describe_signature(signature),
881                 "scenario_count": counts.get(signature, 0),
882                 "is_q3_baseline_plan": signature == baseline_signature,
883             }
884             for signature in ordered
885         ]
886     )
887     return common, split, one_factor, catalog
888
889
890 def _plan_cost_distribution(
891     plan: Mapping[str, str],
892     factor_distributions: Mapping[tuple[str, str], pd.DataFrame],
893     purchase_cost: float,
894     medium_cost: float,
895     major_cost: float,
896 ) -> np.ndarray:
897     total: np.ndarray | None = None
898     for asset in ASSETS:
899         factors = factor_distributions[(asset, str(plan[asset]))]
900         cost = (
901             purchase_cost * factors["purchase annuity factor"].to_numpy(dtype=float)
902             + medium_cost * factors["medium annuity factor"].to_numpy(dtype=float)
903             + major_cost * factors["major annuity factor"].to_numpy(dtype=float)
904         )
905         total = cost.copy() if total is None else total + cost
906     if total is None:
907         raise ValueError("空维护方案无法计算成本分布")
908     return total
909
910
911 def _named_scenario_uncertainty(
912     coefficients: pd.DataFrame,
913     factor_distributions: Mapping[tuple[str, str], pd.DataFrame],
914     baseline_plan: Mapping[str, str],
915     robust_plan: Mapping[str, str],
916     uniform_baseline_policy: str,
917     robust_uniform_policy: str,
918 ) -> pd.DataFrame:
919     scenarios = (
920         ("baseline", 1.0, 1.0, 1.0),
921         ("purchase_down_40pct", 0.6, 1.0, 1.0),
922         ("purchase_up_40pct", 1.4, 1.0, 1.0),
923         ("maintenance_down_40pct", 1.0, 0.6, 0.6),
924         ("maintenance_up_40pct", 1.0, 1.4, 1.4),
925         ("purchase_up_maintenance_down", 1.4, 0.6, 0.6),
926         ("purchase_down_maintenance_up", 0.6, 1.4, 1.4),
927         ("medium_up_major_down", 1.0, 2.0, 0.5),
928         ("medium_down_major_up", 1.0, 0.5, 2.0),
929     )
930     rows: list[dict[str, Any]] = []
931     for name, purchase_ratio, medium_ratio, major_ratio in scenarios:
932         purchase_cost = BASE_COSTS.purchase * purchase_ratio
933         medium_cost = BASE_COSTS.medium * medium_ratio
934         major_cost = BASE_COSTS.major * major_ratio
935         scored = reprice_policy_coefficients(
936             coefficients, purchase_cost, medium_cost, major_cost

```



```

937     )
938     optimal_rows = select_asset_rows(scored)
939     optimal_plan = dict(
940         zip(optimal_rows["asset"].astype(str), optimal_rows["policy_id"].astype(str))
941     )
942     plans = {
943         "q3_asset_specific": dict(baseline_plan),
944         "reoptimized_asset_specific": optimal_plan,
945         "minimax_asset_specific": dict(robust_plan),
946         "q3_uniform": {asset: uniform_baseline_policy for asset in ASSETS},
947         "minimax_uniform": {asset: robust_uniform_policy for asset in ASSETS},
948     }
949     theoretical_min = float(
950         scored.groupby("asset")["mean_annual_cost"].min().sum()
951     )
952     for plan_type, plan in plans.items():
953         distribution = _plan_cost_distribution(
954             plan,
955             factor_distributions,
956             purchase_cost,
957             medium_cost,
958             major_cost,
959         )
960         mean_cost = float(np.mean(distribution))
961         rows.append(
962             {
963                 "scenario": name,
964                 "purchase_ratio": purchase_ratio,
965                 "medium_ratio": medium_ratio,
966                 "major_ratio": major_ratio,
967                 "plan_type": plan_type,
968                 "plan_signature": mapping_signature(plan),
969                 "mean_fleet_annual_cost": mean_cost,
970                 "p10_fleet_annual_cost": float(np.quantile(distribution, 0.10)),
971                 "p90_fleet_annual_cost": float(np.quantile(distribution, 0.90)),
972                 "regret_percent_vs_theoretical_min": (
973                     (mean_cost - theoretical_min) / theoretical_min * 100.0
974                 ),
975                 "operational_feasible": True,
976             }
977         )
978     return pd.DataFrame(rows)
979
980
981 def plot_price_stability_map(
982     price_grid: pd.DataFrame,
983     catalog: pd.DataFrame,
984 ) -> Path:
985     codes = catalog["plan_code"].tolist()
986     code_to_index = {code: index for index, code in enumerate(codes)}
987     work = price_grid.copy()
988     used_codes = [code for code in codes if code in set(work["optimal_plan_code"])]
989     local_index = {code: index for index, code in enumerate(used_codes)}
990     work["code_index"] = work["optimal_plan_code"].map(local_index)
991     pivot = work.pivot(
992         index="maintenance_ratio", columns="purchase_ratio", values="code_index"
993     ).sort_index(ascending=True)
994     colors = plt.get_cmap("tab20")(np.linspace(0.0, 1.0, max(len(codes), 2)))
995     fig, ax = plt.subplots(figsize=(10.5, 7.0))
996     used_colors = [colors[code_to_index[code]] for code in used_codes]
997     image = ax.imshow(
998         pivot.to_numpy(dtype=float),
999         origin="Lower",
1000         aspect="auto",
1001         extent=[
1002             float(pivot.columns.min()),
1003             float(pivot.columns.max()),
1004             float(pivot.index.min()),
1005             float(pivot.index.max()),
1006         ],
1007         interpolation="nearest",
1008         cmap=ListedColormap(used_colors),
1009         vmin=-0.5,
1010         vmax=len(used_codes) - 0.5,
1011     )
1012     colorbar = fig.colorbar(
1013         image,
1014         ax=ax,
1015         ticks=range(len(used_codes)),
1016         fraction=0.055,
1017         pad=0.03,
1018     )
1019     colorbar.ax.set_yticklabels(used_codes)
1020     colorbar.set_label("方案编号")
1021     ax.scatter([1.0], [1.0], marker="*", s=115, c="white", edgecolors="black", zorder=3)
1022     ax.annotate("基准", (1.0, 1.0), xytext=(6, 5), textcoords="offset points")
1023     ax.set_xlabel("购置价格倍率")
1024     ax.set_ylabel("中/大维护共同价格倍率")
1025     ax.set_title("价格波动下逐设备优选方案稳定区域")
1026     fig.tight_layout()
1027     return _save_figure(fig, "q4_01_price_stability_map.png")
1028
1029
1030 def plot_q3_plan_regret(price_grid: pd.DataFrame) -> Path:

```

```

1031 pivot = price_grid.pivot(
1032     index="maintenance_ratio",
1033     columns="purchase_ratio",
1034     values="q3_plan_regret_percent",
1035 ).sort_index(ascending=True)
1036 x = pivot.columns.to_numpy(dtype=float)
1037 y = pivot.index.to_numpy(dtype=float)
1038 z = pivot.to_numpy(dtype=float)
1039 fig, ax = plt.subplots(figsize=(9.0, 6.5))
1040 mesh = ax.pcolormesh(x, y, z, shading="auto", cmap="YlOrRd")
1041 if float(np.nanmin(z)) <= 1.0 <= float(np.nanmax(z)):
1042     contour = ax.contour(x, y, z, levels=[1.0], colors="#2457a7", linewidths=1.8)
1043     ax.clabel(contour, fmt="{1.0: '1%近优边界'", inline=True)
1044 colorbar = fig.colorbar(mesh, ax=ax)
1045 colorbar.set_label("问题三逐设备方案后悔值 (%) ")
1046 ax.set_xlabel("购置价格倍率")
1047 ax.set_ylabel("中/大维护共同价格倍率")
1048 ax.set_title("问题三方案的价格后悔值")
1049 fig.tight_layout()
1050 return _save_figure(fig, "q4_02_q3_plan_regret.png")
1051
1052
1053 def plot_split_maintenance_map(
1054     split_grid: pd.DataFrame,
1055     catalog: pd.DataFrame,
1056 ) -> Path:
1057     codes = catalog["plan_code"].tolist()
1058     code_to_index = {code: index for index, code in enumerate(codes)}
1059     work = split_grid.copy()
1060     used_codes = [code for code in codes if code in set(work["optimal_plan_code"])]
1061     local_index = {code: index for index, code in enumerate(used_codes)}
1062     work["code_index"] = work["optimal_plan_code"].map(local_index)
1063     pivot = work.pivot(
1064         index="major_ratio", columns="medium_ratio", values="code_index"
1065     ).sort_index(ascending=True)
1066     colors = plt.get_cmap("tab20")(np.linspace(0.0, 1.0, max(len(codes), 2)))
1067     fig, ax = plt.subplots(figsize=(10.5, 7.0))
1068     used_colors = [colors[code_to_index[code]] for code in used_codes]
1069     image = ax.imshow(
1070         pivot.to_numpy(dtype=float),
1071         origin="Lower",
1072         aspect="auto",
1073         extent=[
1074             float(pivot.columns.min()),
1075             float(pivot.columns.max()),
1076             float(pivot.index.min()),
1077             float(pivot.index.max()),
1078         ],
1079         interpolation="nearest",
1080         cmap=ListedColormap(used_colors),
1081         vmin=-0.5,
1082         vmax=len(used_codes) - 0.5,
1083     )
1084     colorbar = fig.colorbar(
1085         image,
1086         ax=ax,
1087         ticks=range(len(used_codes)),
1088         fraction=0.055,
1089         pad=0.03,
1090     )
1091     colorbar.ax.set_yticklabels(used_codes)
1092     colorbar.set_label("方案编号")
1093     ax.scatter([1.0], [1.0], marker="*", s=115, c="white", edgecolors="black", zorder=3)
1094     ax.annotate("基准", (1.0, 1.0), xytext=(6, 5), textcoords="offset points")
1095     ax.set_xlabel("中维护价格倍率")
1096     ax.set_ylabel("大维护价格倍率")
1097     ax.set_title("中、大维护价格分别波动时的方案切换")
1098     fig.tight_layout()
1099     return _save_figure(fig, "q4_03_split_maintenance_map.png")
1100
1101
1102 def plot_one_factor_regret(one_factor: pd.DataFrame) -> Path:
1103     labels = {
1104         "purchase": "购置价",
1105         "maintenance": "全部维护价",
1106         "medium": "中维护价",
1107         "major": "大维护价",
1108     }
1109     fig, axes = plt.subplots(2, 2, figsize=(10.5, 7.2), sharey=False)
1110     for ax, factor_name in zip(axes.flat, labels):
1111         block = one_factor.loc[one_factor["factor_name"].eq(factor_name)].sort_values(
1112             "factor_ratio"
1113         )
1114         ax.plot(
1115             block["factor_ratio"],
1116             block["q3_plan_regret_percent"],
1117             label="逐设备方案",
1118             color="#2457a7",
1119         )
1120         ax.plot(
1121             block["factor_ratio"],
1122             block["uniform_q3_regret_percent"],
1123             label="全厂统一方案",

```

```

1124         color="#d95f02",
1125     )
1126     ax.axhline(1.0, color="#777777", ls="--", lw=1.0)
1127     ax.axvline(1.0, color="#999999", ls=":", lw=1.0)
1128     ax.set_title(labels[factor_name])
1129     ax.set_xlabel("价格倍率")
1130     ax.set_ylabel("后悔值 (%)")
1131     ax.grid(alpha=0.2)
1132     axes[0, 0].legend(frameon=False)
1133     fig.suptitle("单项价格波动下问题三方案的经济稳健性")
1134     fig.tight_layout()
1135     return _save_figure(fig, "q4_04_one_factor_regret.png")
1136
1137
1138 def run_q4_formal(
1139     *
1140     simulation_paths: int = Q4_PATHS,
1141     seed: int = Q4_SEED,
1142 ) -> dict[str, Any]:
1143     """运行正式价格敏感性、边界和鲁棒后悔分析。"""
1144
1145     if simulation_paths <= 0:
1146         raise ValueError("问题四仿真路径数必须为正")
1147     ensure_output_dirs()
1148     q3_key_path = RESULTS_DIR / "q3_key_findings.json"
1149     if not q3_key_path.is_file():
1150         run_q3_formal()
1151     with q3_key_path.open(encoding="utf-8") as file:
1152         q3_key = json.load(file)
1153     baseline_plan = {
1154         str(asset): str(policy_id)
1155         for asset, policy_id in q3_key["optimal_policy_by_asset"].items()
1156     }
1157     uniform_baseline_policy = str(q3_key["uniform_policy_id"])
1158     baseline_signature = _mapping_signature(baseline_plan)
1159
1160     point_coefficients = _point_cost_coefficients()
1161     common_grid = _build_price_grid(COMMON_PRICE_RATIOS, COMMON_PRICE_RATIOS)
1162     split_grid = _build_split_maintenance_grid()
1163     one_factor_prices = _one_factor_price_table()
1164     candidate_pairs = _screen_candidate_pairs(
1165         point_coefficients,
1166         (common_grid, split_grid, one_factor_prices),
1167         baseline_plan,
1168         uniform_baseline_policy,
1169     )
1170     coefficients, factor_distributions = _simulate_price_coefficients(
1171         candidate_pairs,
1172         paths=simulation_paths,
1173         seed=seed,
1174     )
1175
1176     common_summary, common_asset = _evaluate_price_grid(
1177         coefficients, common_grid, baseline_plan, uniform_baseline_policy
1178     )
1179     split_summary, split_asset = _evaluate_price_grid(
1180         coefficients, split_grid, baseline_plan, uniform_baseline_policy
1181     )
1182     one_factor = _factor_sensitivity(
1183         coefficients, one_factor_prices, baseline_plan, uniform_baseline_policy
1184     )
1185     common_summary, split_summary, one_factor, catalog = _attach_plan_codes(
1186         common_summary,
1187         split_summary,
1188         one_factor,
1189         baseline_signature,
1190     )
1191     intervals = _switch_interval_table(one_factor)
1192
1193     robust_plan, robust_max_regret, robust_mean_regret = _minimax_asset_plan(
1194         coefficients, common_grid
1195     )
1196     robust_uniform, robust_uniform_max, robust_uniform_mean = _minimax_uniform_policy(
1197         coefficients, common_grid
1198     )
1199     robust_rows = [
1200         {
1201             "plan_type": "asset_specific_minimax",
1202             "asset": asset,
1203             "policy_id": robust_plan[asset],
1204             "max_regret_percent": robust_max_regret,
1205             "mean_regret_percent": robust_mean_regret,
1206         }
1207         for asset in ASSETS
1208     ]
1209     robust_rows.append(
1210         {
1211             "plan_type": "uniform_minimax",
1212             "asset": "ALL",
1213             "policy_id": robust_uniform,
1214             "max_regret_percent": robust_uniform_max,
1215             "mean_regret_percent": robust_uniform_mean,
1216         }
1217     )
1218     robust_table = pd.DataFrame(robust_rows)

```

```

1219 named_uncertainty = _named_scenario_uncertainty(
1220     coefficients,
1221     factor_distributions,
1222     baseline_plan,
1223     robust_plan,
1224     uniform_baseline_policy,
1225     robust_uniform,
1226 )
1227
1228 output_tables = {
1229     "q4_policy_coefficients.csv": coefficients,
1230     "q4_price_grid.csv": common_summary,
1231     "q4_asset_selection.csv": common_asset,
1232     "q4_split_maintenance_grid.csv": split_summary,
1233     "q4_split_asset_selection.csv": split_asset,
1234     "q4_plan_catalog.csv": catalog,
1235     "q4_one_factor_sensitivity.csv": one_factor,
1236     "q4_switch_intervals.csv": intervals,
1237     "q4_robust_policy.csv": robust_table,
1238     "q4_scenario_uncertainty.csv": named_uncertainty,
1239 }
1240 for name, table in output_tables.items():
1241     table.to_csv(RESULTS_DIR / name, index=False)
1242
1243 configure_plot_style()
1244 figures = [
1245     plot_price_stability_map(common_summary, catalog),
1246     plot_q3_plan_regret(common_summary),
1247     plot_split_maintenance_map(split_summary, catalog),
1248     plot_one_factor_regret(one_factor),
1249 ]
1250
1251 interval_lookup = intervals.set_index(["factor_name", "plan_scope"])
1252 asset_intervals = {
1253     factor: {
1254         "exact_low": float(
1255             interval_lookup.loc[
1256                 (factor, "asset_specific_q3_plan"),
1257                 "exact_optimal_low_ratio",
1258             ]
1259         ),
1260         "exact_high": float(
1261             interval_lookup.loc[
1262                 (factor, "asset_specific_q3_plan"),
1263                 "exact_optimal_high_ratio",
1264             ]
1265         ),
1266         "near_low": float(
1267             interval_lookup.loc[
1268                 (factor, "asset_specific_q3_plan"),
1269                 "within_1pct_low_ratio",
1270             ]
1271         ),
1272         "near_high": float(
1273             interval_lookup.loc[
1274                 (factor, "asset_specific_q3_plan"),
1275                 "within_1pct_high_ratio",
1276             ]
1277         ),
1278     }
1279     for factor in ("purchase", "maintenance", "medium", "major")
1280 }
1281 uniform_intervals = {
1282     factor: {
1283         "exact_low": float(
1284             interval_lookup.loc[
1285                 (factor, "uniform_q3_plan"), "exact_optimal_low_ratio"
1286             ]
1287         ),
1288         "exact_high": float(
1289             interval_lookup.loc[
1290                 (factor, "uniform_q3_plan"), "exact_optimal_high_ratio"
1291             ]
1292         ),
1293         "near_low": float(
1294             interval_lookup.loc[
1295                 (factor, "uniform_q3_plan"), "within_1pct_low_ratio"
1296             ]
1297         ),
1298         "near_high": float(
1299             interval_lookup.loc[
1300                 (factor, "uniform_q3_plan"), "within_1pct_high_ratio"
1301             ]
1302         ),
1303     }
1304     for factor in ("purchase", "maintenance", "medium", "major")
1305 }
1306 key_findings = {
1307     "analysis_version": "q4-formal-v1",
1308     "simulation_paths_per_asset_policy": simulation_paths,
1309     "seed": seed,
1310     "point_policy_space": int(point_coefficients["policy_id"].nunique()),
1311     "monte_carlo_candidate_pairs": len(candidate_pairs),
1312     "common_price_grid_scenarios": len(common_summary),
1313     "common_price_ratio_range": [

```

```

1314         min(COMMON_PRICE_RATIOS),
1315         max(COMMON_PRICE_RATIOS),
1316     ],
1317     "split_maintenance_grid_scenarios": len(split_summary),
1318     "split_maintenance_ratio_range": [
1319         min(SPLIT_MAINTENANCE_RATIOS),
1320         max(SPLIT_MAINTENANCE_RATIOS),
1321     ],
1322     "one_factor_ratio_range": [
1323         min(ONE_FACTOR_RATIOS),
1324         max(ONE_FACTOR_RATIOS),
1325     ],
1326     "q3_asset_plan_exact_share_common_grid": float(
1327         common_summary["q3_plan_exactly_selected"].mean()
1328     ),
1329     "q3_asset_plan_within_1pct_share_common_grid": float(
1330         common_summary["q3_plan_within_1pct"].mean()
1331     ),
1332     "q3_uniform_plan_exact_share_common_grid": float(
1333         common_summary["uniform_q3_exactly_selected"].mean()
1334     ),
1335     "q3_uniform_plan_within_1pct_share_common_grid": float(
1336         common_summary["uniform_q3_within_1pct"].mean()
1337     ),
1338     "q3_plan_max_regret_common_grid_percent": float(
1339         common_summary["q3_plan_regret_percent"].max()
1340     ),
1341     "q3_uniform_max_regret_common_grid_percent": float(
1342         common_summary["uniform_q3_regret_percent"].max()
1343     ),
1344     "asset_specific_price_intervals": asset_intervals,
1345     "uniform_price_intervals": uniform_intervals,
1346     "asset_specific_minimax_plan": robust_plan,
1347     "asset_specific_minimax_max_regret_percent": robust_max_regret,
1348     "asset_specific_minimax_mean_regret_percent": robust_mean_regret,
1349     "uniform_minimax_policy_id": robust_uniform,
1350     "uniform_minimax_max_regret_percent": robust_uniform_max,
1351     "uniform_minimax_mean_regret_percent": robust_uniform_mean,
1352     "operational_feasibility": (
1353         "价格不进入状态转移, 问题三方案寿命、维护次数和1-4次大维护约束均不变"
1354     ),
1355     "limitations": [
1356         "价格网格未包含停机损失和并联产能约束, 题目未提供这些成本",
1357         "价格边界基于问题三候选规则集, 不外推到完全不同的维护技术",
1358         "物理参数不确定性由问题三模型混合路径传播, 未假设价格与退化参数相关",
1359     ],
1360     "figures": [path.name for path in figures],
1361 }
1362 with (RESULTS_DIR / "q4_key_findings.json").open("w", encoding="utf-8") as file:
1363     json.dump(key_findings, file, ensure_ascii=False, indent=2)
1364
1365 return {
1366     "policy_coefficients": coefficients,
1367     "price_grid": common_summary,
1368     "asset_selection": common_asset,
1369     "split_maintenance_grid": split_summary,
1370     "split_asset_selection": split_asset,
1371     "plan_catalog": catalog,
1372     "one_factor_sensitivity": one_factor,
1373     "switch_intervals": intervals,
1374     "robust_policy": robust_table,
1375     "scenario_uncertainty": named_uncertainty,
1376     "figures": figures,
1377     "key_findings": key_findings,
1378 }
1379

```